

Elementary Number Theory

Elementary Number Theory

Author

Muho Choi

Email: sihyeon8240@gmail.com

GitHub: <https://github.com/sihyeon8240>

Source Code & Contributions

GitHub: <https://github.com/sihyeon8240/math>

Feel free to open issues or pull requests for corrections.

© 2026. Muho Choi.

This textbook is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/) (CC BY 4.0).

You are free to: Share, copy, and redistribute the material in any medium or format; adapt, remix, transform, and build upon the material for any purpose, even commercially.

Under the following terms: You must give appropriate credit, provide a link to the license, and indicate if changes were made.

Contents

Preface	iv
1 Divisibility Theory in the Integers	1
1.1 The Division Algorithm	1
1.2 The Greatest Common Divisor	4
1.3 The Euclidean Algorithm	15
1.4 The Diophantine Equation $ax+by=c$	23
2 Primes and Their Distribution	27
2.1 The Fundamental Theorem of Arithmetic	27
2.2 The Sieve of Eratosthenes	37
2.3 The Goldbach Conjecture	43
3 The Theory of Congruences	51
3.1 Basic Properties of Congruence	51
3.2 Binary and Decimal Representations of Integers	58
3.3 Linear Congruences and the Chinese Remainder Theorem	64
4 Fermat's Theorem	76
4.1 Fermat's Little Theorem and Pseudoprimes	76
4.2 Wilson's Theorem	83
4.3 The Fermat-Kraitchik Factorization Method	91
5 Number-Theoretic Functions	92
5.1 The Sum and Number of Divisors	92
5.2 The Möbius Inversion Formula	106
5.3 The Greatest Integer Function	113
5.4 An Application to the Calendar	122
6 Euler's Generalization of Fermat's Theorem	123
6.1 Euler's Phi-Function	123
6.2 Euler's Theorem	123
6.3 Some Properties of the Phi-Function	123

7	Primitive Roots and Indices	124
7.1	The Order of an Integer Modulo n	124
7.2	Primitive Roots for Primes	124
7.3	Composite Numbers Having Primitive Roots	124
7.4	The Theory of Indices	124
8	The Quadratic Reciprocity Law	125
8.1	Euler's Criterion	125
8.2	The Legendre Symbol and Its Properties	125
8.3	Quadratic Reciprocity	125
8.4	Quadratic Congruences with Composite Moduli	125
9	Introduction to Cryptography	126
9.1	From Caesar Cipher to Public Key Cryptography	126
9.2	The Knapsack Cryptosystem	126
9.3	An Application of Primitive Roots to Cryptography	126
10	Numbers of Special Form	127
10.1	Perfect Numbers	127
10.2	Mersenne Primes and Amicable Numbers	127
10.3	Fermat Numbers	127
11	Certain Nonlinear Diophantine Equations	128
11.1	The Equation $x^2 + y^2 = z^2$	128
11.2	Fermat's Last Theorem	128
12	Representation of Integers as Sums of Squares	129
12.1	Sums of Two Squares	129
12.2	Sums of More Than Two Squares	129
13	Fibonacci Numbers	130
13.1	The Fibonacci Sequence	130
13.2	Certain Identities Involving Fibonacci Numbers	130
14	Continued Fractions	131
14.1	Finite Continued Fractions	131
14.2	Infinite Continued Fractions	131
14.3	Farey Fractions	131
14.4	Pell's Equation	131
15	Some Modern Developments	132
15.1	Primality Testing and Factorization	132
15.2	An Application to Factoring: Remote Coin Flipping	132
15.3	The Prime Number Theorem and Zeta Function	132

Preface

This textbook uses Lean 4 to formalize selected mathematical statements and proofs.

- **Tools.** Lean is an interactive theorem prover; Mathlib is a library of formalized mathematics. This edition uses Lean 4.34.0 and Mathlib v4.34.0 (revision [5ed296525643](#)).
- **Reading the proofs.** Each paired statement or proof step gives a mathematical explanation followed by its Lean code. The explanations can be read independently; code fragments are read in order within the complete declaration.
- **Verification.** Lean checks formal proofs under their stated assumptions. Verification covers only statements linked to checked declarations. Agreement between the explanations, displayed code, and checked proofs is reviewed separately.

Chapter 1

Divisibility Theory in the Integers

1.1 The Division Algorithm

Theorem 1.1.1. *Let $a \in \mathbb{Z}$ and $b \in \mathbb{N}^+$. There is a unique pair $(q, r) \in \mathbb{Z} \times \mathbb{N}$ such that*

$$a = qb + r, \quad r < b.$$

*We call q the **quotient** and r the **remainder**.*

```
theorem existsUnique_quotient_remainder (a : ℤ) (b : ℕ+) :  
  ∃! qr : ℤ × ℕ,  
    a = qr.1 * b + qr.2 ∧ qr.2 < b
```

Proof. Since $b \geq 1$,

$$a - (-|a|)b = a + |a|b \geq a + |a| \geq 0.$$

```
classical  
  
have hb : (1 : ℤ) ≤ (b : ℤ) := by  
  exact_mod_cast b.pos  
  
have hnonneg : 0 ≤ a - (-|a|) * (b : ℤ) := by  
  have ha : -|a| ≤ a := neg_abs_le a  
  have habs : 0 ≤ |a| := abs_nonneg a  
  nlinarith
```

Define

$$S := \{n \in \mathbb{N} : a - qb = n \text{ for some } q \in \mathbb{Z}\}.$$

By the preceding inequality, $S \neq \emptyset$. Let $r = \min S$ and choose $q \in \mathbb{Z}$ with $a - qb = r$.

```

let h :  $\exists n : \mathbb{N}, \exists q : \mathbb{Z}, a - q * b = (n : \mathbb{Z})$  :=
  ((a - (-|a|) * b).toNat, -|a|, (Int.toNat_of_nonneg hnonneg).symm)

obtain (q, hq) := Nat.find_spec h

let r :  $\mathbb{N}$  := Nat.find h

have hr0 : (0 :  $\mathbb{Z}$ )  $\leq$  r := Int.natCast_nonneg _

```

By minimality, for $s \in \mathbb{Z}$,

$$0 \leq s, \quad s = a - xb \ (x \in \mathbb{Z}) \implies r \leq s.$$

```

have hminimal :
   $\forall s : \mathbb{Z}, 0 \leq s \rightarrow (\exists x : \mathbb{Z}, a - x * b = s) \rightarrow (r : \mathbb{Z}) \leq s$  := by
  intro s hs (x, hx)

have hn : Nat.find h  $\leq$  s.toNat :=
  Nat.find_min' h (x, hx.trans (Int.toNat_of_nonneg hs).symm)

have hc : (Nat.find h :  $\mathbb{Z}$ )  $\leq$  (s.toNat :  $\mathbb{Z}$ ) :=
  Int.ofNat_le.mpr hn

  rwa [Int.toNat_of_nonneg hs] at hc

```

If $r \geq b$, then

$$0 \leq r - b = a - (q + 1)b < r,$$

contradicting minimality. Hence $r < b$.

```

have hrb_int : (r :  $\mathbb{Z}$ ) < b := by
  by_contra hnot

have hs :  $0 \leq (r : \mathbb{Z}) - b$  := by
  linarith

have hsmaller := hminimal ((r :  $\mathbb{Z}$ ) - b) hs (q + 1, by
  dsimp [r]
  nlinarith [hq])

  linarith

```

Thus $a = qb + r$ and $0 \leq r < b$, proving existence.

```

have ha : a = q * b + r := by
  dsimp [r]
  linarith [hq]

have hrb : r < (b : ℕ) := by
  exact_mod_cast hrb_int

refine ((q, r), (ha, hrb), ?_)

```

Suppose another pair $(q', r') \in \mathbb{Z} \times \mathbb{N}$ satisfies $a = q'b + r'$ and $0 \leq r' < b$.

```

rintro (q', r') (ha', hr'b)

change a = q' * b + r' at ha'

have hr'0 : (0 : ℤ) ≤ r' := Int.natCast_nonneg _

have hr'b_int : (r' : ℤ) < b := by
  exact_mod_cast hr'b

```

Since $q, q' \in \mathbb{Z}$,

$$\begin{aligned}
 q' < q &\implies q - q' \geq 1 \implies r' = r + (q - q')b \geq b, \\
 q' > q &\implies q' - q \geq 1 \implies r = r' + (q' - q)b \geq b.
 \end{aligned}$$

Both contradict the remainder bounds, so $q' = q$.

```

have hqq : q' = q := by
  rcases lt_trichotomy q' q with hlt | heq | hgt

  · have hstep : q' + 1 ≤ q := hlt
    nlinarith

  · exact heq

  · have hstep : q + 1 ≤ q' := hgt
    nlinarith

```

Then $r' = a - q'b = a - qb = r$, so $(q', r') = (q, r)$, proving uniqueness.

```

have hrr_int : (r' : ℤ) = r := by
  rw [hqq] at ha'
  linarith

have hrr : r' = r := by
  exact_mod_cast hrr_int

exact Prod.ext hqq hrr

```

□

1.2 The Greatest Common Divisor

For integers c, z , define $c \mid z$ to mean that $z = ck$ for some $k \in \mathbb{Z}$. In particular, $0 \mid z$ if and only if $z = 0$.

Theorem 1.2.1. *Suppose $a, b \in \mathbb{Z}$ are not both zero. There is a positive natural number d which divides both a and b , is an integer linear combination of them, and is divisible by every common divisor.*

```

theorem exists_positive_common_divisor (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  ∃ d : ℕ, 0 < d ∧ (d : ℤ) ∣ a ∧ (d : ℤ) ∣ b ∧
    (∃ x y : ℤ, (d : ℤ) = a * x + b * y) ∧
    (∀ c : ℤ, c ∣ a → c ∣ b → c ∣ (d : ℤ))

```

Proof. Since a, b are not both zero, $a^2 + b^2 > 0$. Thus the set of positive natural numbers of the form $ax + by$, with $x, y \in \mathbb{Z}$, is nonempty.

```

classical

have hpos : 0 < a * a + b * b := by
  rcases hab with ha | hb

  · nlinarith [sq_pos_of_ne_zero ha, sq_nonneg b]
  · nlinarith [sq_nonneg a, sq_pos_of_ne_zero hb]

have hS : ∃ n : ℕ, 0 < n ∧ ∃ x y : ℤ, a * x + b * y = (n : ℤ) := by
  refine ((a * a + b * b).toNat, _, a, b, _)

  · omega
  · exact (Int.toNat_of_nonneg (le_of_lt hpos)).symm

```

Let d be its least element and choose $x, y \in \mathbb{Z}$ with $ax + by = d$. Every positive natural number represented by such a combination is at least d .

```

let d : ℕ := Nat.find hS

obtain ⟨hdpos, x, y, hxy⟩ := Nat.find_spec hS

have hminimal (n : ℕ) (hn : 0 < n)
  (hlinear : ∃ u v : ℤ, a * u + b * v = (n : ℤ)) : d ≤ n :=
  Nat.find_min' hS ⟨hn, hlinear⟩

```

Suppose $u, v \in \mathbb{Z}$. Apply the division algorithm to $au + bv$ and the positive divisor d , obtaining $au + bv = qd + r$ with $0 \leq r < d$.

```

have hddiv (u v : ℤ) : (d : ℤ) ∣ a * u + b * v := by
  obtain ⟨⟨q, r⟩, ⟨heq, hr⟩, _⟩ :=
    existsUnique_quotient_remainder (a * u + b * v) ⟨d, hdpos⟩

  change a * u + b * v = q * (d : ℤ) + (r : ℤ) at heq
  change r < d at hr

```

The remainder is again a linear combination: $r = a(u - qx) + b(v - qy)$.

```

have hrlinear : a * (u - q * x) + b * (v - q * y) = (r : ℤ) := by
  calc
    a * (u - q * x) + b * (v - q * y) =
      (a * u + b * v) - q * (a * x + b * y) := by
        ring
    _ = r := by
      rw [hxy]
      linarith [heq]

```

If $r \neq 0$, then $r > 0$ and minimality gives $d \leq r$, contradicting $r < d$. Hence $r = 0$.

```

have hrzero : r = 0 := by
  by_contra hne

  have hle := hminimal r (Nat.pos_of_ne_zero hne)
    ⟨u - q * x, v - q * y, hrlinear⟩

  omega

```

Therefore $d \mid au + bv$. Taking $(u, v) = (1, 0)$ and $(0, 1)$ gives $d \mid a$ and $d \mid b$.

```

refine (q, ?_)
rw [hrzero] at heq
simp only [Nat.cast_zero, add_zero, mul_comm] using heq

have hda : (d : ℤ) | a := by
  simp only [mul_one, mul_zero, add_zero] using hddiv 1 0

have hdb : (d : ℤ) | b := by
  simp only [mul_one, mul_zero, zero_add] using hddiv 0 1

```

Suppose $c \mid a$ and $c \mid b$. Choose $u, v \in \mathbb{Z}$ with $a = cu$ and $b = cv$. Then $d = ax + by = c(ux + vy)$, so $c \mid d$. This proves all the asserted properties.

```

have hcommon (c : ℤ) (hca : c ∣ a) (hcb : c ∣ b) : c ∣ (d : ℤ) := by
  obtain ⟨u, hu⟩ := hca
  obtain ⟨v, hv⟩ := hcb

  refine (u * x + v * y, ?_)
  rw [← hxy, hu, hv]
  ring

exact ⟨d, hdpow, hda, hdb, ⟨x, y, hxy.symm⟩, hcommon⟩

```

□

Definition 1.2.2. For $a, b \in \mathbb{Z}$ not both zero, define $\gcd(a, b)$ to be the positive common divisor constructed above from the least positive linear combination. Define $\gcd(0, 0) = 0$.

```

noncomputable def gcd (a b : ℤ) : ℕ :=
  if hab : a ≠ 0 ∨ b ≠ 0 then
    Classical.choose (exists_positive_common_divisor a b hab)
  else 0

```

Theorem 1.2.3. For integers a, b not both zero, $\gcd(a, b)$ is positive, divides both inputs, is an integer linear combination of them, and is divisible by every common divisor.

```

theorem gcd_spec (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  0 < gcd a b ∧ (gcd a b : ℤ) ∣ a ∧ (gcd a b : ℤ) ∣ b ∧
  (∃ x y : ℤ, (gcd a b : ℤ) = a * x + b * y) ∧
  (∀ c : ℤ, c ∣ a → c ∣ b → c ∣ (gcd a b : ℤ))

```

Proof. The definition selects the number constructed above, together with its established properties.

```
rw [gcd, dite_eq_left hab]
exact Classical.choose_spec (exists_positive_common_divisor a b hab)
```

□

Theorem 1.2.4. *Suppose $a, b \in \mathbb{Z}$ are not both zero and $c \in \mathbb{Z}$ divides both. Then $c \leq \gcd(a, b)$.*

```
theorem common_divisor_le_gcd (a b c :  $\mathbb{Z}$ ) (hab : a  $\neq$  0  $\vee$  b  $\neq$  0)
  (hca : c  $\mid$  a) (hcb : c  $\mid$  b) : c  $\leq$  (gcd a b :  $\mathbb{Z}$ )
```

Proof. Write $d = \gcd(a, b) > 0$. Since $c \mid d$, choose $k \in \mathbb{Z}$ with $d = ck$.

```
obtain (hpos, _, _, _, hcommon) := gcd_spec a b hab

have hdpos : (0 :  $\mathbb{Z}$ ) < gcd a b := by
  exact_mod_cast hpos

obtain (k, hk) := hcommon c hca hcb
```

If $c > 0$, positivity of d forces $k > 0$, hence $k \geq 1$ and $d \geq c$. If $c \leq 0$, then $c < d$ immediately.

```
by_cases hc : 0 < c

· have hkpos : 0 < k := by
  nlinarith

  have hkone : 1  $\leq$  k := hkpos

  nlinarith

· llinarith
```

□

Theorem 1.2.5. *The zero-pair convention is $\gcd(0, 0) = 0$.*

```
theorem gcd_zero_zero : gcd 0 0 = 0
```

Proof. Neither input is nonzero, so the definition assigns the value zero. This is a convention; the common divisors of zero and zero have no greatest integer.

```

unfold gcd
rw [dite_eq_right (by
  rintro (h | h)
  exact h rfl
  exact h rfl)]

```

□

Theorem 1.2.6. *Suppose $a, b \in \mathbb{Z}$ are not both zero and $d \in \mathbb{N}$ divides both. If every integer common divisor is at most d , then $\gcd(a, b) = d$.*

```

theorem gcd_eq_of_greatest (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) (d : ℕ)
  (hda : (d : ℤ) ∣ a) (hdb : (d : ℤ) ∣ b)
  (hgreatest : ∀ c : ℤ, c ∣ a → c ∣ b → c ≤ (d : ℤ)) : gcd a b = d

```

Proof. Since $\gcd(a, b)$ is a common divisor, the hypothesis gives $\gcd(a, b) \leq d$.

```

have hspec := gcd_spec a b hab
have hle := hgreatest (gcd a b) hspec.2.1 hspec.2.2.1

```

The preceding bound applied to d gives $d \leq \gcd(a, b)$. Antisymmetry proves equality.

```

have hge := common_divisor_le_gcd a b d hab hda hdb
exact_mod_cast (le_antisymm hle hge)

```

□

Theorem 1.2.7. *Suppose $a, b \in \mathbb{Z}$ are not both zero. There are integers x, y such that $\gcd(a, b) = ax + by$.*

```

theorem exists_gcd_eq_mul_add_mul (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  ∃ x y : ℤ, (gcd a b : ℤ) = a * x + b * y

```

Proof. The linear-combination representation is one of the properties established when constructing the gcd.

```
exact (gcd_spec a b hab).2.2.2.1
```

□

Theorem 1.2.8. Suppose $a, b \in \mathbb{Z}$ are not both zero. The set of integer linear combinations $ax + by$ is exactly the set of integer multiples of $\gcd(a, b)$.

```
theorem linear_combinations_eq_gcd_multiples (a b : ℤ) (hab : a ≠ 0 ∨ b
↪ ≠ 0) :
  {z : ℤ | ∃ x y : ℤ, z = a * x + b * y} =
  {z : ℤ | ∃ n : ℤ, z = n * (gcd a b : ℤ)}
```

Proof. Write $d = \gcd(a, b) = ax_0 + by_0$. Since d divides both inputs, choose $u, v \in \mathbb{Z}$ with $a = du$ and $b = dv$. We prove both set inclusions.

```
obtain ⟨x₀, y₀, hbezout⟩ := exists_gcd_eq_mul_add_mul a b hab

have hcommon := (gcd_spec a b hab).2

obtain ⟨u, hu⟩ := hcommon.1
obtain ⟨v, hv⟩ := hcommon.2.1

apply Set.ext
intro z
constructor
```

If $z = ax + by$, then $z = (ux + vy)d$, so z is an integer multiple of d .

```
· rintro ⟨x, y, hz⟩

refine ⟨u * x + v * y, ?_⟩
calc
  z = a * x + b * y := hz
  _ = ((gcd a b : ℤ) * u) * x + ((gcd a b : ℤ) * v) * y := by
    rw [← hu, ← hv]
  _ = (u * x + v * y) * (gcd a b : ℤ) := by
    ring
```

Conversely, if $z = nd$, then $z = a(nx_0) + b(ny_0)$, an integer linear combination of a, b .

```

· rintro ⟨n, hz⟩

  refine ⟨n * x₀, n * y₀, ?_⟩
  rw [hz, hbezout]
  ring

```

□

Theorem 1.2.9. *Let $a, b \in \mathbb{Z}$ be not both zero. They are relatively prime, meaning $\gcd(a, b) = 1$, if and only if there are integers x, y with $1 = ax + by$.*

```

theorem gcd_eq_one_iff_exists_mul_add_mul (a b : ℤ) (hab : a ≠ 0 ∨ b ≠
↪ 0) :
  gcd a b = 1 ↔ ∃ x y : ℤ, 1 = a * x + b * y

```

Proof. For the forward implication, substitute $\gcd(a, b) = 1$ in the previously proved Bezout identity.

```

constructor

· intro h

  obtain ⟨x, y, hxy⟩ := exists_gcd_eq_mul_add_mul a b hab

  exact ⟨x, y, by simp only [h, Nat.cast_one] using hxy⟩

```

Conversely, 1 divides both inputs. If c is a common divisor, write $a = cu$ and $b = cv$. Then $1 = c(ux + vy)$. For $c > 0$ the second factor is a positive integer, hence at least one, so $c \leq 1$; for $c \leq 0$ the same bound is immediate. The order characterization of the gcd gives the result.

```

· rintro ⟨x, y, hxy⟩
  apply gcd_eq_of_greatest a b hab 1

  · exact ⟨a, by ring⟩
  · exact ⟨b, by ring⟩
  · intro c hca hcb

  obtain ⟨u, hu⟩ := hca
  obtain ⟨v, hv⟩ := hcb

  have hone : 1 = c * (u * x + v * y) := by
    calc

```

```

1 = a * x + b * y := hxy
_ = c * (u * x + v * y) := by
  rw [hu, hv]
  ring

by_cases hc : 0 < c

· have hk : 0 < u * x + v * y := by
  nlinarith

  have hkone : 1 ≤ u * x + v * y := hk

  norm_num only [Nat.cast_one]
  nlinarith

· norm_num only [Nat.cast_one]
  omega

```

□

Corollary 1.2.10. *For integers a, b not both zero, set $d = \gcd(a, b)$. Then $\gcd(a/d, b/d) = 1$, where the quotients are integers.*

```

theorem gcd_div_gcd_eq_one (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  gcd (a / (gcd a b : ℤ)) (b / (gcd a b : ℤ)) = 1

```

Proof. Since the positive gcd divides both inputs, write $a = du$ and $b = dv$. The integer quotients are exactly u and v , since $d \neq 0$.

```

obtain ⟨hdpos, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ := gcd_spec a b hab

generalize hg : gcd a b = d at *

have hdne : (d : ℤ) ≠ 0 := by
  exact_mod_cast (ne_of_gt hdpos)

have ha : a / (d : ℤ) = u := by
  rw [hu]
  exact Int.mul_ediv_cancel_left u hdne

have hb : b / (d : ℤ) = v := by
  rw [hv]
  exact Int.mul_ediv_cancel_left v hdne

```

Choose Bezout coefficients with $d = ax + by$. Substitution gives $d = d(ux + vy)$, and cancellation of the nonzero integer d yields $1 = ux + vy$.

```

have hone : (1 : ℤ) = u * x + v * y := by
  apply mul_left_cancel₀ hdne
  calc
    (d : ℤ) * 1 = a * x + b * y := by
      simp using hxy
    _ = (d : ℤ) * (u * x + v * y) := by
      rw [hu, hv]
    ring

```

The identity rules out $u = v = 0$. Apply the preceding characterization to conclude that the two quotients are relatively prime.

```

have huv : u ≠ 0 ∧ v ≠ 0 := by
  by_contra h
  push Not at h

  obtain ⟨rfl, rfl⟩ := h

  norm_num at hone

  rw [ha, hb]
  exact (gcd_eq_one_iff_exists_mul_add_mul u v huv).2 ⟨x, y, hone⟩

```

□

Corollary 1.2.11. *Let $a, b, c \in \mathbb{Z}$. If $\gcd(a, b) = 1$, $a \mid c$, and $b \mid c$, then $ab \mid c$.*

```

theorem mul_dvd_of_coprime (a b c : ℤ) (hab : gcd a b = 1)
  (hac : a ∣ c) (hbc : b ∣ c) : a * b ∣ c

```

Proof. The inputs cannot both be zero because $\gcd(0, 0) = 0$. Thus the coprimality characterization supplies integers x, y with $1 = ax + by$.

```

have hnz : a ≠ 0 ∧ b ≠ 0 := by
  by_contra h
  push Not at h

  obtain ⟨rfl, rfl⟩ := h

  rw [gcd_zero_zero] at hab

```

contradiction

```
obtain ⟨x, y, hxy⟩ := (gcd_eq_one_iff_exists_mul_add_mul a b hnz).1 hab
```

Write $c = ar = bs$. Multiplying the Bezout identity by c and using these two expressions gives $c = acx + bcy = ab(sx + ry)$. The integer $sx + ry$ is the required divisibility witness.

```
obtain ⟨r, hr⟩ := hac

obtain ⟨s, hs⟩ := hbc

refine ⟨s * x + r * y, ?_⟩
calc
  c = c * (a * x + b * y) := by
    rw [← hxy]
    ring
  _ = a * c * x + b * c * y := by
    ring
  _ = a * (b * s) * x + b * (a * r) * y := by
    nth_rw 1 [hs]
    rw [hr]
  _ = (a * b) * (s * x + r * y) := by
    ring
```

□

Theorem 1.2.12. *Let $a, b, c \in \mathbb{Z}$ with $\gcd(a, b) = 1$. If $a \mid bc$, then $a \mid c$ (Euclid's lemma).*

```
theorem dvd_of_dvd_mul_of_coprime (a b c : ℤ) (hab : gcd a b = 1)
  (hdiv : a ∣ b * c) : a ∣ c
```

Proof. As before, coprimality excludes the zero pair, so choose integers x, y with $1 = ax + by$.

```
have hnz : a ≠ 0 ∨ b ≠ 0 := by
  by_contra h
  push Not at h

obtain ⟨rfl, rfl⟩ := h

rw [gcd_zero_zero] at hab
contradiction
```

```
obtain ⟨x, y, hxy⟩ := (gcd_eq_one_iff_exists_mul_add_mul a b hnz).1 hab
```

Write $bc = ak$. Multiplication by c gives $c = acx + bcy = a(cx + ky)$, which exhibits an integer witness for $a \mid c$.

```
obtain ⟨k, hk⟩ := hdiv

refine ⟨c * x + k * y, ?_⟩
calc
  c = (a * x + b * y) * c := by
    rw [← hxy]
    ring
  _ = a * (c * x) + (b * c) * y := by
    ring
  _ = a * (c * x + k * y) := by
    rw [hk]
    ring
```

□

Theorem 1.2.13. *Let $a, b \in \mathbb{Z}$ be not both zero and let $d \in \mathbb{Z}$ be positive. Then $d = \gcd(a, b)$ if and only if d divides both inputs and every integer common divisor of a, b divides d .*

```
theorem eq_gcd_iff_common_divisor (a b d : ℤ) (hab : a ≠ 0 ∨ b ≠ 0)
  (hd : 0 < d) :
  d = (gcd a b : ℤ) ↔
    d ∣ a ∧ d ∣ b ∧ (∀ c : ℤ, c ∣ a → c ∣ b → c ∣ d)
```

Proof. If $d = \gcd(a, b)$, all three divisibility properties follow from the construction of the gcd.

```
constructor

· intro heq
  subst d

obtain ⟨_, hda, hdb, _, hcommon⟩ := gcd_spec a b hab

exact ⟨hda, hdb, hcommon⟩
```

Conversely, let $g = \gcd(a, b) > 0$. Because d is a common divisor, $d \leq g$. Since g divides both inputs, the last hypothesis gives $d = gk$. Positivity forces the integer k to be at least one, so $g \leq d$. The two bounds imply $d = g$.

```

• rintro (hda, hdb, hcommon)

  have hle := common_divisor_le_gcd a b d hab hda hdb

  obtain (hgpos, hga, hgb, _, _) := gcd_spec a b hab
  obtain (k, hk) := hcommon (gcd a b) hga hgb

  have hgpos' : (0 : ℤ) < gcd a b := by
    exact_mod_cast hgpos

  have hkpos : 0 < k := by
    nlinarith

  have hkone : 1 ≤ k := hkpos

  have hge : (gcd a b : ℤ) ≤ d := by
    nlinarith

  exact le_antisymm hle hge

```

□

1.3 The Euclidean Algorithm

We retain the nonnegative gcd, with $\gcd(0, 0) = 0$. Pairs with the same common divisors have equal gcds: for nonzero pairs, each gcd is at most the other; a zero pair is characterized by having zero as a common divisor.

Lemma 1.3.1. *If $a, b, q, r \in \mathbb{Z}$ and $a = qb + r$, then $\gcd(a, b) = \gcd(b, r)$. No sign or size condition on the remainder is needed for this identity.*

```

theorem gcd_eq_gcd_remainder (a b q r : ℤ) (ha : a = q * b + r) :
  gcd a b = gcd b r

```

Proof. It suffices to compare common divisors. If $a = cu$ and $b = cv$, then $r = c(u - qv)$. Thus a common divisor of a, b also divides b, r .

```

apply gcd_eq_of_common_divisors
intro c
constructor

· rintro ⟨⟨u, hu⟩, ⟨v, hv⟩⟩
  refine ⟨⟨v, hv⟩, ⟨u - q * v, ?_⟩⟩
  calc
    r = a - q * b := by
      linarith [ha]
    _ = c * (u - q * v) := by
      rw [hu, hv]
      ring

```

Conversely, if $b = cu$ and $r = cv$, then $a = c(qu + v)$. Both implications use explicit integer witnesses.

```

· rintro ⟨⟨u, hu⟩, ⟨v, hv⟩⟩
  exact ⟨⟨q * u + v, by
    rw [ha, hu, hv]
    ring⟩, ⟨u, hu⟩⟩

```

□

Lemma 1.3.2. *For all integers a, b , taking absolute values leaves the gcd unchanged: $\gcd(|a|, |b|) = \gcd(a, b)$.*

```

theorem gcd_abs (a b : ℤ) : gcd |a| |b| = gcd a b

```

Proof. An integer c divides $|z|$ exactly when it divides z . For $z \geq 0$ this is immediate; for $z < 0$, negate the multiplication witness.

```

have hdiv (c z : ℤ) : c | |z| ↔ c | z := by
  by_cases hz : 0 ≤ z

  · rw [abs_of_nonneg hz]
  · rw [abs_of_neg (lt_of_not_ge hz)]
  constructor <=> rintro ⟨k, hk⟩ <=> exact ⟨-k, by linarith⟩

```

Apply this equivalence to both inputs and use the common-divisor characterization of the gcd.

```

exact gcd_eq_of_common_divisors _ _ _ (fun c => by rw [hdiv, hdiv])

```

□

Absolute values reduce the computation to nonnegative inputs. No initial ordering is necessary: dividing a smaller input by a larger one simply gives quotient zero. A finite division trace records every quotient and remainder. Its terminal pair is $(g, 0)$; each preceding pair (a, b) satisfies $a = qb + r$ with $0 \leq r < b$, and is followed by (b, r) .

Definition 1.3.3. The relation `EuclideanTrace` $a\ b\ g$ records such a finite computation. The natural number types encode nonnegativity, and $r < b$ records strict decrease.

```
inductive EuclideanTrace :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbf{Prop}$ 
| done (a :  $\mathbb{N}$ ) : EuclideanTrace a 0 a
| step (a b r g :  $\mathbb{N}$ ) (q :  $\mathbb{Z}$ ) (hr : r < b)
  (ha : (a :  $\mathbb{Z}$ ) = q * (b :  $\mathbb{Z}$ ) + (r :  $\mathbb{Z}$ ))
  (tail : EuclideanTrace b r g) : EuclideanTrace a b g
```

Theorem 1.3.4. For every $a, b \in \mathbb{N}$, a finite Euclidean division trace starts at (a, b) and ends at some $(g, 0)$.

```
theorem euclideanTrace_exists (a b :  $\mathbb{N}$ ) :  $\exists\ g, \text{EuclideanTrace } a\ b\ g$ 
```

Proof. Use strong induction on the second input, allowing the first input to vary. If the second input is zero, the computation has already ended.

```
induction b using Nat.strong_induction_on generalizing a with
| h b ih =>
  by_cases hb : b = 0
  · subst b
    exact ⟨a, EuclideanTrace.done a⟩
```

Otherwise the previously proved division algorithm supplies $a = qb + r$ with $r < b$. The induction hypothesis gives a finite trace from (b, r) ; prefix the recorded division to obtain the required trace.

```
· obtain ⟨(q, r), ⟨ha, hr⟩, _⟩ :=
  existsUnique_quotient_remainder (a :  $\mathbb{Z}$ ) (b, Nat.pos_of_ne_zero hb)

obtain ⟨g, hg⟩ := ih r hr b

exact ⟨g, EuclideanTrace.step a b r g q hr ha hg⟩
```

□

Theorem 1.3.5. If a Euclidean trace starts at (a, b) and ends at $(g, 0)$, then $\gcd(a, b) = g$.

```
theorem EuclideanTrace.gcd_eq {a b g : ℕ} (h : EuclideanTrace a b g) :
  gcd (a : ℤ) (b : ℤ) = g
```

Proof. Induct on the finite trace. At the terminal pair, $\gcd(a, 0) = a$ for $a \in \mathbb{N}$, by greatestness (and the zero-pair convention). At each earlier division, the remainder lemma preserves the gcd, so the induction hypothesis gives the same terminal value.

```
induction h with
| done a => exact gcd_zero_right a
| step a b r g q hr ha tail ih =>
  exact (gcd_eq_gcd_remainder _ _ q _ ha).trans ih
```

□

Corollary 1.3.6. *If the initial second input b is positive, then the terminal entry g of a Euclidean trace is positive. Hence it is the last nonzero remainder, or the initial divisor if the first remainder is zero.*

```
theorem EuclideanTrace.positive {a b g : ℕ} (h : EuclideanTrace a b g)
  (hb : 0 < b) : 0 < g
```

Proof. The terminal entry equals $\gcd(a, b)$. Since $b > 0$, the inputs are not both zero, and positivity follows from the local gcd specification.

```
rw [← h.gcd_eq]
exact (gcd_spec a b (0r.inr (by exact_mod_cast (ne_of_gt hb)))) .1
```

□

Theorem 1.3.7. *For integers a, b, k with $k > 0$, we have $\gcd(ka, kb) = k \gcd(a, b)$.*

```
theorem gcd_mul_of_pos (a b k : ℤ) (hk : 0 < k) :
  (gcd (k * a) (k * b) : ℤ) = k * (gcd a b : ℤ)
```

Proof. First assume a, b are not both zero. Write $d = \gcd(a, b)$, $a = du$, $b = dv$, and $d = ax + by$. The scaled pair is not both zero, and $kd > 0$.

```

by_cases hab : a ≠ 0 ∨ b ≠ 0

· obtain ⟨hdpos, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ := gcd_spec a b hab

  have hpair : k * a ≠ 0 ∨ k * b ≠ 0 := by
    rcases hab with ha | hb

    · exact Or.inl (mul_ne_zero (ne_of_gt hk) ha)
    · exact Or.inr (mul_ne_zero (ne_of_gt hk) hb)

  have hdpos' : (0 : ℤ) < gcd a b := by
    exact_mod_cast hdpos

```

The witnesses u, v show that kd divides both scaled inputs. If c divides them with $ka = cs$ and $kb = ct$, then $kd = k(ax + by) = c(sx + ty)$, so every common divisor divides kd . The local divisibility characterization identifies kd as their gcd.

```

apply Eq.symm
apply (eq_gcd_iff_common_divisor _ _ _ hpair (mul_pos hk hdpos')).2
refine ⟨⟨u, by
  nth_rw 1 [hu]
  ring⟩, ⟨v, by
  nth_rw 1 [hv]
  ring⟩, ?_⟩
rintro c ⟨s, hs⟩ ⟨t, ht⟩
refine ⟨s * x + t * y, ?_⟩
calc
  k * (gcd a b : ℤ) = (k * a) * x + (k * b) * y := by
    rw [hxy]
    ring
  _ = c * (s * x + t * y) := by
    rw [hs, ht]
    ring

```

If both original inputs are zero, both sides vanish by the zero-pair convention.

```

· push Not at hab

  obtain ⟨rfl, rfl⟩ := hab

  simp [gcd_zero_zero]

```

□

Corollary 1.3.8. *For integers a, b and any nonzero integer k , we have $\gcd(ka, kb) = |k| \gcd(a, b)$.*

```
theorem gcd_mul_of_ne_zero (a b k : ℤ) (hk : k ≠ 0) :
  (gcd (k * a) (k * b) : ℤ) = |k| * (gcd a b : ℤ)
```

Proof. Take absolute values of the scaled inputs, use $|kz| = |k||z|$, and apply the positive-factor result to $|k| > 0$. Finally remove the absolute values from the original gcd.

```
calc
  (gcd (k * a) (k * b) : ℤ) = gcd (|k| * |a|) (|k| * |b|) := by
    rw [← gcd_abs (k * a) (k * b), abs_mul, abs_mul]
  _ = |k| * (gcd |a| |b| : ℤ) := gcd_mul_of_pos _ _ _ (abs_pos.mpr hk)
  _ = |k| * (gcd a b : ℤ) := by
    rw [gcd_abs]
```

□

Lemma 1.3.9. *Any two nonzero integers a, b have a positive natural number that is divisible by both.*

```
theorem exists_positive_common_multiple (a b : ℤ) (ha : a ≠ 0) (hb : b ≠
  0) :
  ∃ m : ℕ, 0 < m ∧ a ∣ (m : ℤ) ∧ b ∣ (m : ℤ)
```

Proof. The integer $(ab)^2$ is positive. Its natural-number representation has the same value.

```
have hp : 0 < (a * b) * (a * b) := mul_self_pos.mpr (mul_ne_zero ha hb)
refine (((a * b) * (a * b)).toNat, ?_, ?_, ?_)
  · omega
```

The identities $(ab)^2 = a(ab^2) = b(a^2b)$ supply the two divisibility witnesses.

```
· rw [Int.toNat_of_nonneg (le_of_lt hp)]
  exact ⟨a * b * b, by ring⟩

· rw [Int.toNat_of_nonneg (le_of_lt hp)]
  exact ⟨a * a * b, by ring⟩
```

□

Definition 1.3.10. For nonzero integers a, b , define $\text{lcm}(a, b)$ to be the least positive common multiple, whose existence was just proved. Extend the definition by zero when either input is zero.

```
noncomputable def lcm (a b : ℤ) : ℕ :=
  if h : a ≠ 0 ∧ b ≠ 0 then
    Nat.find (exists_positive_common_multiple a b h.1 h.2)
  else 0
```

Theorem 1.3.11. For nonzero integers a, b , the number $m = \text{lcm}(a, b)$ is positive, satisfies $a \mid m$ and $b \mid m$, and obeys $m \leq c$ for every positive integer common multiple c .

```
theorem lcm_spec (a b : ℤ) (ha : a ≠ 0) (hb : b ≠ 0) :
  0 < lcm a b ∧ a ∣ (lcm a b : ℤ) ∧ b ∣ (lcm a b : ℤ) ∧
  (∀ c : ℤ, a ∣ c → b ∣ c → 0 < c → (lcm a b : ℤ) ≤ c)
```

Proof. The least-element construction supplies positivity and both divisibility properties.

```
classical

rw [lcm, dite_eq_left {ha, hb}]

obtain {hp, hma, hmb} := Nat.find_spec (exists_positive_common_multiple
  ↪ a b ha hb)

refine {hp, hma, hmb, ?_}
```

A positive integer common multiple c is also a positive natural number with the same value. Minimality among natural common multiples gives $m \leq c$, after converting back to integers.

```
intro c hac hbc hc

have hc' : (c.toNat : ℤ) = c := Int.toNat_of_nonneg (le_of_lt hc)

have hmin := Nat.find_min' (exists_positive_common_multiple a b ha hb)
  (show 0 < c.toNat ∧ a ∣ (c.toNat : ℤ) ∧ b ∣ (c.toNat : ℤ) from
    {by omega, by rwa [hc'], by rwa [hc']})

exact_mod_cast (show (Nat.find (exists_positive_common_multiple a b ha
  ↪ hb) : ℤ) ≤ c by
  exact (Int.ofNat_le.mpr hmin).trans_eq hc')
```

□

Theorem 1.3.12. *For positive integers a, b , we have $\gcd(a, b) \operatorname{lcm}(a, b) = ab$.*

```
theorem gcd_mul_lcm (a b : ℤ) (ha : 0 < a) (hb : 0 < b) :
  (gcd a b : ℤ) * (lcm a b : ℤ) = a * b
```

Proof. Put $d = \gcd(a, b) > 0$. Choose $u, v, x, y \in \mathbb{Z}$ such that $a = du$, $b = dv$, and $d = ax + by$. Since $b > 0$, we have $v > 0$. The positive integer av is a common multiple because $av = bu$.

```
obtain ⟨hd, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ :=
  gcd_spec a b (Or.inl (ne_of_gt ha))

generalize hg : gcd a b = d at *

have hd' : (0 : ℤ) < d := by
  exact_mod_cast hd

have hvpos : 0 < v := by
  nlinarith [hv]

have hnpos : 0 < a * v := mul_pos ha hvpos

have ham : a ∣ a * v := ⟨v, rfl⟩

have hbm : b ∣ a * v := ⟨u, by
  rw [hu, hv]
  ring⟩
```

Every common multiple c is divisible by av . Indeed, write $c = as = bt$ and multiply the Bezout identity by c : $dc = ab(tx + sy) = d av(tx + sy)$. Cancel the nonzero factor d to obtain the required witness.

```
have hdiv (c : ℤ) (hac : a ∣ c) (hbc : b ∣ c) : a * v ∣ c := by
  obtain ⟨s, hs⟩ := hac
  obtain ⟨t, ht⟩ := hbc

  refine ⟨t * x + s * y, ?_⟩
  apply mul_left_cancel₀ (ne_of_gt hd')
  calc
    (d : ℤ) * c = (a * x + b * y) * c := by
      rw [hxy]
    _ = a * c * x + b * c * y := by
      ring
    _ = a * (b * t) * x + b * (a * s) * y := by
      nth_rw 1 [ht]
      rw [hs]
```

```

_ = (d : ℤ) * (a * v * (t * x + s * y)) := by
  rw [hv]
  ring

```

Let $\ell = \text{lcm}(a, b) > 0$. Minimality gives $\ell \leq av$, while the preceding argument gives $\ell = avk$. Positivity forces the integer $k \geq 1$, so $\ell \geq av$. Therefore $\ell = av$, and $d\ell = dav = ab$.

```

obtain ⟨hlpos, hla, hlb, hleast⟩ := lcm_spec a b (ne_of_gt ha) (ne_of_gt
  ↪ hb)

have hle : (lcm a b : ℤ) ≤ a * v := hleast _ ham hbm hnpos

obtain ⟨k, hk⟩ := hdiv (lcm a b) hla hlb

have hlpos' : (0 : ℤ) < lcm a b := by
  exact_mod_cast hlpos

have hkpos : 0 < k := by
  nlinarith

have hkone : 1 ≤ k := hkpos

have heq : (lcm a b : ℤ) = a * v := by
  nlinarith

rw [heq, hv]
ring

```

□

1.4 The Diophantine Equation $ax+by=c$

All coefficients and unknowns are integers. We use the nonnegative gcd constructed above.

Theorem 1.4.1. *The equation $ax+by = c$ has an integer solution if and only if $\text{gcd}(a, b) \mid c$. This includes $a = b = 0$, when both conditions mean $c = 0$.*

```

theorem exists_mul_add_mul_eq_iff (a b c : ℤ) :
  (∃ x y : ℤ, a * x + b * y = c) ↔ (gcd a b : ℤ) ∣ c

```

Proof. First suppose that the coefficients are not both zero. Write $a = du$, $b = dv$, and $d = ar + bs$, where $d = \text{gcd}(a, b) > 0$.

```

by_cases hab : a ≠ 0 ∨ b ≠ 0

· obtain (_, ⟨u, hu⟩), ⟨v, hv⟩, ⟨r, s, hrs⟩, _ := gcd_spec a b hab
  generalize hg : gcd a b = d at *
  constructor

```

Any solution gives $c = ax + by = d(ux + vy)$, so $d \mid c$.

```

· rintro ⟨x, y, rfl⟩
  refine ⟨u * x + v * y, ?_⟩
  rw [hu, hv]
  ring

```

Conversely, if $c = dk$, multiplying the Bezout identity by k gives the solution $x = rk$, $y = sk$.

```

· rintro ⟨k, rfl⟩
  refine ⟨r * k, s * k, ?_⟩
  rw [hrs]
  ring

```

If both coefficients vanish, their gcd is zero. The equation is solvable exactly when $c = 0$, which is also the meaning of $0 \mid c$.

```

· have ha : a = 0 := Classical.byContradiction (fun h => hab (Or.inl h))
  have hb : b = 0 := Classical.byContradiction (fun h => hab (Or.inr h))
  simp [ha, hb, gcd_zero_zero, eq_comm]

```

□

Theorem 1.4.2. Suppose a, b are not both zero, put $d = \gcd(a, b)$, and fix a solution (x_0, y_0) of $ax + by = c$. An integer pair (x, y) is a solution if and only if, for some $t \in \mathbb{Z}$,

$$x = x_0 + \frac{b}{d}t, \quad y = y_0 - \frac{a}{d}t.$$

The quotients are integers since d divides both coefficients.

```

theorem mul_add_mul_eq_iff (a b c x_0 y_0 x y : ℤ) (hab : a ≠ 0 ∨ b ≠ 0)
  (h_0 : a * x_0 + b * y_0 = c) :
  a * x + b * y = c ↔ ∃ t : ℤ,
    x = x_0 + (b / (gcd a b : ℤ)) * t ∧
    y = y_0 - (a / (gcd a b : ℤ)) * t

```

Proof. Choose u, v, r, s with $a = du$, $b = dv$, and $d = ar + bs$. Since $d > 0$, the exact quotients a/d and b/d are u and v .

```

obtain ⟨hd, ⟨u, hu⟩, ⟨v, hv⟩, ⟨r, s, hrs⟩, _⟩ := gcd_spec a b hab
generalize hg : gcd a b = d at *

have hdne : (d : ℤ) ≠ 0 := by
  exact_mod_cast (ne_of_gt hd)

have ha : a / (d : ℤ) = u := by
  rw [hu]
  exact Int.mul_ediv_cancel_left u hdne

have hb : b / (d : ℤ) = v := by
  rw [hv]
  exact Int.mul_ediv_cancel_left v hdne

```

Substituting into the Bezout identity and cancelling d gives $ur + vs = 1$.

```

have hone : u * r + v * s = 1 := by
  apply mul_left_cancel₀ hdne
calc
  (d : ℤ) * (u * r + v * s) = a * r + b * s := by
    rw [hu, hv]
    ring
  _ = (d : ℤ) * 1 := by linarith [hrs]

rw [ha, hb]
constructor

```

For another solution, subtract the two equations and cancel d . With $X = x - x_0$ and $Y = y - y_0$, this gives $uX + vY = 0$.

```

· intro h
have hdiff : u * (x - x₀) + v * (y - y₀) = 0 := by
  apply mul_left_cancel₀ hdne
calc
  (d : ℤ) * (u * (x - x₀) + v * (y - y₀)) =
    (a * x + b * y) - (a * x₀ + b * y₀) := by
      rw [hu, hv]
      ring
  _ = (d : ℤ) * 0 := by
    rw [h, h₀]
    ring

```

Put $t = sX - rY$. Then $(ur + vs)X - r(uX + vY) = vt$ and $(ur + vs)Y - s(uX + vY) = -ut$. Hence $X = vt$ and $Y = -ut$, as required.

```

refine (s * (x - x₀) - r * (y - y₀), ?_, ?_)

· nlinarith [congrArg (fun z => z * (x - x₀)) hone,
  congrArg (fun z => z * r) hdiff]
· nlinarith [congrArg (fun z => z * (y - y₀)) hone,
  congrArg (fun z => z * s) hdiff]

```

Conversely, substitution of $x = x_0 + vt$ and $y = y_0 - ut$ cancels the terms involving t , because $av = bu$. The result is $ax_0 + by_0 = c$.

```

· rintro (t, rfl, rfl)
  calc
    a * (x₀ + v * t) + b * (y₀ - u * t) = a * x₀ + b * y₀ := by
      rw [hu, hv]
      ring
      _ = c := h₀

```

□

Corollary 1.4.3. *If $\gcd(a, b) = 1$ and $ax_0 + by_0 = c$, then the integer solutions are exactly $x = x_0 + bt$, $y = y_0 - at$ for $t \in \mathbb{Z}$.*

```

theorem mul_add_mul_eq_iff_of_coprime (a b c x₀ y₀ x y : ℤ)
  (hab : gcd a b = 1) (h₀ : a * x₀ + b * y₀ = c) :
  a * x + b * y = c ↔ ∃ t : ℤ, x = x₀ + b * t ∧ y = y₀ - a * t

```

Proof. The coefficients cannot both be zero, since $\gcd(0, 0) = 0$.

```

have hnz : a ≠ 0 ∨ b ≠ 0 := by
  by_contra h
  push Not at h
  obtain (rfl, rfl) := h
  rw [gcd_zero_zero] at hab
  contradiction

```

Apply the general solution formula and substitute $d = 1$.

```

simpa only [hab, Nat.cast_one, Int.ediv_one] using
  mul_add_mul_eq_iff a b c x₀ y₀ x y hnz h₀

```

□

Chapter 2

Primes and Their Distribution

2.1 The Fundamental Theorem of Arithmetic

Definition 2.1.1. A natural number $p > 1$ is *prime* when every natural divisor is 1 or p . Since $p > 0$, zero cannot divide p ; thus this is precisely the condition on positive divisors.

```
def isPrime (p : ℕ) : Prop :=  
  1 < p ∧ ∀ d : ℕ, d | p → d = 1 ∨ d = p
```

Definition 2.1.2. A natural number $n > 1$ which is not prime is *composite*. In particular, neither zero nor one is prime or composite.

```
def isComposite (n : ℕ) : Prop :=  
  1 < n ∧ ¬isPrime n
```

Theorem 2.1.3. Let p be prime and let $a, b \in \mathbb{Z}$. If $p \mid ab$, then $p \mid a$ or $p \mid b$.

```
theorem prime_dvd_mul (p : ℕ) (hp : isPrime p) (a b : ℤ)  
  (h : (p : ℤ) | a * b) : (p : ℤ) | a ∨ (p : ℤ) | b
```

Proof. If $p \mid a$, the assertion is immediate. Otherwise, p is nonzero because $p > 1$.

```

classical
by_cases hpa : (p : ℤ) | a

· exact Or.inl hpa

· have hnz : (p : ℤ) ≠ 0 := by
  have := hp.1
  omega

```

The positive integer $d = \gcd(p, a)$ divides p , so it is 1 or p . The second possibility would imply $p \mid a$, contrary to the assumption. Thus $\gcd(p, a) = 1$.

```

obtain (_, hdp, hda, _, _) := Chapter01.gcd_spec (p : ℤ) a (Or.inl hnz)
have hd : Chapter01.gcd (p : ℤ) a | p := Int.natCast_dvd_natCast.mp hdp
have hcoprime : Chapter01.gcd (p : ℤ) a = 1 := by
  rcases hp.2 _ hd with hone | heq

· exact hone
· rw [heq] at hda
  exact (hpa hda).elim

```

The previously proved Euclid lemma for coprime integers now gives $p \mid b$.

```

exact Or.inr (Chapter01.dvd_of_dvd_mul_of_coprime (p : ℤ) a b hcoprime
  ↪ h)

```

□

Corollary 2.1.4. *If a prime p divides the product of a finite list of integers, then it divides at least one member of that list.*

```

theorem prime_dvd_list_prod (p : ℕ) (hp : isPrime p) (l : List ℤ)
  (h : (p : ℤ) | l.prod) : ∃ a ∈ l, (p : ℤ) | a

```

Proof. Induct on the list. The empty product is 1. If $1 = pk$, then $p > 1$ forces $k > 0$, whence $pk \geq p > 1$, a contradiction.

```

induction l with
| nil =>
  obtain (k, hk) := h
  have hpos : (1 : ℤ) < p := by
    exact_mod_cast hp.1
  change 1 = (p : ℤ) * k at hk
  have hkpos : 0 < k := by
    nlinarith
  nlinarith

```

For a nonempty list with first entry a , the preceding theorem says that p divides a or the product of the remaining entries. In the latter case apply the induction hypothesis.

```

| cons a l ih =>
  rcases prime_dvd_mul p hp a l.prod h with ha | hl

  · exact ⟨a, List.mem_cons_self, ha⟩
  · obtain ⟨b, hb, hpb⟩ := ih hl
    exact ⟨b, List.mem_cons_of_mem a hb, hpb⟩

```

□

Lemma 2.1.5. *If p is prime and $a, b \in \mathbb{N}$, then $p \mid ab$ implies $p \mid a$ or $p \mid b$.*

```

theorem prime_dvd_mul_nat (p a b : ℕ) (hp : isPrime p) (h : p ∣ a * b) :
  p ∣ a ∨ p ∣ b

```

Proof. View the natural numbers as integers and apply the product theorem. Divisibility of natural numbers is unchanged by this inclusion, so the conclusion returns to $\mathbb{N}$.

```

have h' : (p : ℤ) ∣ (a : ℤ) * (b : ℤ) := by
  exact_mod_cast h
rcases prime_dvd_mul p hp a b h' with ha | hb

  · exact 0r.inl (Int.natCast_dvd_natCast.mp ha)
  · exact 0r.inr (Int.natCast_dvd_natCast.mp hb)

```

□

Corollary 2.1.6. *If p and all members of a finite list $q_1, \dots, q_n$ are prime, and $p \mid q_1 \cdots q_n$, then $p = q_k$ for some $1 \leq k \leq n$. List membership expresses this conclusion without choosing an indexing convention.*

```
theorem prime_eq_of_dvd_prod (p : ℕ) (hp : isPrime p) (l : List ℕ)
  (hl : ∀ q ∈ l, isPrime q) (h : p ∣ l.prod) : p ∈ l
```

Proof. Induct on the list. As before, a prime cannot divide the empty product 1.

```
induction l with
| nil =>
  obtain (k, hk) := h
  change 1 = p * k at hk
  have hkpos : 0 < k := by
    nlinarith [hp.1]
  nlinarith [hp.1]
```

For a list beginning with q , either $p \mid q$ or p divides the remaining product. In the first case the divisors of the prime q give $p = 1$ or $p = q$. Since $p > 1$, only $p = q$ is possible.

```
| cons q l ih =>
  rcases prime_dvd_mul_nat p q l.prod hp h with hq | ht

  · rcases (hl q List.mem_cons_self).2 p hq with hone | heq

    · have := hp.1
      omega
    · exact List.mem_cons.mpr (Or.inl heq)
```

In the second case apply the induction hypothesis to the remaining prime entries.

```
· exact List.mem_cons.mpr
  (Or.inr (ih (fun r hr => hl r (List.mem_cons_of_mem q hr)) ht))
```

□

A list records factors with their multiplicities. Its product is denoted by `List.prod`; the empty product is 1. Two lists are permutations of each other when they have the same entries with the same multiplicities, possibly in different orders.

Lemma 2.1.7. *Every natural number $n > 0$ is the product of a finite list of primes; for $n = 1$ the list may be empty.*

```
theorem exists_prime_factors (n : ℕ) (hn : 0 < n) :
  ∃ l : List ℕ, (∀ p ∈ l, isPrime p) ∧ l.prod = n
```

Proof. Use strong induction on n . The empty list has product 1. If $n > 1$ is prime, the one-element list containing n has the required product.

```

classical
induction n using Nat.strong_induction_on with
| h n ih =>
  by_cases hone : n = 1

  · exact ([], by simp, by simp using hone.symm)

  · have hgt : 1 < n := by
      omega
    by_cases hp : isPrime n

    · exact ([n], by simp, by simp)

```

If $n > 1$ is not prime, it has a divisor d different from 1 and n . Write $n = dk$.

```

· have hdiv : ∃ d : ℕ, d | n ∧ d ≠ 1 ∧ d ≠ n := by
  simp only [isPrime, hgt, true_and, not_forall, not_or, exists_prop]
  ↪ using hp

obtain ⟨d, ⟨k, hk⟩, hd1, hdn⟩ := hdiv

```

Since $n = dk > 0$ and d, k are natural numbers, both factors are positive.

```

have hdpos : 0 < d := by
  nlinarith
have hkpos : 0 < k := by
  nlinarith

```

If $d \geq n$, then $k = 1$, contradicting $d \neq n$; hence $d < n$. Also $d \geq 2$, so $k < n$. These bounds allow the induction hypothesis to be applied to both factors.

```

have hdl1 : d < n := by
  by_contra h
  have hk1 : k = 1 := by
    nlinarith
  simp [hk1] at hk
  exact hdn hk.symm

have hkl1 : k < n := by
  have hd2 : 2 ≤ d := by
    omega
  nlinarith

```

By the induction hypothesis, d and k have prime factor lists. Concatenating them preserves primality of every entry, and its product is $dk = n$.

```

obtain ⟨l, hl, heql⟩ := ih d hdl t hdp os
obtain ⟨m, hm, heqm⟩ := ih k hkl t hkpos
refine ⟨l ++ m, ?_, ?_⟩

· intro p hp
  rcases List.mem_append.mp hp with hpl | hpm

  · exact hl p hpl
  · exact hm p hpm

· rw [List.prod_append, heql, heqm, ← hk]

```

□

Lemma 2.1.8. *Two finite lists of primes with the same product are permutations of one another.*

```

theorem prime_factors_perm (l m : List ℕ) (hl : ∀ p ∈ l, isPrime p)
  (hm : ∀ p ∈ m, isPrime p) (hprod : l.prod = m.prod) : l.Perm m

```

Proof. Induct on the first list. If it is empty, its product is 1. The second list cannot begin with a prime $q > 1$, since $1 = qM$ would imply $M > 0$ and hence $qM > 1$. Thus both lists are empty.

```

classical
induction l generalizing m with
| nil =>
  cases m with
| nil => exact List.Perm.refl []
| cons q m =>
  have hq := (hm q List.mem_cons_self).1
  change 1 = q * m.prod at hprod
  have hmpos : 0 < m.prod := by
    nlinarith
  nlinarith

```

If the first list begins with p , then p divides the second product. The preceding corollary ensures that p occurs in the second list.

```

| cons p l ih =>
  have hp := hl p List.mem_cons_self
  have hpm : p ∈ m := by
    apply prime_eq_of_dvd_prod p hp m hm
    exact (l.prod, hprod.symm)

```

Move one occurrence of p to the front of the second list and remove it. Since $p > 0$, cancellation shows that the two remaining products are equal.

```

have hperm : m.Perm (p :: m.erase p) := List.perm_cons_erase hpm
have htail : l.prod = (m.erase p).prod := by
  have heq := hperm.prod_eq
  simp only [List.prod_cons] at hprod heq
  exact Nat.eq_of_mul_eq_mul_left hp.1.le (hprod.trans heq)

```

The remaining lists still consist of primes, so the induction hypothesis makes them permutations. Restoring the removed occurrence of p gives the desired permutation of the original lists.

```

have htailperm : l.Perm (m.erase p) := ih (m.erase p)
  (fun q hq => hl q (List.mem_cons_of_mem p hq))
  (fun q hq => hm q (List.mem_of_mem_erase hq)) htail
exact (List.Perm.cons p htailperm).trans hperm.symm

```

□

Theorem 2.1.9. *Every natural number $n > 1$ is a prime or a product of primes. Its prime factor list is nonempty and is unique up to permutation; a list with one entry represents the prime case.*

```

theorem fundamental_theorem_of_arithmetic (n : ℕ) (hn : 1 < n) :
  ∃ l : List ℕ, l ≠ [] ∧ (∀ p ∈ l, isPrime p) ∧ l.prod = n ∧
  ∀ m : List ℕ, (∀ p ∈ m, isPrime p) → m.prod = n → m.Perm l

```

Proof. The existence lemma gives a prime factor list for n . It cannot be empty, since the empty product is 1 and $n > 1$.

```

obtain ⟨l, hl, hprod⟩ := exists_prime_factors n (by omega)
refine ⟨l, ?_, hl, hprod, ?_⟩

· intro heq
  subst l
  simp only [List.prod_nil] at hprod
  omega

```

Any other prime factor list for n has the same product, so the uniqueness lemma makes it a permutation of this list.

```

· intro m hm hmn
  exact prime_factors_perm m l hm hl (hmn.trans hprod.symm)

```

□

To group equal prime factors, use a finitely supported function $e: \mathbb{N} \rightarrow \mathbb{N}$. Its support is the finite set of p with $e(p) \neq 0$; these and only these indices have positive exponents. In Lean this type is `Finsupp`, and its product means $\prod_{p \in \text{supp}(e)} p^{e(p)}$. A multiset forgets the

order of a list while retaining its multiplicities. Passing between multisets and exponent functions only groups or repeats entries; it does not invoke a factorization result.

Lemma 2.1.10. *For every $n > 0$ there is a unique finitely supported function $e: \mathbb{N} \rightarrow \mathbb{N}$ whose support consists of primes and for which $\prod_{p \in \text{supp}(e)} p^{e(p)} = n$.*

```

theorem existsUnique_prime_exponents (n : ℕ) (hn : 0 < n) :
  ∃! e : ℕ →0 ℕ, (∀ p ∈ e.support, isPrime p) ∧
  e.prod (fun p k => p ^ k) = n

```

Proof. Choose a prime factor list and let $e(p)$ count its occurrences of p . Every member of the support is prime. Grouping repeated factors replaces their product by the product of prime powers, still equal to n .

```

classical
obtain ⟨l, hl, hprod⟩ := exists_prime_factors n hn
let e := (l : Multiset ℕ).toFinsupp
have he : (∀ p ∈ e.support, isPrime p) ∧ e.prod (fun p k => p ^ k) = n
  ↪ := by
  constructor

· intro p hp

```

```

exact h1 p (by simp [e] using hp)

· rw [← Finsupp.prod_toMultiset, Multiset.toFinsupp_toMultiset]
  exact hprod

```

For any other exponent function f , repeat each supported prime p exactly $f(p)$ times to form a list. All its entries are prime, and its product is the specified power product n .

```

refine (e, he, ?_)
rintro f (hf, hfn)
let m := f.toMultiset.toList
have hm : ∀ p ∈ m, isPrime p := by
  intro p hp
  exact hf p (by simp [m] using hp)

have hmprod : m.prod = n := by
  rw [Multiset.prod_toList, Finsupp.prod_toMultiset]
  exact hfn

```

The uniqueness lemma makes the two expanded lists permutations. Their multisets, and hence all their multiplicities, agree. Thus $f = e$.

```

have hperm := prime_factors_perm m l hm h1 (hmprod.trans hprod.symm)
have hmulti : f.toMultiset = (l : Multiset ℕ) := by
  simp [m] using Multiset.coe_eq_coe.mpr hperm

have heq := congrArg Multiset.toFinsupp hmulti
simp [e] using heq

```

□

Definition 2.1.11. A *canonical factorization* of n is a pair (L, e) : the list L is strictly increasing, its entries are exactly the support of the finitely supported exponent function e , every supported entry is prime, and the product of the corresponding prime powers is n . The notation *Pairwise* with $<$ requires every earlier entry to be less than every later entry.

```

def isCanonicalFactorization (n : ℕ) (r : List ℕ × (ℕ →0 ℕ)) : Prop :=
  r.1.Pairwise (· < ·) ∧ r.1.toFinset = r.2.support ∧
  (∀ p ∈ r.2.support, isPrime p) ∧ r.2.prod (fun p k => p ^ k) = n

```

Corollary 2.1.12. Every natural number $n > 1$ has exactly one canonical factorization (L, e) .

```
theorem existsUnique_canonical_factorization (n : ℕ) (hn : 1 < n) :
  ∃! r : List ℕ × (ℕ →0 ℕ), isCanonicalFactorization n r
```

Proof. Take the unique exponent function and arrange its finite support in increasing order. The support has no repeated elements, so the resulting list is strictly increasing.

```
classical
obtain ⟨e, he, huniq⟩ := existsUnique_prime_exponents n (by omega)
let l := e.support.sort (· ≤ ·)
have hl : l.Pairwise (· < ·) :=
  List.sortedLT_iff_pairwise.mp (Finset.sortedLT_sort e.support)
```

This pair is a canonical factorization. For any other pair (M, f) , uniqueness of prime exponents gives $f = e$.

```
refine ⟨(l, e), ⟨hl, Finset.sort_toFinset _ _, he⟩, ?_⟩
rintro ⟨m, f⟩ ⟨hm, hsupport, hf⟩
have hfe : f = e := huniq f hf
subst f
```

The list M is strictly increasing and has the same underlying finite set as L . Sorting this finite set therefore returns M as well as L , so the lists agree and the pairs are equal.

```
have hsorted : e.support.sort (· ≤ ·) = m := by
  rw [← hsupport]
  exact (List.toFinset_sort (· ≤ ·) hm.nodup).mpr
    (hm.imp (fun h => Nat.le_of_lt h))

exact Prod.ext hsorted.symm rfl
```

□

Corollary 2.1.13. *If (L, e) is the canonical factorization of $n > 1$, then L is nonempty. Writing $L = (p_1, \dots, p_r)$ and $k_i = e(p_i)$, we have*

$$n = p_1^{k_1} \cdots p_r^{k_r}, \quad p_1 < \cdots < p_r, \quad k_i > 0,$$

and each p_i is prime. Together with the preceding uniqueness result, this is the canonical prime-power form.

```
theorem canonical_factorization_spec (n : ℕ) (hn : 1 < n)
  (r : List ℕ × (ℕ →0 ℕ)) (hr : isCanonicalFactorization n r) :
  r.1 ≠ [] ∧ r.1.Pairwise (· < ·) ∧
  (∀ p ∈ r.1, isPrime p ∧ 0 < r.2 p) ∧
  (r.1.map (fun p => p ^ r.2 p)).prod = n
```

Proof. The list has distinct entries and its underlying set is the support. Thus the product along the list equals the product over the support, which is n .

```
obtain {hsorted, hsupport, hprime, hprod} := hr
have hproduct : (r.1.map (fun p => p ^ r.2 p)).prod = n := by
  rw [← List.prod_toFinset _ hsorted.nodup, hsupport]
  exact hprod

refine {?_, hsorted, ?_, hproduct}
```

If the list were empty, this product would be 1, contrary to $n > 1$.

```
· intro hnil
  rw [hnil] at hproduct
  simp only [List.map_nil, List.prod_nil] at hproduct
  omega
```

Every listed prime belongs to the support. By definition of the support its exponent is nonzero, hence positive; primality is part of the canonical factorization condition.

```
· intro p hp
have hmem : p ∈ r.2.support := by
  rw [← hsupport]
  exact List.mem_toFinset.mpr hp

exact {hprime p hmem, Nat.pos_of_ne_zero (Finsupp.mem_support_iff.mp
  ↪ hmem)}
```

□

2.2 The Sieve of Eratosthenes

Lemma 2.2.1. *Every natural number $n > 1$ has a prime divisor $p \leq n$.*

```
theorem exists_prime_divisor (n : ℕ) (hn : 1 < n) :
  ∃ p : ℕ, isPrime p ∧ p | n ∧ p ≤ n
```

Proof. The fundamental theorem of arithmetic supplies a nonempty list of prime factors. Take its first entry p and write $n = pM$, where M is the product of the remaining entries.

```
obtain ⟨l, hne, hl, hprod, _⟩ := fundamental_theorem_of_arithmetic n hn
cases l with
| nil => exact (hne rfl).elim
| cons p l =>
  have hp := hl p List.mem_cons_self
  have heq : n = p * l.prod := hprod.symm
```

Since $n > 0$, the natural number M is positive and hence at least one. Thus $p \leq n$, and p is the required prime divisor.

```
have hpos : 0 < l.prod := by
  nlinarith
exact ⟨p, hp, ⟨l.prod, heq⟩, by nlinarith⟩
```

□

Theorem 2.2.2. *If $n > 1$ is composite, it has a prime divisor p satisfying $p \leq \sqrt{n}$. For integer p , this is equivalent to $p \leq \lfloor \sqrt{n} \rfloor$; Lean uses the natural square root, characterized by $p \leq \text{sqrt}(n)$ if and only if $p^2 \leq n$.*

```
theorem exists_prime_divisor_le_sqrt (n : ℕ) (hn : isComposite n) :
  ∃ p : ℕ, isPrime p ∧ p | n ∧ p ≤ Nat.sqrt n
```

Proof. Since n is composite, choose a divisor d other than 1 and n , and write $n = dk$. Positivity and the proper-divisor condition give $d > 1$ and $k > 1$.

```
classical
have hdiv : ∃ d : ℕ, d | n ∧ d ≠ 1 ∧ d ≠ n := by
  simpa only [isPrime, hn.1, true_and, not_forall, not_or, exists_prop]
  ↪ using hn.2

obtain ⟨d, ⟨k, hk⟩, hd1, hdn⟩ := hdiv
have hdpos : 0 < d := by
  nlinarith [hn.1]
have hd : 1 < d := by
  omega
have hkgt : 1 < k := by
  by_contra h
  have hk1 : k = 1 := by
    nlinarith [hn.1]
```

```
simp only [hk1, mul_one] at hk
exact hdn hk.symm
```

Choose a prime divisor p of $\min(d, k)$. If p divides d , write $d = pt$ and obtain $n = p(tk)$. If it divides k , write $k = pt$ and obtain $n = p(dt)$. Thus $p \mid n$ in either case.

```
obtain ⟨p, hp, ⟨t, ht⟩, hle⟩ := exists_prime_divisor (min d k) (by
  ↪ omega)
refine ⟨p, hp, ?_, Nat.le_sqrt.mpr ?_⟩

· rcases le_total d k with hdk | hkd

  · rw [min_eq_left hdk] at ht
    refine ⟨t * k, ?_⟩
    rw [hk, ht]
    ring

  · rw [min_eq_right hkd] at ht
    refine ⟨d * t, ?_⟩
    rw [hk, ht]
    ring
```

The choice of p gives $p \leq d$ and $p \leq k$. Therefore $p^2 \leq dk = n$, proving the square-root bound.

```
· have hpd : p ≤ d := le_trans hle (min_le_left __)
  have hpk : p ≤ k := le_trans hle (min_le_right __)
  nlinarith [Nat.mul_le_mul hpd hpk]
```

□

Consequently, a sieve need only test prime divisors up to the square root: a number greater than one with no such prime divisor cannot be composite.

Lemma 2.2.3. *For natural numbers k, n with $0 < k \leq n$, there is a natural number t such that $n! = kt$.*

```
theorem factorial_multiple (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  ∃ t : ℕ, n.factorial = k * t
```

Proof. Induct on n . The case $n = 0$ has no admissible k . At the next step, if $k = n + 1$, use $(n + 1)! = (n + 1)n!$.

```

induction n with
| zero => omega
| succ n ih =>
  by_cases heq : k = n + 1

  · exact ⟨n.factorial, by rw [heq, Nat.factorial_succ]⟩

```

Otherwise $k \leq n$. Multiply the inductive equality $n! = kt$ by $n + 1$ to obtain the required witness.

```

· obtain ⟨t, ht⟩ := ih (by omega)
  refine ⟨(n + 1) * t, ?_⟩
  rw [Nat.factorial_succ, ht]
  ring

```

□

Lemma 2.2.4. *If a prime p divides $M + 1$, it does not divide M .*

```

theorem not_dvd_of_dvd_add_one (p M : ℕ) (hp : isPrime p) (h : p ∣ M +
  ↪ 1) :
  ¬p ∣ M

```

Proof. If $M = pj$ and $M + 1 = pk$, then $j < k$. Hence $M + p = p(j + 1) \leq pk = M + 1$, implying $p \leq 1$, a contradiction.

```

rintro ⟨j, hj⟩
obtain ⟨k, hk⟩ := h
have hjk : j + 1 ≤ k := by
  nlinarith [hp.1]
nlinarith [Nat.mul_le_mul_left p hjk, hp.1]

```

□

Lemma 2.2.5. *For every natural number m , some prime exceeds m .*

```

theorem exists_prime_gt (m : ℕ) : ∃ p : ℕ, isPrime p ∧ m < p

```

Proof. The number $m! + 1$ exceeds one, so it has a prime divisor p .

```

obtain ⟨p, hp, hdiv, _⟩ :=
  exists_prime_divisor (m.factorial + 1) (by
    have := Nat.factorial_pos m
    omega)

```

If $p \leq m$, the factorial lemma gives $p \mid m!$. This contradicts the fact that a prime cannot divide both $m!$ and $m! + 1$. Thus $p > m$.

```

refine ⟨p, hp, ?_⟩
by contra h
obtain ⟨t, ht⟩ := factorial_multiple m p (lt_trans Nat.zero_lt_one hp.1)
  ↪ (by omega)
exact not_dvd_of_dvd_add_one p m.factorial hp hdiv ⟨t, ht⟩

```

□

Theorem 2.2.6. *There are infinitely many prime numbers.*

```

theorem infinite_primes : Set.Infinite {p : ℕ | isPrime p}

```

Proof. Every finite set of natural numbers is bounded. The preceding lemma produces a prime beyond any proposed bound, so the set of primes is infinite.

```

apply Set.infinite_of_forall_exists_gt
intro m
obtain ⟨p, hp, hgt⟩ := exists_prime_gt m
exact ⟨p, hp, hgt⟩

```

□

List the primes in increasing order as $p_1, p_2, \dots$. The general enumeration Nat.nth starts at zero, so $\text{Nat.nth isPrime } i$ denotes p_{i+1} . The infinitude just proved ensures that this enumeration always gives a prime, is increasing, and contains every prime. Put $P_0 = 1$ and $P_n = p_1 \cdots p_n$ for $n > 0$.

Lemma 2.2.7. *For every $n \geq 0$, the next prime satisfies $p_{n+1} \leq P_n + 1$.*

```

theorem nth_prime_le_prod_add_one (n : ℕ) :
  Nat.nth isPrime n ≤ (∏ i ∈ Finset.range n, Nat.nth isPrime i) + 1

```

Proof. All factors in P_n are positive, and the empty product is also positive. Thus $P_n + 1 > 1$.

```

classical
let P :=  $\prod$  i  $\in$  Finset.range n, Nat.nth isPrime i
have hP : 0 < P := by
  apply Finset.prod_pos
  intro i _
  exact lt_trans Nat.zero_lt_one (Nat.nth_mem_of_infinite
     $\hookrightarrow$  infinite_primes i).1

```

Choose a prime divisor $q \leq P_n + 1$ and let its enumeration index be k , so $q = p_{k+1}$. If $k < n$, then q is a factor of P_n , contradicting the consecutive-number lemma. Therefore $k \geq n$.

```

obtain (q, hq, hdiv, hqle) := exists_prime_divisor (P + 1) (by omega)
obtain (k, hk) := Nat.subset_range_nth (p := isPrime) (show q  $\in$ 
 $\hookrightarrow$  Set.ofPred isPrime from hq)
have hnk : n  $\leq$  k := by
  by_contra h
  have hmem : k  $\in$  Finset.range n := Finset.mem_range.mpr (by omega)
  have hfactor := Finset.prod_erase_mul (Finset.range n) (Nat.nth
     $\hookrightarrow$  isPrime) hmem
  have hqP : q  $\mid$  P := by
    refine ( $\prod$  i  $\in$  (Finset.range n).erase k, Nat.nth isPrime i, ?_)
    dsimp [P]
    rw [ $\leftarrow$  hfactor, hk, mul_comm]

exact not_dvd_of_dvd_add_one q P hq hdiv hqP

```

Increasing order gives $p_{n+1} \leq p_{k+1} = q \leq P_n + 1$.

```

calc
  Nat.nth isPrime n  $\leq$  Nat.nth isPrime k := Nat.nth_monotone
 $\hookrightarrow$  infinite_primes hnk
_ = q := hk
_  $\leq$  P + 1 := hqle

```

□

Lemma 2.2.8. *For every $n \geq 0$, we have $P_n < 2^{2^n}$.*

```

theorem prod_initial_primes_lt (n :  $\mathbb{N}$ ) :
  ( $\prod$  i  $\in$  Finset.range n, Nat.nth isPrime i) < 2 ^ (2 ^ n)

```

Proof. Induct on n . For $n = 0$, the assertion is $1 < 2$. If $P_n < B = 2^{2^n}$, integrality gives $P_n + 1 \leq B$, so the preceding bound yields $p_{n+1} \leq B$.

```

induction n with
| zero => simp
| succ n ih =>
  have hnext := nth_prime_le_prod_add_one n
  have hbound : Nat.nth isPrime n ≤ 2 ^ (2 ^ n) := by
    omega

```

It follows that $P_{n+1} = P_n p_{n+1} \leq P_n B < B^2 = 2^{2^{n+1}}$.

```

rw [Finset.prod_range_succ]
calc
  (∏ i ∈ Finset.range n, Nat.nth isPrime i) * Nat.nth isPrime n ≤
    (∏ i ∈ Finset.range n, Nat.nth isPrime i) * 2 ^ (2 ^ n) :=
    Nat.mul_le_mul_left _ hbound
  < 2 ^ (2 ^ n) * 2 ^ (2 ^ n) :=
    Nat.mul_lt_mul_of_pos_right ih (pow_pos (by omega) _)
  = 2 ^ (2 ^ (n + 1)) := by
    rw [pow_succ (2 : ℕ) n, pow_mul, pow_two]

```

□

Theorem 2.2.9. *For the n th prime p_n , with $n \geq 1$, we have $p_n \leq 2^{2^{n-1}}$.*

```

theorem nth_prime_le_double_exponential (n : ℕ) (hn : 0 < n) :
  Nat.nth isPrime (n - 1) ≤ 2 ^ (2 ^ (n - 1))

```

Proof. Write $n = m + 1$. The preceding lemmas give $p_n \leq P_m + 1$ and $P_m < 2^{2^m}$. Since these are integers, $P_m + 1 \leq 2^{2^m}$, proving the assertion.

```

obtain ⟨m, rfl⟩ := Nat.exists_eq_succ_of_ne_zero (Nat.ne_of_gt hn)
have hproduct := prod_initial_primes_lt m
have hnext := nth_prime_le_prod_add_one m
simp only [Nat.succ_sub_one] using
  (show Nat.nth isPrime m ≤ 2 ^ (2 ^ m) by omega)

```

□

2.3 The Goldbach Conjecture

Theorem 2.3.1. *For every positive integer n , there exist n consecutive composite integers.*

```
theorem exists_consecutive_composites (n : ℕ) (_hn : 0 < n) :
  ∃ M : ℕ, ∀ i : ℕ, i < n → isComposite (M + i)
```

Proof. Let $F = (n + 1)!$ and consider $F + 2, F + 3, \dots, F + n + 1$. For $0 \leq i < n$, the integer $i + 2$ divides F , so it also divides $F + i + 2$.

```
let F := (n + 1).factorial
refine ⟨F + 2, ?_⟩
intro i hi
obtain (t, ht) := factorial_multiple (n + 1) (i + 2) (by omega) (by
  ↪ omega)
have hF : 0 < F := Nat.factorial_pos (n + 1)
have hdiv : i + 2 | F + 2 + i := by
  refine ⟨t + 1, ?_⟩
  dsimp [F]
  rw [ht]
  ring
```

Each displayed integer exceeds one. Its divisor $i + 2$ is greater than one and is strictly smaller than $F + i + 2$, since $F > 0$. Thus it cannot be prime.

```
refine (by omega, ?_)
intro hp
rcases hp.2 (i + 2) hdiv with hone | heq

• omega
• omega
```

□

Lemma 2.3.2. *A product of two or more integers of the form $4k + 1$, with $k \in \mathbb{Z}$, again has that form. More generally, this holds for any finite list, including the empty product.*

```
theorem prod_four_mul_add_one (l : List ℤ)
  (hl : ∀ a ∈ l, ∃ k : ℤ, a = 4 * k + 1) :
  ∃ k : ℤ, l.prod = 4 * k + 1
```

Proof. Induct on the number of factors. The empty product is $1 = 4 \cdot 0 + 1$.

```
induction l with
| nil => exact ⟨0, by simp⟩
```

If the first factor is $4u + 1$ and the remaining product is $4v + 1$, their product is $4(4uv + u + v) + 1$.

```
| cons a l ih =>
  obtain ⟨u, hu⟩ := hl a List.mem_cons_self
  obtain ⟨v, hv⟩ := ih (fun b hb => hl b (List.mem_cons_of_mem a hb))
  refine ⟨4 * u * v + u + v, ?_⟩
  rw [List.prod_cons, hu, hv]
  ring
```

□

The notation $a \bmod 4$ below means the remainder upon division by 4; in Lean it is written $a \% 4$. The division algorithm gives $a = 4\lfloor a/4 \rfloor + (a \bmod 4)$ with remainder in $\{0, 1, 2, 3\}$.

Lemma 2.3.3. *A prime p is either 2, or has remainder 1 or 3 upon division by 4.*

```
theorem prime_eq_two_or_mod_four (p : ℕ) (hp : isPrime p) :
  p = 2 ∨ p % 4 = 1 ∨ p % 4 = 3
```

Proof. The remainder lies between zero and three. The desired conclusion already holds when it is one or three.

```
have hmod : p % 4 < 4 := Nat.mod_lt p (by omega)
by_cases hone : p % 4 = 1
  · exact Or.inr (Or.inl hone)
  · by_cases hthree : p % 4 = 3
    · exact Or.inr (Or.inr hthree)
```

For remainder zero, $p = 2(2\lfloor p/4 \rfloor)$; for remainder two, $p = 2(2\lfloor p/4 \rfloor + 1)$. Thus $2 \mid p$, and primality forces $p = 2$.

```
· have hdiv : 2 ∣ p := by
  rcases (show p % 4 = 0 ∨ p % 4 = 2 by omega) with hzero | htwo
    · exact ⟨2 * (p / 4), by omega⟩
    · exact ⟨2 * (p / 4) + 1, by omega⟩
  rcases hp.2 2 hdiv with h | h
    · omega
```

```
· exact 0r.inl h.symm
```

□

Lemma 2.3.4. *If $n \bmod 4 = 3$, then n has a prime divisor whose remainder upon division by 4 is also 3.*

```
theorem exists_prime_divisor_mod_four_three (n : ℕ) (hn : n % 4 = 3) :
  ∃ p : ℕ, isPrime p ∧ p % 4 = 3
```

Proof. Choose a prime factor list for n and suppose that no prime divisor has remainder three. The factor 2 is impossible, since n has odd remainder. By the preceding lemma every prime factor must therefore have remainder one.

```
classical
obtain ⟨l, hl, hprod⟩ := exists_prime_factors n (by omega)
by_contra h
push Not at h
have hone : ∀ p ∈ l, p % 4 = 1 := by
  intro p hp
  have hdiv : p | n := ⟨(l.erase p).prod, by rw [← hprod,
    ↪ List.prod_erase hp]⟩
  rcases prime_eq_two_or_mod_four p (hl p hp) with htwo | hrest

  · obtain ⟨k, hk⟩ := hdiv
    rw [htwo] at hk
    omega

  · exact hrest.resolve_right (h p (hl p hp) hdiv)
```

For each prime factor p , division by four gives $p = 4\lfloor p/4 \rfloor + 1$. View these equalities as integer equalities to apply the product lemma.

```
have hform : ∀ a ∈ l.map (fun p : ℕ => (p : ℤ)), ∃ k : ℤ, a = 4 * k + 1
  ↪ := by
    intro a ha
    obtain ⟨p, hp, rfl⟩ := List.mem_map.mp ha
    refine ⟨(p / 4 : ℕ), ?_⟩
    have heq : p = 4 * (p / 4) + 1 := by
      have := hone p hp
      omega
    exact_mod_cast heq
```

The product lemma makes the entire product equal to $4k + 1$ for some integer k . Since that product is n , this contradicts $n \bmod 4 = 3$.

```

obtain ⟨k, hk⟩ := prod_four_mul_add_one _ hform
have hcast : (l.map (fun p : ℕ => (p : ℤ))).prod = (n : ℤ) := by
  rw [← Nat.cast_list_prod, hprod]

rw [hcast] at hk
omega

```

□

Lemma 2.3.5. *For every natural number m , there is a prime $p > m$ of the form $p = 4k + 3$ with $k \in \mathbb{N}$.*

```

theorem exists_prime_mod_four_three_gt (m : ℕ) :
  ∃ p : ℕ, isPrime p ∧ m < p ∧ ∃ k : ℕ, p = 4 * k + 3

```

Proof. Put $N = 4m! - 1$. Since $m! \geq 1$, this is positive and has remainder three modulo four. Choose a prime divisor p with that remainder, and write $p = 4\lfloor p/4 \rfloor + 3$.

```

let N := 4 * m.factorial - 1
have hfac := Nat.factorial_pos m
have hN : N % 4 = 3 := by
  dsimp [N]
  omega
obtain ⟨p, hp, hdiv, hmod⟩ := exists_prime_divisor_mod_four_three N hN
refine ⟨p, hp, ?_, p / 4, by omega⟩

```

If $p \leq m$, write $m! = pt$. Then $N + 1 = 4m! = p(4t)$, so p divides $N + 1$ as well as N .

```

by_contra h
obtain ⟨t, ht⟩ := factorial_multiple m p (lt_trans Nat.zero_lt_one hp.1)
  ↪ (by omega)
have hsucc : p | N + 1 := by
  refine ⟨4 * t, ?_⟩
  have hNsucc : N + 1 = 4 * m.factorial := by
    dsimp [N]
    omega

  rw [hNsucc, ht]
  ring

```

This contradicts the consecutive-number lemma. Hence $p > m$.

```

exact not_dvd_of_dvd_add_one p N hp hsucc hdiv

```

□

Theorem 2.3.6. *There are infinitely many primes of the form $4k + 3$, with $k \in \mathbb{N}$.*

```
theorem infinite_primes_four_mul_add_three :
  Set.Infinite {p : ℕ | isPrime p ∧ ∃ k : ℕ, p = 4 * k + 3}
```

Proof. The preceding construction produces such a prime above every bound. A finite set of natural numbers is bounded, so this set of primes is infinite.

```
apply Set.infinite_of_forall_exists_gt
intro m
obtain ⟨p, hp, hgt, hform⟩ := exists_prime_mod_four_three_gt m
exact ⟨p, ⟨hp, hform⟩, hgt⟩
```

□

Theorem 2.3.7. *Let $p, d \in \mathbb{N}$ and $n > 2$. If every term of*

$$p, p + d, p + 2d, \dots, p + (n - 1)d$$

is prime, then every prime $q < n$ divides d . The statement includes the constant progression $d = 0$.

```
theorem prime_dvd_common_difference (p d n : ℕ) (hn : 2 < n)
  (hterms : ∀ i : ℕ, i < n → isPrime (p + i * d)) :
  ∀ q : ℕ, isPrime q → q < n → q ∣ d
```

Proof. The first term p is prime. Fix a prime $q < n$. If $d = 0$, then $q \mid d$ immediately.

```
have hp : isPrime p := by
  simpa only [zero_mul, add_zero] using hterms 0 (by omega)

intro q hq hqn
by_cases hd : d = 0

· exact ⟨0, by rw [hd, mul_zero]⟩
```

Suppose $d > 0$. We must have $n \leq p$: otherwise the term with index p occurs in the progression and equals $p + pd = p(1 + d)$. Its divisor $p > 1$ is smaller than the term, contradicting primality.

```

• have hnp : n ≤ p := by
  by_contra h
  have hterm := hterms p (by omega)
  have hdiv : p ∣ p + p * d := (1 + d, by ring)
  rcases hterm.2 p hdiv with hone | heq

• have := hp.1
  omega
• have hdpos : 0 < d := by
  omega
  nlinarith [hp.1]

```

Assume, seeking a contradiction, that q does not divide d . The gcd of q and d divides the prime q , so it is 1 or q . The latter would imply $q \mid d$; hence the gcd is 1.

```

by_contra hqd
have hqpos : 0 < q := lt_trans Nat.zero_lt_one hq.1
have hqne : (q : ℤ) ≠ 0 := by
  exact_mod_cast (Nat.ne_of_gt hqpos)
obtain {_, hgq, hgd, _, _} :=
  Chapter01.gcd_spec (q : ℤ) (d : ℤ) (Or.inl hqne)
have hg : Chapter01.gcd (q : ℤ) (d : ℤ) = 1 := by
  rcases hq.2 _ (Int.natCast_dvd_natCast.mp hgq) with hone | heq

• exact hone
• rw [heq] at hgd
  exact (hqd (Int.natCast_dvd_natCast.mp hgd)).elim

```

Choose integers r, s with $1 = qr + ds$. Divide $-ps$ by q , obtaining $-ps = aq + i$ with $0 \leq i < q$.

```

obtain {r, s, hrs} :=
  (Chapter01.gcd_eq_one_iff_exists_mul_add_mul (q : ℤ) (d : ℤ) (Or.inl
    ↪ hqne)).mp hg
obtain {a, i}, {hrem, hi}, _ :=
  Chapter01.existsUnique_quotient_remainder (-(p : ℤ) * s) (q, hqpos)
change -(p : ℤ) * s = a * (q : ℤ) + (i : ℤ) at hrem
change i < q at hi

```

These two equalities give $p + id = q(pr - da)$. Thus $q \mid p + id$.

```

have hdiv : q ∣ p + i * d := by
  apply Int.natCast_dvd_natCast.mp
  refine ((p : ℤ) * r - (d : ℤ) * a, ?_)
  push_cast
  nlinarith [congrArg (fun z => (p : ℤ) * z) hrs,
    congrArg (fun z => (d : ℤ) * z) hrem]

```

Since $i < q < n$, this term is prime. Its prime divisor q must equal the term. But $p + id \geq p \geq n > q$, a contradiction. Therefore $q \mid d$.

```
have hterm := hterms i (lt_trans hi hqn)
rcases hterm.2 q hdiv with hone | heq
```

- have := hq.1
 - omega
 - omega

□

Chapter 3

The Theory of Congruences

3.1 Basic Properties of Congruence

Theorem 3.1.1. *For integers n, a, b , congruence $a \equiv b \pmod{n}$ is equivalent to $n \mid a - b$.*

```
theorem modEq_iff_dvd_sub (n a b : ℤ) : a ≡ b [ZMOD n] ↔ n ∣ a - b
```

Proof. The remainder formulation gives divisibility of $b - a$. Negating a divisibility witness reverses the sign of this difference.

```
rw [Int.modEq_iff_dvd]
constructor

· rintro ⟨k, hk⟩
  exact ⟨-k, by linarith⟩

· rintro ⟨k, hk⟩
  exact ⟨-k, by linarith⟩
```

□

Theorem 3.1.2. *Integers are congruent modulo n exactly when their remainders on division by n agree.*

```
theorem modEq_iff_emod_eq (n a b : ℤ) : a ≡ b [ZMOD n] ↔ a % n = b % n
```

Proof. This is the remainder definition of the congruence relation.

```
Iff.rfl
```

□

Theorem 3.1.3. *For all integers n, a , one has $a \equiv a \pmod{n}$.*

```
theorem modEq_refl (n a : ℤ) : a ≡ a [ZMOD n]
```

Proof. The remainder of a equals itself.

```
rfl
```

□

Theorem 3.1.4. *If $a \equiv b \pmod{n}$, then $b \equiv a \pmod{n}$.*

```
theorem modEq_symm {n a b : ℤ} (h : a ≡ b [ZMOD n]) : b ≡ a [ZMOD n]
```

Proof. Reverse the equality of remainders.

```
Eq.symm h
```

□

Theorem 3.1.5. *If $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$, then $a \equiv c \pmod{n}$.*

```
theorem modEq_trans {n a b c : ℤ} (h : a ≡ b [ZMOD n]) (h' : b ≡ c [ZMOD  
↪ n]) :  
  a ≡ c [ZMOD n]
```

Proof. Transitivity of equality applies to the three remainders.

```
Eq.trans h h'
```

□

Theorem 3.1.6. *If $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$, then $a + c \equiv b + d \pmod{n}$.*

```
theorem modEq_add {n a b c d : ℤ} (h : a ≡ b [ZMOD n]) (h' : c ≡ d [ZMOD
↪ n]) :
  a + c ≡ b + d [ZMOD n]
```

Proof. Write $a - b = nu$ and $c - d = nv$. The difference of the sums is $n(u + v)$.

```
obtain ⟨u, hu⟩ := (modEq_iff_dvd_sub n a b).1 h
obtain ⟨v, hv⟩ := (modEq_iff_dvd_sub n c d).1 h'

exact (modEq_iff_dvd_sub n _ _).2 ⟨u + v, by linarith⟩
```

□

Theorem 3.1.7. *Under the same hypotheses, $ac \equiv bd \pmod{n}$.*

```
theorem modEq_mul {n a b c d : ℤ} (h : a ≡ b [ZMOD n]) (h' : c ≡ d [ZMOD
↪ n]) :
  a * c ≡ b * d [ZMOD n]
```

Proof. The identity $ac - bd = (a - b)c + b(c - d)$ expresses the difference as $n(uc + bv)$.

```
obtain ⟨u, hu⟩ := (modEq_iff_dvd_sub n a b).1 h
obtain ⟨v, hv⟩ := (modEq_iff_dvd_sub n c d).1 h'

exact (modEq_iff_dvd_sub n _ _).2 ⟨u * c + b * v, by
  nlinarith [congrArg (fun z => z * c) hu, congrArg (fun z => z * b)
    ↪ hv]⟩
```

□

Theorem 3.1.8. *If $a \equiv b \pmod{n}$, then $a^k \equiv b^k \pmod{n}$ for every natural number k , including zero.*

```
theorem modEq_pow {n a b : ℤ} (h : a ≡ b [ZMOD n]) (k : ℕ) :
  a ^ k ≡ b ^ k [ZMOD n]
```

Proof. At exponent zero both sides are one.

```

induction k with
| zero =>
  simp only [pow_zero]
  exact modEq_refl n 1

```

Multiply the induction hypothesis by the original congruence to pass to the next exponent.

```

| succ k ih => simp only [pow_succ] using modEq_mul ih h

```

□

Theorem 3.1.9. *For integers n, a , one has $a \equiv 0 \pmod{n}$ if and only if $n \mid a$.*

```

theorem modEq_zero_iff (n a : ℤ) : a ≡ 0 [ZMOD n] ↔ n ∣ a

```

Proof. Set the second integer equal to zero in the divisibility characterization.

```

simp only [sub_zero] using modEq_iff_dvd_sub n a 0

```

□

Theorem 3.1.10. *For every positive modulus n and integer a , there is exactly one natural number $r < n$ with $a \equiv r \pmod{n}$.*

```

theorem existsUnique_residue (n : ℕ+) (a : ℤ) :
  ∃! r : ℕ, r < n ∧ a ≡ (r : ℤ) [ZMOD (n : ℤ)]

```

Proof. The division algorithm gives $a = qn + r$ with $0 \leq r < n$. The identity $a - r = qn$ shows that r is a congruent representative.

```

obtain ⟨(q, r), ⟨ha, hr⟩, hu⟩ :=
  ↪ Chapter01.existsUnique_quotient_remainder a n

refine ⟨r, ⟨hr, ?_⟩, ?_⟩

· apply (modEq_iff_dvd_sub _ _ _).2
  exact ⟨q, by
    change a = q * (n : ℤ) + r at ha
    nlinarith)

```

Conversely, any congruent representative s in this interval supplies a quotient through $a - s = kn$. Uniqueness of the quotient–remainder pair forces $s = r$.

```

· intro s hs

obtain ⟨k, hk⟩ := (modEq_iff_dvd_sub _ _ _).1 hs.2

have hp : (k, s) = (q, r) := hu (k, s) ⟨by
  dsimp
  nlinarith, hs.1⟩

exact congrArg Prod.snd hp

```

□

Definition 3.1.11. A family of n integers is a *complete residue system* modulo n if every integer is congruent to exactly one member of the family.

```

def IsCompleteResidueSystem (n : ℕ) (a : Fin n → ℤ) : Prop :=
  ∀ x : ℤ, ∃! i : Fin n, x ≡ a i [ZMOD (n : ℤ)]

```

Theorem 3.1.12. For $n > 0$, a family $(a_i)_{i \in \{0, \dots, n-1\}}$ is a complete residue system precisely when congruent members have equal indices.

```

theorem complete_residue_system_iff (n : ℕ) (hn : 0 < n) (a : Fin n → ℤ)
↔ :
  IsCompleteResidueSystem n a ↔ ∀ i j : Fin n, a i ≡ a j [ZMOD (n :
  ℤ)] → i = j

```

Proof. Uniqueness of a representative immediately gives pairwise incongruence.

```

let : NeZero n := ⟨ne_of_gt hn⟩

constructor

· intro h i j hij

obtain ⟨k, hk, hu⟩ := h (a i)

exact (hu i (modEq_refl _ _)).trans (hu j hij).symm

```

Conversely, pairwise incongruence makes the map from the n indices to the n residue classes injective.

```

• intro h

have hinj : Function.Injective (fun i : Fin n => (a i : ZMod n)) := by
  intro i j heq
  exact h i j ((ZMod.intCast_eq_intCast_iff _ _ n).1 heq)

```

Finite counting makes this map surjective as well. Every integer therefore has a representative, and injectivity makes it unique.

```

have hsurj := (Finite.injective_iff_surjective_of_equiv (ZMod.finEquiv
  ↪ n).toEquiv).1 hinj

intro x

obtain ⟨i, hi⟩ := hsurj (x : ZMod n)

refine ⟨i, (ZMod.intCast_eq_intCast_iff _ _ n).1 hi.symm, ?_⟩

intro j hj
apply hinj
exact ((ZMod.intCast_eq_intCast_iff _ _ n).2 hj).symm.trans hi.symm

```

□

Theorem 3.1.13. *If $\gcd(n, c) = 1$ and $ca \equiv cb \pmod{n}$, then $a \equiv b \pmod{n}$.*

```

theorem modEq_cancel {n c a b : ℤ} (hc : Chapter01.gcd n c = 1)
  (h : c * a ≡ c * b [ZMOD n]) : a ≡ b [ZMOD n]

```

Proof. The hypothesis says $n \mid c(a - b)$. The previously proved coprime form of Euclid's lemma gives $n \mid a - b$.

```

apply (modEq_iff_dvd_sub n a b).2
apply Chapter01.dvd_of_dvd_mul_of_coprime n c (a - b) hc
simp only [mul_sub] using (modEq_iff_dvd_sub n _ _).1 h

```

□

Theorem 3.1.14. *Let $n \neq 0$ and $g = \gcd(n, c)$. If $ca \equiv cb \pmod{n}$, then $a \equiv b \pmod{n/g}$.*

```
theorem modEq_cancel_gcd {n c a b : ℤ} (hn : n ≠ 0)
  (h : c * a ≡ c * b [ZMOD n]) :
  a ≡ b [ZMOD (n / (Chapter01.gcd n c : ℤ))]
```

Proof. Write $n = gu$ and $c = gv$. Dividing the pair by its positive gcd gives $\gcd(u, v) = 1$.

```
obtain ⟨hd, ⟨u, hu⟩, ⟨v, hv⟩, _, _⟩ := Chapter01.gcd_spec n c (0r.inl
  ↪ hn)

have hcop := Chapter01.gcd_div_gcd_eq_one n c (0r.inl hn)
generalize hg : Chapter01.gcd n c = g at *

have hdne : (g : ℤ) ≠ 0 := by
  exact_mod_cast ne_of_gt hd

have hnu : n / (g : ℤ) = u := by
  rw [hu]
  exact Int.mul_ediv_cancel_left u hdne

have hcv : c / (g : ℤ) = v := by
  rw [hv]
  exact Int.mul_ediv_cancel_left v hdne
rw [hnu, hcv] at hcop
```

A witness for $n \mid c(a - b)$ yields $guk = gv(a - b)$. Cancel the nonzero factor g , then apply Euclid's lemma to $u \mid v(a - b)$.

```
obtain ⟨k, hk⟩ := (modEq_iff_dvd_sub n _ _).1 h

have hdiv : u ∣ v * (a - b) := by
  refine ⟨k, ?_⟩
  apply mul_left_cancel₀ hdne
calc
  (g : ℤ) * (v * (a - b)) = c * a - c * b := by
    rw [hv]
    ring
  _ = (g : ℤ) * (u * k) := by
    rw [hk, hu]
    ring
rw [hnu]
exact (modEq_iff_dvd_sub _ _ _).2
  (Chapter01.dvd_of_dvd_mul_of_coprime u v (a - b) hcop hdiv)
```

□

Theorem 3.1.15. *For a prime p , the congruence $ab \equiv 0 \pmod{p}$ implies $a \equiv 0 \pmod{p}$ or $b \equiv 0 \pmod{p}$.*

```
theorem modEq_mul_zero_prime (p : ℕ) (hp : Chapter02.isPrime p) (a b :
  ↪ ℤ)
  (h : a * b ≡ 0 [ZMOD (p : ℤ)]) :
  a ≡ 0 [ZMOD (p : ℤ)] ∨ b ≡ 0 [ZMOD (p : ℤ)]
```

Proof. Translate the congruences into divisibility and apply the prime-divides-a-product theorem from Chapter 2.

```
rw [modEq_zero_iff] at h ⊢
rw [modEq_zero_iff]
exact Chapter02.prime_dvd_mul p hp a b h
```

□

3.2 Binary and Decimal Representations of Integers

Definition 3.2.1. For a list of natural digits written with the units digit first, define its base- b value recursively by $V_b([]) = 0$ and $V_b(d :: D) = d + bV_b(D)$.

```
def digitValue (b : ℕ) : List ℕ → ℕ
| [] => 0
| d :: ds => d + b * digitValue b ds
```

Definition 3.2.2. A digit list is *canonical* when every digit is less than b and its last, most significant digit is nonzero. The empty list is the canonical representation of zero.

```
def CanonicalDigits (b : ℕ) : List ℕ → Prop
| [] => True
| d :: ds => d < b ∧ (ds = [] → d ≠ 0) ∧ CanonicalDigits b ds
```

Theorem 3.2.3. *For each natural number n and base $b > 1$, there is exactly one canonical digit list with value n . For positive n , this list is nonempty.*

```
theorem existsUnique_digits (b : ℕ) (hb : 1 < b) (n : ℕ) :
  ∃! ds : List ℕ, CanonicalDigits b ds ∧ digitValue b ds = n
```

Proof. Use strong induction on n . Zero has the empty expansion. For positive n , division by b gives $n = bq + r$ with $0 \leq r < b$.

```
have hex : ∀ n : ℕ, ∃ ds : List ℕ, CanonicalDigits b ds ∧ digitValue b
  ↪ ds = n := by
  intro n
  induction n using Nat.strong_induction_on with
  | h n ih =>
    by_cases hn : n = 0

    · exact ([], trivial, by exact hn.symm)

    · obtain (<{q, r}, <{heq, hr}, _) :=
      Chapter01.existsUnique_quotient_remainder n (b, by omega)
      change (n : ℤ) = q * b + r at heq
      change r < b at hr
```

Since $b > 1$ and $n > 0$, the quotient satisfies $0 \leq q < n$. Its natural-number representation can therefore be expanded by induction.

```
have hb' : (1 : ℤ) < b := by
  exact_mod_cast hb
have hr' : (r : ℤ) < b := by
  exact_mod_cast hr
have hq : 0 ≤ q := by
  nlinarith
have hnpos : (0 : ℤ) < n := by
  omega
have hqn : q < n := by
  nlinarith
have hlt : q.toNat < n := by
  omega

obtain (ds, hds, hval) := ih q.toNat hlt
```

Prepend r to this expansion. The digit is less than b , and if the tail is empty, positivity of n ensures $r \neq 0$.

```

refine (r :: ds, (hr, ?_, hds), ?_)

· intro hnil hrzero
  rw [hnil] at hval
  change 0 = q.toNat at hval

  have : q = 0 := by
    omega
  rw [this, hrzero] at heq
  simp only [zero_mul, Nat.cast_zero, add_zero] at heq
  exact hn (by exact_mod_cast heq)

```

The positional value of the new list is $r + bq = n$, completing the existence proof.

```

· change r + b * digitValue b ds = n
  rw [hval]

  have hc : (q.toNat : ℤ) = q := Int.toNat_of_nonneg hq
  exact_mod_cast (show (r : ℤ) + (b : ℤ) * q.toNat = n by
    rw [hc]
    nlinarith)

```

The construction gives existence. For uniqueness, equality of two positional values gives equal quotient–remainder pairs on division by b : the first digits agree and the tails have equal values. Repeating this argument proves equality of the complete lists.

```

obtain (ds, hd, hv) := hex n

exact (ds, (hd, hv), fun es he => digitValue_injective b hb es ds he.1
  ↪ hd (he.2.trans hv.symm))

```

□

Theorem 3.2.4. *Every natural number has a unique canonical base-two expansion. Since each digit is less than two, the digits are zero and one.*

```

theorem existsUnique_binary_digits (n : ℕ) :
  ∃! ds : List ℕ, CanonicalDigits 2 ds ∧ digitValue 2 ds = n

```

Proof. Specialize the positional representation theorem to base two.

```

existsUnique_digits 2 (by decide) n

```

□

Definition 3.2.5. For integer coefficients written in increasing order of degree, define $P_b([]) = 0$ and $P_b(d :: D) = d + bP_b(D)$. Thus the same notation evaluates polynomials and positional digit lists.

```
def positionalValue (b : ℤ) : List ℤ → ℤ
| [] => 0
| d :: ds => d + b * positionalValue b ds
```

Theorem 3.2.6. An integer polynomial takes congruent values at congruent arguments: if $a \equiv b \pmod{n}$, then $P(a) \equiv P(b) \pmod{n}$.

```
theorem positionalValue_modEq (n a b : ℤ) (h : a ≡ b [ZMOD n]) (ds :
  → List ℤ) :
  positionalValue a ds ≡ positionalValue b ds [ZMOD n]
```

Proof. Induct on the coefficient list. The zero polynomial gives the base case. The recursive step adds a common constant coefficient to congruent products.

```
induction ds with
| nil => exact modEq_refl n 0
| cons d ds ih => exact modEq_add (modEq_refl n d) (modEq_mul h ih)
```

□

Definition 3.2.7. The alternating sum starts with the units digit: $A([]) = 0$ and $A(d :: D) = d - A(D)$.

```
def alternatingSum : List ℤ → ℤ
| [] => 0
| d :: ds => d - alternatingSum ds
```

Theorem 3.2.8. A decimal positional value is divisible by 9 precisely when the sum of its digits is divisible by 9. The identity also holds for arbitrary integer coefficients.

```
theorem nine_dvd_iff_digit_sum (ds : List ℤ) :
  9 | positionalValue 10 ds ↔ 9 | ds.sum
```

Proof. Since $10 \equiv 1 \pmod{9}$, substitute one for ten in the positional polynomial. Its value at one is the digit sum.

```
have h := positionalValue_modEq 9 10 1 (by decide) ds
rw [positionalValue_one] at h
exact dvd_iff_of_modEq h
```

□

Theorem 3.2.9. *A decimal positional value is divisible by 3 precisely when the sum of its digits is divisible by 3. The identity also holds for arbitrary integer coefficients.*

```
theorem three_dvd_iff_digit_sum (ds : List ℤ) :
  3 | positionalValue 10 ds ↔ 3 | ds.sum
```

Proof. The same argument uses $10 \equiv 1 \pmod{3}$.

```
have h := positionalValue_modEq 3 10 1 (by decide) ds
rw [positionalValue_one] at h
exact dvd_iff_of_modEq h
```

□

Theorem 3.2.10. *A decimal positional value is divisible by 11 precisely when the alternating sum of its digits, starting with the units digit is divisible by 11. The identity also holds for arbitrary integer coefficients.*

```
theorem eleven_dvd_iff_alternating_sum (ds : List ℤ) :
  11 | positionalValue 10 ds ↔ 11 | alternatingSum ds
```

Proof. Use $10 \equiv -1 \pmod{11}$. Evaluation at minus one is the alternating sum.

```

have h := positionalValue_modEq 11 10 (-1) (by decide) ds
rw [positionalValue_neg_one] at h
exact dvd_iff_of_modEq h

```

□

Theorem 3.2.11. *A nonempty decimal expansion is divisible by two exactly when its units digit is even.*

```

theorem two_dvd_iff_last_digit (d : ℤ) (ds : List ℤ) :
  2 | positionalValue 10 (d :: ds) ↔ 2 | d

```

Proof. All other terms contain a factor of ten, which is divisible by two.

```

dvd_positionalValue_cons_iff 2 10 d ds (by decide)

```

□

Theorem 3.2.12. *For a units digit $0 \leq d < 10$, a decimal expansion is divisible by five exactly when $d = 0$ or $d = 5$.*

```

theorem five_dvd_iff_last_digit (d : ℤ) (ds : List ℤ) (hd : 0 ≤ d ∧ d <
  10) :
  5 | positionalValue 10 (d :: ds) ↔ d = 0 ∨ d = 5

```

Proof. Remove the terms divisible by ten. A multiple of five in the interval from zero to nine is zero or five.

```

rw [dvd_positionalValue_cons_iff 5 10 d ds (by decide)]
constructor

· rintro ⟨k, hk⟩
  omega

```

Conversely, both zero and five are divisible by five.

```

· rintro (rfl | rfl) <=> decide

```

□

Theorem 3.2.13. *A decimal positional value with units digit d and tens digit e is divisible by four exactly when $d + 10e$ is divisible by four.*

```
theorem four_dvd_iff_last_two_digits (d e : ℤ) (ds : List ℤ) :
  4 | positionalValue 10 (d :: e :: ds) ↔ 4 | d + 10 * e
```

Proof. The remaining contribution is one hundred times the tail value, hence is divisible by four.

```
apply dvd_iff_of_modEq
exact (modEq_iff_dvd_sub _ _ _).2
  (25 * positionalValue 10 ds, by
    simp only [positionalValue]
    ring)
```

□

Theorem 3.2.14. If $\gcd(10, w) = 1$ and d, e are distinct decimal digits, then $wd \not\equiv we \pmod{10}$.

```
theorem detects_single_digit_error (w d e : ℤ) (hw : Chapter01.gcd 10 w
  ↔ = 1)
  (hd : 0 ≤ d ∧ d < 10) (he : 0 ≤ e ∧ e < 10) (hne : d ≠ e) :
  ¬w * d ≡ w * e [ZMOD 10]
```

Proof. Otherwise coprime cancellation would give $10 \mid d - e$. Since $-10 < d - e < 10$, this forces equality of the digits.

```
intro h
obtain (k, hk) := (modEq_iff_dvd_sub _ _ _).1 (modEq_cancel hw h)
omega
```

□

3.3 Linear Congruences and the Chinese Remainder Theorem

Theorem 3.3.1. For integers n, a, b, x , the congruence $ax \equiv b \pmod{n}$ is equivalent to the existence of an integer y with $ax + ny = b$.

```
theorem linear_modEq_iff (n a b x : ℤ) :
  a * x ≡ b [ZMOD n] ↔ ∃ y : ℤ, a * x + n * y = b
```

Proof. Negate the witness for $n \mid ax - b$ to obtain y , and reverse this construction in the other direction.

```
rw [modEq_iff_dvd_sub]
constructor

· rintro ⟨k, hk⟩
  exact ⟨-k, by nlinarith⟩

· rintro ⟨y, hy⟩
  exact ⟨-y, by nlinarith⟩
```

□

Theorem 3.3.2. *The congruence $ax \equiv b \pmod{n}$ is solvable exactly when $\gcd(a, n) \mid b$.*

```
theorem exists_linear_modEq_iff (n a b : ℤ) :
  (∃ x : ℤ, a * x ≡ b [ZMOD n]) ↔ (Chapter01.gcd a n : ℤ) ∣ b
```

Proof. Use the preceding translation and the solvability criterion for linear Diophantine equations.

```
simp only [linear_modEq_iff]
exact Chapter01.exists_mul_add_mul_eq_iff a n b
```

□

Theorem 3.3.3. *Let $n \neq 0$, $g = \gcd(a, n)$, and let x_0 solve $ax \equiv b \pmod{n}$. The solutions are precisely $x = x_0 + (n/g)t$ with $t \in \mathbb{Z}$.*

```
theorem linear_modEq_iff_eq_add (n a b x₀ x : ℤ) (hn : n ≠ 0)
  (h₀ : a * x₀ ≡ b [ZMOD n]) :
  a * x ≡ b [ZMOD n] ↔ ∃ t : ℤ,
    x = x₀ + (n / (Chapter01.gcd a n : ℤ)) * t
```

Proof. Choose a companion y_0 with $ax_0 + ny_0 = b$. For any other solution, the previously proved parametrization of Diophantine solutions gives the displayed first coordinate.

```

obtain ⟨y0, hy0⟩ := (linear_modEq_iff n a b x0).1 h0
rw [linear_modEq_iff]
constructor

· rintro ⟨y, hy⟩

obtain ⟨t, ht, _⟩ :=
  (Chapter01.mul_add_mul_eq_iff a n b x0 y0 x y (0r.inr hn) hy0).1 hy

exact ⟨t, ht⟩

```

Conversely, accompany any such x by $y_0 - (a/g)t$, where $g = \gcd(a, n)$. The parametrization then verifies the original equation.

```

· rintro ⟨t, ht⟩
refine ⟨y0 - (a / (Chapter01.gcd a n : ℤ)) * t, ?_⟩
exact (Chapter01.mul_add_mul_eq_iff a n b x0 y0 x _ (0r.inr hn) hy0).2
  ⟨t, ht, rfl⟩

```

□

Theorem 3.3.4. *For $n > 0$, a solvable linear congruence has exactly $g = \gcd(a, n)$ incongruent solutions. They are represented by $x_0 + (n/g)t$ for $0 \leq t < g$, and each solution belongs to exactly one of these classes.*

```

theorem linear_solution_classes (n a b x0 : ℤ) (hn : 0 < n)
  (h0 : a * x0 ≡ b [ZMOD n]) :
  (∀ t : Fin (Chapter01.gcd a n),
    a * (x0 + (n / (Chapter01.gcd a n : ℤ)) * t.val) ≡ b [ZMOD n]) ∧
  (∀ x : ℤ, a * x ≡ b [ZMOD n] →
    ∃! t : Fin (Chapter01.gcd a n),
      x ≡ x0 + (n / (Chapter01.gcd a n : ℤ)) * t.val [ZMOD n])

```

Proof. Write $n = gu$ with $u > 0$. Every proposed representative is a solution by the parametrization.

```

have hnne := ne_of_gt hn

obtain ⟨hg, _, ⟨u, hu⟩, _, _⟩ := Chapter01.gcd_spec a n (0r.inr hnne)

have hgne : (Chapter01.gcd a n : ℤ) ≠ 0 := by
  exact_mod_cast ne_of_gt hg

```

```

have hquot : n / (Chapter01.gcd a n : ℤ) = u := by
  calc
    n / (Chapter01.gcd a n : ℤ) =
      ((Chapter01.gcd a n : ℤ) * u) / (Chapter01.gcd a n : ℤ) :=
        congrArg (fun z => z / (Chapter01.gcd a n : ℤ)) hu
    _ = u := Int.mul_ediv_cancel_left u hgne

have hupos : 0 < u := by
  have hgz : (0 : ℤ) < Chapter01.gcd a n := by
    exact_mod_cast hg
  nlinarith

constructor

· intro t
  exact (linear_modEq_iff_eq_add n a b x₀ _ hnne h₀).2 ⟨t.val, rfl⟩

```

For an arbitrary solution, divide its integer parameter by $g = \gcd(a, n)$ and retain the remainder r . The discarded multiple of g changes x by a multiple of n , so r supplies a representative.

```

· intro x hx

obtain ⟨t, ht⟩ := (linear_modEq_iff_eq_add n a b x₀ x hnne h₀).1 hx
obtain ⟨⟨q, r⟩, ⟨htr, hr⟩, _⟩ :=
  Chapter01.existsUnique_quotient_remainder t ⟨Chapter01.gcd a n, hg⟩

change t = q * (Chapter01.gcd a n : ℤ) + r at htr
change r < Chapter01.gcd a n at hr

have hrep : x ≡ x₀ + (n / (Chapter01.gcd a n : ℤ)) * r [ZMOD n] := by
  apply (modEq_iff_dvd_sub _ _ _).2
  refine ⟨q, ?_⟩
  rw [hquot] at ht
  rw [ht]
  nlinarith [congrArg (fun z => z * q) hu, congrArg (fun z => u * z)
    ↪ htr]

```

If another remainder s represents the same class, subtracting the two congruences and cancelling $u > 0$ makes $s - r$ a multiple of g .

```

refine (<r, hr>, hrep, ?_)
intro s hs

have hdiff := modEq_trans (modEq_symm hs) hrep

obtain <k, hk> := (modEq_iff_dvd_sub _ _ _).1 hdiff
rw [hquot] at hk

have he : (s.val : ℤ) - r = (Chapter01.gcd a n : ℤ) * k := by
  apply mul_left_cancel0 (ne_of_gt hupos)
  calc
    u * ((s.val : ℤ) - r) = (x0 + u * s.val) - (x0 + u * r) := by ring
    _ = u * ((Chapter01.gcd a n : ℤ) * k) := by
      rw [hk]
      nlinarith [congrArg (fun z => z * k) hu]

```

Both remainders lie between zero and $g - 1$. Their difference is therefore strictly between $-g$ and g , forcing that multiple to be zero.

```

have hsbound : (s.val : ℤ) < Chapter01.gcd a n := by
  exact_mod_cast s.isLt

have hrbound : (r : ℤ) < Chapter01.gcd a n := by
  exact_mod_cast hr

have hgz : (0 : ℤ) < Chapter01.gcd a n := by
  exact_mod_cast hg

have hkzero : k = 0 := by
  rcases lt_trichotomy k 0 with hneg | hzero | hpos

  · have : k ≤ -1 := by
      omega
      nlinarith [Int.natCast_nonneg s.val, Int.natCast_nonneg r]

  · exact hzero

  · have : 1 ≤ k := hpos
      nlinarith [Int.natCast_nonneg s.val, Int.natCast_nonneg r]

```

Thus $s = r$, proving uniqueness of the representative.

```

apply Fin.ext

have hval : (s.val : ℤ) = r := by
  rw [hkzero, mul_zero] at he
  linarith
exact_mod_cast hval

```

□

Definition 3.3.5. An integer u is a multiplicative inverse of a modulo n when $au \equiv 1 \pmod{n}$.

```

def IsModularInverse (n a u : ℤ) : Prop := a * u ≡ 1 [ZMOD n]

```

Theorem 3.3.6. If $\gcd(a, n) = 1$, an inverse of a modulo n exists.

```

theorem exists_modularInverse (n a : ℤ) (ha : Chapter01.gcd a n = 1) :
  ∃ u : ℤ, IsModularInverse n a u

```

Proof. Apply the linear congruence existence criterion with right-hand side one.

```

apply (exists_linear_modEq_iff n a 1).2
rw [ha, Nat.cast_one]

```

□

Theorem 3.3.7. For $n \neq 0$ and $\gcd(a, n) = 1$, every linear congruence $ax \equiv b \pmod{n}$ has one solution class modulo n .

```

theorem exists_unique_linear_modEq (n a b : ℤ) (hn : n ≠ 0)
  (ha : Chapter01.gcd a n = 1) :
  ∃ x₀ : ℤ, a * x₀ ≡ b [ZMOD n] ∧
  ∀ x : ℤ, a * x ≡ b [ZMOD n] ↔ x ≡ x₀ [ZMOD n]

```

Proof. Existence follows because one divides every integer.

```

obtain (x₀, hx₀) := (exists_linear_modEq_iff n a b).2 (by
  rw [ha, Nat.cast_one]
  exact (b, by ring))

```

The solution step $n/\gcd(a, n)$ is now n , so the parametrization is exactly one congruence class.

```

refine (x₀, hx₀, ?_)
intro x
rw [linear_modEq_iff_eq_add n a b x₀ x hn hx₀, ha, Nat.cast_one,
  ↪ Int.ediv_one,
  modEq_iff_dvd_sub]

constructor

· rintro (t, ht)
  exact (t, by linarith)

· rintro (t, ht)
  exact (t, by linarith)

```

□

Theorem 3.3.8. *If $au \equiv 1 \pmod{n}$, then $ax \equiv b \pmod{n}$ is equivalent to $x \equiv ub \pmod{n}$.*

```

theorem linear_modEq_iff_inverse (n a b u x : ℤ) (hu : IsModularInverse
  ↪ n a u) :
  a * x ≡ b [ZMOD n] ↔ x ≡ u * b [ZMOD n]

```

Proof. Suppose $ax \equiv b \pmod{n}$. Multiplication by u gives $u(ax) \equiv ub$, while the inverse relation gives $(au)x \equiv x$.

```

constructor

· intro hx

  have h₁ := modEq_mul (modEq_refl n u) hx
  have h₂ := modEq_mul hu (modEq_refl n x)

```

Since $u(ax) = (au)x$, these congruences imply $x \equiv ub \pmod{n}$.

```

have heq : u * (a * x) = (a * u) * x := by
  ring
  rw [heq] at h1

  exact modEq_trans (modEq_symm (by simp only [one_mul] using h2)) h1

```

Conversely, suppose $x \equiv ub \pmod{n}$. Multiplication by a gives $ax \equiv a(ub)$, and the inverse relation gives $(au)b \equiv b$.

```

· intro hx

have h1 := modEq_mul (modEq_refl n a) hx
have h2 := modEq_mul hu (modEq_refl n b)

```

Rewriting $a(ub) = (au)b$ yields $ax \equiv b \pmod{n}$.

```

have heq : a * (u * b) = (a * u) * b := by
  ring
  rw [heq] at h1

  exact modEq_trans h1 (by simp only [one_mul] using h2)

```

□

Theorem 3.3.9. *Let $(n_i)_{i \in S}$ be a finite family of nonzero, pairwise coprime integer moduli. For any integers a_i , the system $x \equiv a_i \pmod{n_i}$ has a solution, and all solutions form one class modulo $\prod_{i \in S} n_i$. The empty system gives the unique class modulo one.*

```

theorem chinese_remainder {ι : Type*} (s : Finset ι) (n a : ι → ℤ)
  (hn : ∀ i ∈ s, n i ≠ 0)
  (hpair : ∀ i ∈ s, ∀ j ∈ s, i ≠ j → Chapter01.gcd (n i) (n j) = 1) :
  ∃ x : ℤ, (∀ i ∈ s, x ≡ a i [ZMOD n i]) ∧
    ∀ y : ℤ, (∀ i ∈ s, y ≡ a i [ZMOD n i]) ↔ y ≡ x [ZMOD ∏ i ∈ s, n i]

```

Proof. For each i , form the complementary product $N_i = \prod_{j \neq i} n_j$. Pairwise coprimality and the product form of Bezout's identity give an inverse u_i of N_i modulo n_i . Set $x = \sum_i a_i N_i u_i$.

```

classical

let N (i : ι) := ∏ j ∈ s.erase i, n j

have hcop (i : ι) (hi : i ∈ s) : Chapter01.gcd (N i) (n i) = 1 := by
  apply gcd_prod_eq_one _ _ _ (hn i hi)
  intro j hj
  exact hpair j (Finset.mem_of_mem_erase hj) i hi
  ↪ (Finset.ne_of_mem_erase hj)

have hinv (i : ι) (hi : i ∈ s) : ∃ u : ℤ, IsModularInverse (n i) (N i) u
  ↪ :=
  exists_modularInverse _ _ (hcop i hi)
  choose u hu using hinv

let x := ∑ i ∈ s.attach, a i * N i * u i i.property

```

Fix i . Subtracting a_i from the constructed sum amounts to subtracting a_i from its i th summand and leaving the others unchanged.

```

have hx (i : ι) (hi : i ∈ s) : x ≡ a i [ZMOD n i] := by
  have hsum : x - a i =
    ∑ j ∈ s.attach, (a j * N j * u j j.property - if j.val = i then a
      ↪ i else 0) := by
    simp only [Finset.sum_sub_distrib]

  have he : (∑ j ∈ s.attach, if j.val = i then a i else 0) = a i := by
    rw [Finset.sum_eq_single {i, hi}]

    · simp only [↓reduceIte]

    · intro j hj hji

      have hne : j.val ≠ i := by
        intro heq
        apply hji
        exact Subtype.ext heq
        simp only [hne, ↓reduceIte]

    · simp
    rw [he]

```

For the i th summand, the inverse relation makes the adjusted term divisible by n_i .

```

apply (modEq_iff_dvd_sub _ _ _).2
rw [hsum]
apply Finset.dvd_sum
intro j hj
by_cases he : j.val = i

· subst i
  simp only [↓reduceIte]

obtain ⟨k, hk⟩ := (modEq_iff_dvd_sub _ _ _).1 (hu j.val j.property)

exact ⟨a j * k, by nlinarith [congrArg (fun z => a j * z) hk]⟩

```

Every other summand contains n_i in its complementary product. Each adjusted summand is thus divisible by n_i , so their sum is too.

```

· simp only [he, ↓reduceIte, sub_zero]

have hmem : i ∈ s.erase j.val := Finset.mem_erase.mpr ⟨Ne.symm he, hi⟩

obtain ⟨k, hk⟩ := Finset.dvd_prod_of_mem n hmem

exact ⟨a j * k * u j j.property, by
  change N j = n i * k at hk
  rw [hk]
  ring⟩

```

Two simultaneous solutions have a difference divisible by every n_i . Repeated use of the coprime product-divisibility lemma makes the difference divisible by their product. Conversely, a multiple of that product is a multiple of each modulus.

```

refine ⟨x, hx, ?_⟩
intro y
constructor

· intro hy
  apply (modEq_iff_dvd_sub _ _ _).2
  apply prod_dvd_of_pairwise s n hn hpair
  intro i hi
  exact (modEq_iff_dvd_sub _ _ _).1 (modEq_trans (hy i hi) (modEq_symm
    ↪ (hx i hi)))

· intro hy i hi

obtain ⟨k, hk⟩ := (modEq_iff_dvd_sub _ _ _).1 hy
obtain ⟨t, ht⟩ := Finset.dvd_prod_of_mem n hi
apply modEq_trans _ (hx i hi)

```

```

exact (modEq_iff_dvd_sub _ _ _).2 {t * k, by
  rw [hk, ht]
  ring)

```

□

Theorem 3.3.10. Let $\Delta = ad - bc$ and suppose $\Delta t \equiv 1 \pmod{n}$. The system $ax + by \equiv r$, $cx + dy \equiv s \pmod{n}$ is equivalent to $x \equiv t(dr - bs)$ and $y \equiv t(as - cr) \pmod{n}$.

```

theorem linear_system_iff (n : ℕ) (a b c d r s t x y : ℤ)
  (ht : (a * d - b * c) * t ≡ 1 [ZMOD (n : ℤ)]) :
  (a * x + b * y ≡ r [ZMOD (n : ℤ)] ∧ c * x + d * y ≡ s [ZMOD (n :
    ↪ ℤ)]) ↔
  x ≡ t * (d * r - b * s) [ZMOD (n : ℤ)] ∧
  y ≡ t * (a * s - c * r) [ZMOD (n : ℤ)]

```

Proof. Work in the residue ring. Subtract b times the second equation from d times the first, and subtract c times the first from a times the second. These give $\Delta x = dr - bs$ and $\Delta y = as - cr$. Multiplication by t gives the formulas; direct substitution verifies their converse. The auxiliary ring calculation is proved in the same checked module.

```

simp only [← ZMod.intCast_eq_intCast_iff, Int.cast_add, Int.cast_mul,
  Int.cast_sub, Int.cast_one] at ht ⊢

exact two_equations (a : ZMod n) b c d r s t x y ht

```

□

Theorem 3.3.11. If $\gcd(ad - bc, n) = 1$, the two-variable system has a unique solution pair modulo n .

```

theorem exists_unique_linear_system (n : ℕ) (a b c d r s : ℤ)
  (hdet : Chapter01.gcd (a * d - b * c) n = 1) :
  ∃ x₀ y₀ : ℤ, ∀ x y : ℤ,
    (a * x + b * y ≡ r [ZMOD (n : ℤ)] ∧ c * x + d * y ≡ s [ZMOD (n :
      ↪ ℤ)]) ↔
    x ≡ x₀ [ZMOD (n : ℤ)] ∧ y ≡ y₀ [ZMOD (n : ℤ)]

```

Proof. Choose an inverse of the determinant and apply the explicit solution formulas.

```
obtain (t, ht) := exists_modularInverse n (a * d - b * c) hdet  
  
exact (t * (d * r - b * s), t * (a * s - c * r),  
  fun x y => linear_system_iff n a b c d r s t x y ht)
```

□

Chapter 4

Fermat's Theorem

4.1 Fermat's Little Theorem and Pseudoprimes

Theorem 4.1.1. *If p is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$.*

```
theorem fermat (p : ℕ) (hp : Chapter02.isPrime p) (a : ℤ) (ha : ¬(p : ℤ)
↪ | a) :
  a ^ (p - 1) ≡ 1 [ZMOD (p : ℤ)]
```

Proof. The class of a modulo p is nonzero, so it is invertible.

```
let : Fact p.Prime := ⟨Nat.prime_def.mpr hp⟩

have hne : (a : ZMod p) ≠ 0 := by
  intro h
  exact ha ((ZMod.intCast_zmod_eq_zero_iff_dvd a p).1 h)
```

Multiplication by this class permutes the nonzero residue classes. Multiplying the equalities gives $a^{p-1}P = P$, where P is the product of these classes. Cancelling the invertible product gives $a^{p-1} = 1$.

```
have h := pow_card_by_permutation (Units.mk0 (a : ZMod p) hne)
```

Passing from units to residue classes gives $a^{p-1} \equiv 1 \pmod{p}$.

```

have hv := congrArg (fun u : (ZMod p)x => (u : ZMod p)) h
simp only [Units.val_pow_eq_pow_val, Units.val_mk0, Units.val_one,
  Fintype.card_units, ZMod.card] at hv

apply (ZMod.intCast_eq_intCast_iff _ _ p).1
simp only [Int.cast_pow, Int.cast_one] using hv

```

□

Theorem 4.1.2. *For every prime p and integer a , one has $a^p \equiv a \pmod{p}$.*

```

theorem fermat_all (p : ℕ) (hp : Chapter02.isPrime p) (a : ℤ) :
  a ^ p ≡ a [ZMOD (p : ℤ)]

```

Proof. If $p \mid a$, both a^p and a are zero modulo p .

```

have hpos : 0 < p := by
  have := hp.1
  omega
by_cases hpa : (p : ℤ) ∣ a

• have hz := Chapter03.modEq_pow ((Chapter03.modEq_zero_iff _ _).2 hpa)
  ↪ p
  rw [zero_pow (ne_of_gt hpos)] at hz
  exact Chapter03.modEq_trans hz
    (Chapter03.modEq_symm ((Chapter03.modEq_zero_iff _ _).2 hpa))

```

Otherwise multiply Fermat's congruence by a to obtain $a^p \equiv a \pmod{p}$.

```

• have h := Chapter03.modEq_mul (fermat p hp a hpa)
  ↪ (Chapter03.modEq_refl (p : ℤ) a)
  simp only [← pow_succ, Nat.sub_add_cancel hpos, one_mul] using h

```

□

Theorem 4.1.3. *If $n > 1$ and $a^n \not\equiv a \pmod{n}$ for some integer a , then n is composite.*

```

theorem composite_of_not_fermat (n : ℕ) (hn : 1 < n) (a : ℤ)
  (h : ¬a ^ n ≡ a [ZMOD (n : ℤ)]) : Chapter02.isComposite n

```

Proof. If n were prime, the unrestricted Fermat congruence would contradict the hypothesis.

```
{hn, fun hp => h (fermat_all n hp a)}
```

□

Theorem 4.1.4. *Let $p \neq q$ be primes. If $a^p \equiv a \pmod{q}$ and $a^q \equiv a \pmod{p}$, then $a^{pq} \equiv a \pmod{pq}$.*

```
theorem fermat_two_primes (p q : ℕ) (hp : Chapter02.isPrime p)
  (hq : Chapter02.isPrime q) (hne : p ≠ q) (a : ℤ)
  (hpq : a ^ p ≡ a [ZMOD (q : ℤ)]) (hqp : a ^ q ≡ a [ZMOD (p : ℤ)]) :
  a ^ (p * q) ≡ a [ZMOD ((p * q : ℕ) : ℤ)]
```

Proof. Raise the congruence modulo p to the p th power, and the one modulo q to the q th power. Fermat's theorem reduces their right-hand sides to a .

```
have h1 := Chapter03.modEq_trans (Chapter03.modEq_pow hqp p) (fermat_all
  ↪ p hp a)

have h2 := Chapter03.modEq_trans (Chapter03.modEq_pow hpq q) (fermat_all
  ↪ q hq a)
rw [← pow_mul, Nat.mul_comm q p] at h1
rw [← pow_mul] at h2
```

The two distinct primes are coprime. Their product therefore divides $a^{pq} - a$, giving the required congruence.

```
apply (Chapter03.modEq_iff_dvd_sub _ _ _).2
rw [Nat.cast_mul]
exact Chapter01.mul_dvd_of_coprime p q _ (gcd_distinct_primes p q hp hq
  ↪ hne)
  ((Chapter03.modEq_iff_dvd_sub _ _ _).1 h1)
  ↪ ((Chapter03.modEq_iff_dvd_sub _ _ _).1 h2)
```

□

Definition 4.1.5. A composite natural number n is a *pseudoprime to base a* if $a^n \equiv a \pmod{n}$. The term pseudoprime without a specified base refers to base two.

```
def IsPseudoprime (a : ℤ) (n : ℕ) : Prop :=
  Chapter02.isComposite n ∧ a ^ n ≡ a [ZMOD (n : ℤ)]
```

Definition 4.1.6. An *absolute pseudoprime*, or Carmichael number, is a composite n satisfying $a^n \equiv a \pmod{n}$ for every integer a .

```
def IsAbsolutePseudoprime (n : ℕ) : Prop :=
  Chapter02.isComposite n ∧ ∀ a : ℤ, a ^ n ≡ a [ZMOD (n : ℤ)]
```

Definition 4.1.7. A positive integer is *square-free* when no square of a prime divides it. In particular, one is square-free.

```
def IsSquareFree (n : ℕ) : Prop :=
  0 < n ∧ ∀ p : ℕ, Chapter02.isPrime p → ¬p ^ 2 | n
```

Theorem 4.1.8. For an odd composite n , the base-two pseudoprime condition is equivalent to $2^{n-1} \equiv 1 \pmod{n}$.

```
theorem pseudoprime_two_iff (n : ℕ) (hn : Odd n) (hc :
  ⇔ Chapter02.isComposite n) :
  IsPseudoprime 2 n ↔ (2 : ℤ) ^ (n - 1) ≡ 1 [ZMOD (n : ℤ)]
```

Proof. An odd integer is coprime to two: writing $n = 2k + 1$ gives the Bezout identity $1 = n - 2k$. Cancel two in the pseudoprime congruence to obtain $2^{n-1} \equiv 1 \pmod{n}$.

```
have hpos : 0 < n := by
  have := hc.1
  omega

constructor

· intro h
  apply Chapter03.modEq_cancel (gcd_odd_two n hn)
  simp only [← pow_succ', Nat.sub_add_cancel hpos, mul_one] using h.2
```

Conversely, multiply $2^{n-1} \equiv 1 \pmod{n}$ by two. This gives the defining pseudoprime congruence, and compositeness is already assumed.

```
· intro h
  refine (hc, ?_)

  have hh := Chapter03.modEq_mul h (Chapter03.modEq_refl (n : ℤ) 2)
  simp only [← pow_succ, Nat.sub_add_cancel hpos, one_mul] using hh
```

□

Definition 4.1.9. The Mersenne number with exponent n is $M_n = 2^n - 1$.

```
def mersenne (n : ℕ) : ℕ := 2 ^ n - 1
```

Theorem 4.1.10. *If n is a base-two pseudoprime, then M_n is a larger base-two pseudoprime. In particular, this holds for every odd pseudoprime.*

```
theorem mersenne_pseudoprime (n : ℕ) (h : IsPseudoprime 2 n) :
  IsPseudoprime 2 (mersenne n) ∧ n < mersenne n
```

Proof. Since n is composite, it has a proper divisor $d > 1$. The identity $d \mid n$ gives $2^d - 1 \mid 2^n - 1$, and this is a nontrivial proper divisor. Thus M_n is composite. Induction also gives $2^n - 1 > n$ for $n \geq 2$.

```
have hn : 2 ≤ n := h.1.1

have hlarge := lt_mersenne n hn

have hpow : 2 ≤ 2 ^ n := by
  have hh : 1 ≤ 2 ^ (n - 1) := by
    have := pow_pos (by decide : 0 < (2 : ℕ)) (n - 1)
    omega

  have he : 2 ^ n = 2 ^ (n - 1) * 2 := by
    rw [← pow_succ, Nat.sub_add_cancel (by omega : 1 ≤ n)]
    omega
```

The pseudoprime congruence gives $n \mid 2^n - 2 = M_n - 1$. Write $M_n - 1 = nk$. Since $2^n \equiv 1 \pmod{M_n}$, raising to the k th power and then multiplying by two proves $2^{M_n} \equiv 2 \pmod{M_n}$.

```
have hdiv : n ∣ mersenne n - 1 := by
  apply Int.natCast_dvd_natCast.mp

  have hh := (Chapter03.modEq_iff_dvd_sub _ _ _).1 h.2

  have he : ((mersenne n - 1 : ℕ) : ℤ) = (2 : ℤ) ^ n - 2 := by
    rw [mersenne, Nat.sub_sub, Nat.cast_sub hpow, Nat.cast_pow,
      ↪ Nat.cast_ofNat]
    rwa [he]

obtain ⟨k, hk⟩ := hdiv

have hbase : (2 : ℤ) ^ n ≡ 1 [ZMOD ((mersenne n : ℕ) : ℤ)] := by
```

```

apply (Chapter03.modEq_iff_dvd_sub _ _ _).2
refine (1, ?_)
simp only [mersenne, Nat.cast_sub (by omega : 1 ≤ 2 ^ n),
  Nat.cast_pow, Nat.cast_ofNat, Nat.cast_one, mul_one]

have hpower := Chapter03.modEq_pow hbase k
rw [← pow_mul, ← hk, one_pow] at hpower

have hfinal := Chapter03.modEq_mul hpower
  (Chapter03.modEq_refl ((mersenne n : ℕ) : ℤ) 2)
rw [← pow_succ, Nat.sub_add_cancel (by omega : 1 ≤ mersenne n), one_mul]
↪ at hfinal

exact (⟨mersenne_composite n h.1, hfinal⟩, hlarge)

```

□

Theorem 4.1.11. *There are infinitely many odd base-two pseudoprimes.*

```

theorem infinite_odd_pseudoprimes : {n : ℕ | Odd n ∧ IsPseudoprime 2
  ↪ n}.Infinite

```

Proof. Start from 341 and repeatedly apply the Mersenne construction. Every new value is odd and strictly larger, so an odd pseudoprime exists above every natural bound. An unbounded subset of the natural numbers is infinite.

```

apply Set.infinite_of_forall_exists_gt
intro B

obtain ⟨n, hB, hodd, hn⟩ := exists_odd_pseudoprime_gt B

exact ⟨n, ⟨hodd, hn⟩, hB⟩

```

□

Theorem 4.1.12. *Let n be a composite product of distinct primes. If $p - 1 \mid n - 1$ for every prime in the product, then n is an absolute pseudoprime.*

```

theorem absolutePseudoprime_of_prime_product (s : Finset ℕ)
  (hp : ∀ p ∈ s, Chapter02.isPrime p)
  (hc : Chapter02.isComposite (∏ p ∈ s, p))
  (hdiv : ∀ p ∈ s, p - 1 ∣ (∏ q ∈ s, q) - 1) :
  IsAbsolutePseudoprime (∏ p ∈ s, p)

```

Proof. Fix a base a . To combine congruences modulo the prime factors, first note that those factors are nonzero and pairwise coprime.

```

refine (hc, ?_)
intro a
apply (Chapter03.modEq_iff_dvd_sub _ _ _).2
rw [Nat.cast_prod]
apply Chapter03.prod_dvd_of_pairwise s (fun p => (p : ℤ))

· intro p hps

  have := (hp p hps).1
  omega

· intro p hps q hqs hne
  exact gcd_distinct_primes p q (hp p hps) (hp q hqs) hne

```

For each prime factor p , either $p \mid a$, or Fermat's theorem and $p - 1 \mid n - 1$ give $a^{n-1} \equiv 1 \pmod{p}$. In either case $a^n \equiv a \pmod{p}$. Pairwise coprimality combines these congruences modulo their product.

```

· intro p hps
  exact (Chapter03.modEq_iff_dvd_sub _ _ _).1
    (fermat_of_sub_one_dvd p _ (hp p hps) (by
      have := hc.1
      omega) (hdiv p hps) a)

```

□

Theorem 4.1.13. *Every absolute pseudoprime is square-free.*

```

theorem absolutePseudoprime_squareFree (n : ℕ) (h :
  ↔ IsAbsolutePseudoprime n) :
  IsSquareFree n

```

Proof. Suppose $p^2 \mid n$ for a prime p . Use the defining congruence with base p . Since $n \geq 2$, both p^n and n have a factor p^2 .

```

refine ⟨by
  have := h.1.1
  omega, ?_⟩
intro p hp hpn

have hn : 2 ≤ n := h.1.1

obtain ⟨k, hk⟩ := (Chapter03.modEq_iff_dvd_sub _ _ _).1 (h.2 p)
obtain ⟨t, ht⟩ := hpn

have ht' : (n : ℤ) = (p : ℤ) ^ 2 * t := by
  exact_mod_cast ht

have hpow : (p : ℤ) ^ n = (p : ℤ) ^ 2 * (p : ℤ) ^ (n - 2) := by
  rw [← pow_add, Nat.add_sub_cancel' hn]

```

Substituting these factorizations into the congruence and cancelling the nonzero factor p gives an integer multiple of p equal to one.

```

have hcancel : (p : ℤ) * ((p : ℤ) ^ (n - 2) - (t : ℤ) * k) = 1 := by
  have hpne : (p : ℤ) ≠ 0 := by
    have := hp.1
    omega

  apply mul_left_cancel₀ hpne
  rw [ht', hpow] at hk
  nlinarith [hk]

```

This would make a prime divide one, a contradiction. Thus no prime square divides n .

```

have hd : p ∣ 1 := Int.natCast_dvd_natCast.mp
  ((p : ℤ) ^ (n - 2) - (t : ℤ) * k, hcancel.symm)

obtain ⟨u, hu⟩ := hd

have := hp.1

have hu' : 0 < u := by
  nlinarith
nlinarith

```

□

4.2 Wilson's Theorem

Theorem 4.2.1. *If p is prime, then $(p - 1)! \equiv -1 \pmod{p}$.*

```
theorem wilson (p : ℕ) (hp : Chapter02.isPrime p) :
  ((p - 1).factorial : ℤ) ≡ -1 [ZMOD (p : ℤ)]
```

Proof. Identify nonzero residue classes with their representatives $1, \dots, p-1$. A nonzero class has its representative in this interval.

```
classical

have : Fact p.Prime := ⟨Nat.prime_def.mpr hp⟩

have hpos : 0 < p := by
  have := hp.1
  omega

have hprod : (∏ u : (ZMod p)×, (u : ZMod p)) =
  ∏ k ∈ Finset.Ico 1 p, (k : ZMod p) := by

  apply Finset.prod_bij (fun (u : (ZMod p)×) _ => (u : ZMod p).val)
    · intro u hu
      apply Finset.mem_Ico.mpr
      refine {?, ZMod.val_lt _}

    have hne : (u : ZMod p).val ≠ 0 := by
      intro hz
      apply Units.ne_zero u
      apply ZMod.val_injective p
      simp only [ZMod.val_zero] using hz
    omega
```

Equal representatives give equal classes, so the correspondence is injective.

```
· intro u hu v hv h
  exact Units.ext (ZMod.val_injective p h)
```

Conversely, an integer in this interval represents a nonzero class and hence a unit. This proves surjectivity.

```
· intro k hk

have hk' := Finset.mem_Ico.mp hk

have hne : (k : ZMod p) ≠ 0 := by
  intro hz
```

```

have hh := congrArg ZMod.val hz
rw [ZMod.val_natCast_of_lt hk'.2, ZMod.val_zero] at hh
omega

exact (Units.mk0 (k : ZMod p) hne, Finset.mem_univ _,
      ZMod.val_natCast_of_lt hk'.2)

```

The representative and its residue class have the same value modulo p , so the bijection preserves the product.

```

· intro u hu
  exact (ZMod.natCast_zmod_val _).symm

```

The product of these representatives is $(p - 1)!$.

```

have hf : (∏ k ∈ Finset.Ico 1 p, (k : ZMod p)) = ((p - 1).factorial :
  ↪ ZMod p) := by
  rw [← Nat.cast_prod, ← Finset.prod_Ico_id_eq_factorial]
  rw [Nat.sub_add_cancel hpos]

```

Pair each nonzero class with its inverse. Fixed points satisfy $x^2 = 1$, so they are one or minus one. All inverse pairs and the fixed point one contribute one, leaving minus one. The inverse-pair lemma therefore gives Wilson's congruence.

```

apply (ZMod.intCast_eq_intCast_iff _ _ p).1
simpa only [Int.cast_natCast, Int.cast_neg, Int.cast_one] using
  (hf.symm.trans (hprod.symm.trans (prod_units_eq_neg_one p)))

```

□

Theorem 4.2.2. *If $n > 1$ and $(n - 1)! \equiv -1 \pmod{n}$, then n is prime.*

```

theorem prime_of_wilson (n : ℕ) (hn : 1 < n)
  (h : ((n - 1).factorial : ℤ) ≡ -1 [ZMOD (n : ℤ)]) :
  ↪ Chapter02.isPrime n

```

Proof. Let d divide n . If $d = n$, the desired alternative already holds. Otherwise positivity and divisibility give $1 \leq d \leq n - 1$.

```

refine ⟨hn, ?_⟩
intro d hd
by_cases he : d = n

· exact Or.inr he

· have hdpos : 0 < d := by
  obtain ⟨k, hk⟩ := hd
  by_contra hh

  have hz : d = 0 := by
    omega
  rw [hz, zero_mul] at hk
  omega

  have hdl : d ≤ n := by
    obtain ⟨k, hk⟩ := hd

  have hkpos : 0 < k := by
    nlinarith
  nlinarith

```

Consequently d divides $(n - 1)!$. The congruence and $d \mid n$ also imply that d divides $(n - 1)! + 1$.

```

have hdf : (d : ℤ) ∣ (n - 1).factorial :=
  Int.natCast_dvd_natCast.mpr (dvd_factorial d (n - 1) hdpos (by omega))

obtain ⟨u, hu⟩ := hdf
obtain ⟨v, hv⟩ := Int.natCast_dvd_natCast.mpr hd
obtain ⟨k, hk⟩ := (Chapter03.modEq_iff_dvd_sub _ _ _).1 h

```

Subtracting these multiples shows that d divides one. Since d is positive, it equals one. Thus every divisor is one or n .

```

have hone : (1 : ℤ) = d * (v * k - u) := by
  rw [hu, hv] at hk
  nlinarith

have hdz : (0 : ℤ) < d := by
  exact_mod_cast hdpos

have hkp : 0 < v * k - u := by
  nlinarith
left

have : (d : ℤ) = 1 := by

```

```
nlinarith
exact_mod_cast this
```

□

Theorem 4.2.3. For $n > 1$, primality is equivalent to $(n - 1)! \equiv -1 \pmod{n}$.

```
theorem prime_iff_wilson (n : ℕ) (hn : 1 < n) :
  Chapter02.isPrime n ↔ ((n - 1).factorial : ℤ) ≡ -1 [ZMOD (n : ℤ)]
```

Proof. Combine Wilson's theorem with its converse.

```
(wilson n, prime_of_wilson n hn)
```

□

Definition 4.2.4. A quadratic congruence modulo n has coefficients $a, b, c \in \mathbb{Z}$ and the form $ax^2 + bx + c \equiv 0 \pmod{n}$, with $a \not\equiv 0 \pmod{n}$.

```
structure QuadraticCongruence (n : ℤ) where
  a : ℤ
  b : ℤ
  c : ℤ
  leading_ne_zero : ¬a ≡ 0 [ZMOD n]
```

Theorem 4.2.5. If $p = 2m + 1$, then in the residue ring modulo p ,

$$(p - 1)! = (-1)^m (m!)^2.$$

```
theorem half_factorial (p m : ℕ) (hp : p = 2 * m + 1) :
  ((p - 1).factorial : ZMod p) = (-1 : ZMod p) ^ m * (m.factorial :
    ↪ ZMod p) ^ 2
```

Proof. Split the factorial into the factors $1, \dots, m$ and $m + 1, \dots, 2m$. Reflection $k \mapsto p - k$ carries the second interval to the first, and each reflected residue is $-k$.

```

have hprefix : (∏ k ∈ Finset.Ico 1 (m + 1), (k : ZMod p)) = (m.factorial
  ↪ : ZMod p) := by
  rw [← Nat.cast_prod, Finset.prod_Ico_id_eq_factorial]

have hreflect : (∏ k ∈ Finset.Ico (m + 1) p, (k : ZMod p)) =
  ∏ k ∈ Finset.Ico 1 (m + 1), ((p - k : ℕ) : ZMod p) := by
  rw [Finset.prod_Ico_reflect _ 1 (by omega : m + 1 ≤ p + 1)]
  rw [show p + 1 - (m + 1) = m + 1 by omega, Nat.add_sub_cancel]

have hnegative : (∏ k ∈ Finset.Ico 1 (m + 1), ((p - k : ℕ) : ZMod p)) =
  (-1 : ZMod p) ^ m * (m.factorial : ZMod p) := by
have hterm : ∀ k ∈ Finset.Ico 1 (m + 1), ((p - k : ℕ) : ZMod p) = -1 *
  ↪ (k : ZMod p) := by
  intro k hk

  have hkp : k ≤ p := by
    have := Finset.mem_Ico.mp hk
    omega

  rw [Nat.cast_sub hkp, ZMod.natCast_self, zero_sub, neg_one_mul]

  rw [Finset.prod_congr rfl hterm, Finset.prod_mul_distrib,
    ↪ Finset.prod_const,
    Nat.card_Ico, Nat.add_sub_cancel, hprefix]

```

The first product is $m!$ and the second is $(-1)^m m!$. Multiplying the two pieces gives the identity.

```

have hfull : ((p - 1).factorial : ZMod p) = ∏ k ∈ Finset.Ico 1 p, (k :
  ↪ ZMod p) := by
  rw [← Nat.cast_prod, ← Finset.prod_Ico_id_eq_factorial]
  rw [Nat.sub_add_cancel (by omega : 1 ≤ p)]

  rw [hfull, ← Finset.prod_Ico_consecutive (fun k => (k : ZMod p))
    (by omega : 1 ≤ m + 1) (by omega : m + 1 ≤ p), hprefix, hreflect,
    ↪ hnegative]
  ring

```

□

Theorem 4.2.6. *For an odd prime p , the congruence $x^2 \equiv -1 \pmod{p}$ has an integer solution exactly when $p \equiv 1 \pmod{4}$.*

```

theorem exists_square_eq_neg_one_iff (p : ℕ) (hp : Chapter02.isPrime p)
  ↪ (hodd : Odd p) :
  (∃ x : ℤ, x ^ 2 ≡ -1 [ZMOD (p : ℤ)]) ↔ p % 4 = 1

```

Proof. Write $p = 2m + 1$. Suppose $x^2 \equiv -1 \pmod{p}$. If $p \mid x$, this congruence would force $p \mid 1$, which is impossible. Thus $p \nmid x$.

```

have hp2 : 2 < p := by
  have hmod := Nat.odd_iff.mp hodd

  have := hp.1
  omega

let m := (p - 1) / 2

have hpm : p = 2 * m + 1 := by
  have hmod := Nat.odd_iff.mp hodd
  dsimp [m]
  omega

constructor

· rintro ⟨x, hx⟩

have hnot : ¬(p : ℤ) ∣ x := by
  intro hd

  have hz := Chapter03.modEq_pow ((Chapter03.modEq_zero_iff _ _).2 hd)
  ↪ 2
  rw [zero_pow (by decide : 2 ≠ 0)] at hz

  have hh := (Chapter03.modEq_iff_dvd_sub _ _ _).1
    (Chapter03.modEq_trans (Chapter03.modEq_symm hz) hx)

  obtain ⟨k, hk⟩ := hh

  have hpz : (2 : ℤ) < p := by
    exact_mod_cast hp2

  have hkpos : 0 < k := by
    nlinarith
    nlinarith

```

Raise the square congruence to the m th power. Since $2m = p - 1$, Fermat's theorem gives $(-1)^m \equiv 1 \pmod{p}$.

```

have hpow := Chapter03.modEq_pow hx m
rw [← pow_mul, show 2 * m = p - 1 by omega] at hpow

have hh := Chapter03.modEq_trans (Chapter03.modEq_symm hpow) (fermat p
↪ hp x hnot)

```

If p were not one modulo four, the odd prime $p > 2$ would be three modulo four and m would be odd. The last congruence would then force $p \mid 2$, contrary to $p > 2$.

```
by_contra hne

have hm : Odd m := Nat.odd_iff.mpr (by omega)
rw [hm.neg_one_pow] at hh

obtain ⟨k, hk⟩ := (Chapter03.modEq_iff_dvd_sub _ _ _).1 hh

have hpz : (2 : ℤ) < p := by
  exact_mod_cast hp2

have hkneg : k < 0 := by
  nlinarith
nlinarith
```

If $p \equiv 1 \pmod{4}$, then m is even. The half-factorial identity and Wilson's theorem give $(m!)^2 \equiv -1$, which supplies the required solution.

```
· intro hpmod

have hm : Even m := Nat.even_iff.mpr (by omega)

have hhalf := half_factorial p m hpm
rw [hm.neg_one_pow, one_mul] at hhalf

have hw := (ZMod.intCast_eq_intCast_iff _ _ p).2 (wilson p hp)
simp only [Int.cast_natCast, Int.cast_neg, Int.cast_one] at hw

refine ⟨m.factorial, ?_⟩
apply (ZMod.intCast_eq_intCast_iff _ _ p).1
simp only [Int.cast_pow, Int.cast_natCast, Int.cast_neg,
  ↪ Int.cast_one] using
  hhalf.symm.trans hw
```

□

Theorem 4.2.7. *Infinitely many composite integers have the form $n! + 1$.*

```
theorem infinite_factorial_composites :
  {k : ℕ | (∃ n : ℕ, k = n.factorial + 1) ∧ Chapter02.isComposite
  ↪ k}.Infinite
```

Proof. Choose primes p arbitrarily large and put $n = p - 1$. Wilson gives $p \mid n! + 1$. For $n \geq 3$, the elementary bound $n! \geq 2n$ makes this divisor proper. These composite values exceed every bound.

```

apply Set.infinite_of_forall_exists_gt
intro B

obtain ⟨n, hB, hn⟩ := exists_composite_factorial_add_one_gt B

exact ⟨n.factorial + 1, ⟨⟨n, rfl⟩, hn⟩, hB⟩

```

□

4.3 The Fermat-Kraitchik Factorization Method

Chapter 5

Number-Theoretic Functions

5.1 The Sum and Number of Divisors

Definition 5.1.1. For positive n , let $\tau(n)$ be the number of positive divisors of n . The total Lean definition assigns zero at zero.

```
def tau (n : ℕ) : ℕ := n.divisors.card
```

Definition 5.1.2. For positive n , define $\sigma(n) = \sum_{d|n} d$. All divisors in this chapter are positive; the divisor set at zero is empty.

```
def sigma (n : ℕ) : ℕ := ∑ d ∈ n.divisors, d
```

Definition 5.1.3. For a finitely supported exponent vector e , write $V(e) = \prod_p p^{e(p)}$, with the product taken over its finite support.

```
def factorValue (e : ℕ →₀ ℕ) : ℕ := e.prod (fun p k => p ^ k)
```

Definition 5.1.4. For positive n , write $e_n(p)$ for its unique prime multiplicities from Chapter 2. The value at zero is assigned the zero vector.

```
noncomputable def primeExponents (n : ℕ) : ℕ →₀ ℕ :=  
  if hn : 0 < n then (Chapter02.existsUnique_prime_exponents n  
    ↪ hn).exists.choose else 0
```

Theorem 5.1.5. *For positive d, n , one has $d \mid n$ precisely when $e_d(p) \leq e_n(p)$ for every prime p .*

```
theorem dvd_iff_primeExponents_le (d n : ℕ) (hd : 0 < d) (hn : 0 < n) :
  d ∣ n ↔ primeExponents d ≤ primeExponents n
```

Proof. If $n = dk$, uniqueness of prime factorization gives $e_n = e_d + e_k$, hence $e_d \leq e_n$.

```
constructor
· rintro ⟨k, hk⟩

have hkpos : 0 < k := by
  nlinarith
rw [hk, primeExponents_mul d k hd hkpos]
exact le_add_right le_rfl
```

Conversely, subtract the exponent vector of d from that of n . Its product supplies the quotient witnessing divisibility.

```
· intro hle

have hsum : primeExponents d + (primeExponents n - primeExponents d) =
  primeExponents n := add_tsub_cancel_of_le hle

refine ⟨factorValue (primeExponents n - primeExponents d), ?_⟩

have hh := congrArg factorValue hsum
rw [factorValue_add, (primeExponents_spec d hd).2,
  ↪ (primeExponents_spec n hn).2] at hh
exact hh.symm
```

□

Theorem 5.1.6. *Every exponent vector $0 \leq e \leq e_n$ gives a positive divisor $V(e)$ of n , whose prime multiplicities are exactly e . Consequently these choices describe the divisors uniquely.*

```
theorem divisor_exponents (n : ℕ) (hn : 0 < n) (e : ℕ →₀ ℕ)
  (he : e ≤ primeExponents n) :
  0 < factorValue e ∧ factorValue e ∣ n ∧ primeExponents (factorValue
    ↪ e) = e
```

Proof. The support of e lies among the prime factors of n , so its product is positive.

```

have hp : ∀ p ∈ e.support, Chapter02.isPrime p := by
  intro p hps
  apply (primeExponents_spec n hn).1
  apply Finsupp.mem_support_iff.mpr

have hpos : 0 < e p := Nat.pos_of_ne_zero (Finsupp.mem_support_iff.mp
  ↪ hps)

have hle := he p
  omega

have hpos := factorValue_pos e hp

```

Uniqueness of prime factorization recovers e from that product. The coordinatewise divisibility criterion then shows that the product divides n .

```

have heq := (primeExponents_unique (factorValue e) hpos e hp rfl).symm

exact ⟨hpos, (dvd_iff_primeExponents_le _ n hpos hn).2 (by
  rw [heq]
  exact he), heq⟩

```

□

Theorem 5.1.7. For positive $n = \prod_p p^{e_n(p)}$,

$$\tau(n) = \prod_{p|n} (e_n(p) + 1).$$

The product is one when $n = 1$.

```

theorem tau_eq_prod (n : ℕ) (hn : 0 < n) :
  tau n = ∏ p ∈ (primeExponents n).support, (primeExponents n p + 1)

```

Proof. The divisor-exponent correspondence is a bijection between the divisors and the integer box $0 \leq e(p) \leq e_n(p)$.

```

classical

have hcard : n.divisors.card = (Finset.Iic (primeExponents n)).card :=
  ↪ by
    apply Finset.card_bij (fun d _ => primeExponents d)

  · intro d hd
    exact Finset.mem_Iic.mpr ((dvd_iff_primeExponents_le d n
      (Nat.pos_of_mem_divisors hd) hn).1 (Nat.dvd_of_mem_divisors hd))

  · intro d hd e he hde

    have hh := congrArg factorValue hde
    rwa [(primeExponents_spec d (Nat.pos_of_mem_divisors hd)).2,
      (primeExponents_spec e (Nat.pos_of_mem_divisors he)).2] at hh

  · intro e he

    obtain ⟨hpos, hdiv, heq⟩ := divisor_exponents n hn e
    ↪ (Finset.mem_Iic.mp he)

    exact ⟨factorValue e, Nat.mem_divisors.mpr ⟨hdiv, ne_of_gt hn⟩, heq⟩

```

Each coordinate has $e_n(p) + 1$ choices. Counting the box therefore gives the stated product.

```

rw [tau, hcard, Finsupp.card_Iic]
simp only [Nat.card_Iic]

```

□

Definition 5.1.8. An integer-valued arithmetic function is *multiplicative* when $f(mn) = f(m)f(n)$ for positive coprime m, n . This convention permits the identically zero function and does not require $f(1) = 1$.

```

def IsMultiplicative (f : ℕ → ℤ) : Prop :=
  ∀ m n : ℕ, 0 < m → 0 < n → Chapter01.gcd m n = 1 → f (m * n) = f m * f n
  ↪ n

```

Theorem 5.1.9. If a multiplicative function is nonzero at some positive argument, then $f(1) = 1$.

```

theorem multiplicative_one (f : ℕ → ℤ) (hf : IsMultiplicative f)
  (hne : ∃ n : ℕ, 0 < n ∧ f n ≠ 0) : f 1 = 1

```

Proof. Choose n with $f(n) \neq 0$. The integers one and n are coprime.

```

obtain ⟨n, hn, hfn⟩ := hne

have hcop : Chapter01.gcd 1 n = 1 :=
  (Chapter01.gcd_eq_one_iff_exists_mul_add_mul 1 n (Or.inl (by
    ↪ decide)))2
  ⟨1, 0, by ring⟩

```

Multiplicativity gives $f(n) = f(1)f(n)$. Cancelling the nonzero value $f(n)$ yields $f(1) = 1$.

```

have h := hf 1 n (by decide) hn hcop
rw [one_mul] at h
exact mul_right_cancel0 hfn ((one_mul (f n)).trans h).symm

```

□

Theorem 5.1.10. *For positive coprime m, n , each divisor $d \mid mn$ has a unique expression $d = ab$ with $a \mid m$ and $b \mid n$.*

```

theorem existsUnique_divisor_split (m n d : ℕ) (hm : 0 < m) (hn : 0 < n)
  (hcop : Chapter01.gcd m n = 1) (hd : d ∈ (m * n).divisors) :
  ∃! ab : ℕ × ℕ, ab.1 ∣ m ∧ ab.2 ∣ n ∧ ab.1 * ab.2 = d

```

Proof. Allocate the prime factors of d to m or n using Euclid's lemma. This gives a factorization $d = ab$ with $a \mid m$ and $b \mid n$.

```

obtain ⟨a, b, ha, hb, hab⟩ := exists_divisor_split m n d
  (Nat.pos_of_mem_divisors hd) (Nat.dvd_of_mem_divisors hd)

```

For uniqueness, suppose $ab = ce$ with $c \mid m$ and $e \mid n$. Inherited coprimality gives $a \mid c$ and $c \mid a$, hence $a = c$. Positivity then permits cancellation to obtain $b = e$.

```

refine ⟨⟨a, b⟩, ⟨ha, hb, hab⟩, ?_⟩
rintro ⟨c, e⟩ ⟨hc, he, hce⟩

obtain ⟨hca, heb⟩ := divisor_pair_injective m n hm hn hcop c e a b hc he
  ↪ ha hb
  (hce.trans hab.symm)

exact Prod.ext hca heb

```

□

Definition 5.1.11. For an integer-valued arithmetic function f , its divisor sum is $F(n) = \sum_{d|n} f(d)$.

```
def divisorSum (f : ℕ → ℤ) (n : ℕ) : ℤ := ∑ d ∈ n.divisors, f d
```

Theorem 5.1.12. If f is multiplicative, then its divisor sum is multiplicative.

```
theorem divisorSum_multiplicative (f : ℕ → ℤ) (hf : IsMultiplicative f)
  : IsMultiplicative (divisorSum f)
```

Proof. For coprime m, n , uniquely write every divisor of mn as ab with $a \mid m$ and $b \mid n$. The divisor sum becomes a double sum.

```
intro m n hm hn hcop
rw [sum_divisors_mul f m n hm hn hcop, divisorSum, divisorSum,
  ↪ Finset.sum_mul_sum]
```

The divisors a, b inherit coprimality. Replace $f(ab)$ by $f(a)f(b)$ term by term to obtain the product of the two divisor sums.

```
apply Finset.sum_congr rfl
intro a ha
apply Finset.sum_congr rfl
intro b hb
exact hf a b (Nat.pos_of_mem_divisors ha) (Nat.pos_of_mem_divisors hb)
  (gcd_divisors_eq_one m n a b hm hn hcop (Nat.dvd_of_mem_divisors ha)
    (Nat.dvd_of_mem_divisors hb))
```

□

Theorem 5.1.13. The function τ is multiplicative.

```
theorem tau_multiplicative : IsMultiplicative (fun n => (tau n : ℤ))
```

Proof. Take the divisor sum of the constant function one, which is multiplicative.

```

have hh := divisorSum_multiplicative (fun _ => 1) (by
  intro m n hm hn hcop
  ring)
simp only [IsMultiplicative, divisorSum, Finset.sum_const,
  ↪ nsmul_eq_mul, mul_one, tau] using hh

```

□

Theorem 5.1.14. *The function σ is multiplicative.*

```

theorem sigma_multiplicative : IsMultiplicative (fun n => (sigma n : ℤ))

```

Proof. Take the divisor sum of $f(d) = d$, which is multiplicative.

```

have hh := divisorSum_multiplicative (fun n => (n : ℤ)) (by
  intro m n hm hn hcop
  exact Nat.cast_mul m n)
simp only [IsMultiplicative, divisorSum, sigma, Nat.cast_sum] using hh

```

□

Theorem 5.1.15. *For $n > 1$, a multiplicative function satisfies $f(n) = \prod_{p|n} f(p^{e_n(p)})$. This also holds for the zero function.*

```

theorem multiplicative_prime_powers (f : ℕ → ℤ) (hf : IsMultiplicative
  ↪ f)
  (n : ℕ) (hn : 1 < n) :
  f n = ∏ p ∈ (primeExponents n).support, f (p ^ primeExponents n p)

```

Proof. The prime-power factorization of $n > 1$ has nonempty support. Otherwise its product would be one.

```

have hnpos : 0 < n := by
  omega

have hspec := primeExponents_spec n hnpos

have hs : (primeExponents n).support.Nonempty := by
  by_contra h

have he := Finset.not_nonempty_iff_eq_empty.mp h

have hv := hspec.2
change ([] p ∈ (primeExponents n).support, p ^ primeExponents n p) = n
↪ at hv
rw [he, Finset.prod_empty] at hv
omega

```

Distinct prime powers are coprime. Apply multiplicativity successively to this nonempty product. Nonemptiness avoids assuming $f(1) = 1$.

```

calc
  f n = f (factorValue (primeExponents n)) := congrArg f hspec.2.symm
  _ = _ := multiplicative_prod _ hs (fun p => p ^ primeExponents n p)
    (fun p hp => pow_pos (by
      have := (hspec.1 p hp).1
      omega) _)
    (fun p hp q hq hne => gcd_prime_powers p q _ _ (hspec.1 p hp)
      ↪ (hspec.1 q hq) hne) f hf

```

□

Theorem 5.1.16. *If p is prime and $k \geq 0$, then $\tau(p^k) = k + 1$.*

```

theorem tau_prime_pow (p k : ℕ) (hp : Chapter02.isPrime p) : tau (p ^ k)
  ↪ = k + 1

```

Proof. The divisors are exactly $1, p, \dots, p^k$, and these powers are distinct.

```

classical
rw [tau, divisors_prime_pow p k hp, Finset.card_image_of_injective,
  Finset.card_range]

exact pow_right_injective (by have := hp.1; omega : 0 < p) (by have :=
  ↪ hp.1; omega : p ≠ 1)

```

□

Theorem 5.1.17. *If p is prime and $k \geq 0$, then $\sigma(p^k) = 1 + p + \cdots + p^k$.*

```
theorem sigma_prime_pow (p k : ℕ) (hp : Chapter02.isPrime p) :
  sigma (p ^ k) = ∑ j ∈ Finset.range (k + 1), p ^ j
```

Proof. Sum the elements of the prime-power divisor list.

```
classical
rw [sigma, divisors_prime_pow p k hp, Finset.sum_image]
intro i hi j hj h
exact pow_right_injective₀ (by have := hp.1; omega : 0 < p) (by have :=
  ↪ hp.1; omega : p ≠ 1) h
```

□

Theorem 5.1.18. *For $n > 1$,*

$$\sigma(n) = \prod_{p|n} (1 + p + \cdots + p^{e_n(p)}).$$

```
theorem sigma_eq_prod (n : ℕ) (hn : 1 < n) :
  (sigma n : ℤ) = ∏ p ∈ (primeExponents n).support,
    ((∑ j ∈ Finset.range (primeExponents n p + 1), p ^ j : ℕ) : ℤ)
```

Proof. Apply the prime-power formula for multiplicative functions to σ , then substitute the geometric sum for each prime-power value.

```
rw [multiplicative_prime_powers _ sigma_multiplicative n hn]
apply Finset.prod_congr rfl
intro p hp
rw [sigma_prime_pow p _ ((primeExponents_spec n (by omega)).1 p hp)]
```

□

Theorem 5.1.19. *For $n > 1$,*

$$\sigma(n) = \prod_{p|n} \frac{p^{e_n(p)+1} - 1}{p - 1}.$$

Each quotient is an integer.

```
theorem sigma_eq_prod_quotients (n : ℕ) (hn : 1 < n) :
  (sigma n : ℤ) = ∏ p ∈ (primeExponents n).support,
    (((p ^ (primeExponents n p + 1) - 1) / (p - 1) : ℕ) : ℤ)
```

Proof. The geometric identity $(p-1)(1+p+\cdots+p^k) = p^{k+1} - 1$ follows by induction. Replace each geometric sum in the product by this exact quotient.

```
rw [sigma_eq_prod n hn]
apply Finset.prod_congr rfl
intro p hp

rw [geometric_sum_eq_div p _ ((primeExponents_spec n (by omega)).1 p
  ↪ hp).1]
```

□

Theorem 5.1.20. *One has $\tau(1) = \sigma(1) = 1$.*

```
theorem tau_sigma_one : tau 1 = 1 ∧ sigma 1 = 1
```

Proof. The only positive divisor of one is one.

```
simp only [tau, sigma, Nat.divisors_one, Finset.card_singleton,
  ↪ Finset.sum_singleton, and_self]
```

□

Theorem 5.1.21. *For $n > 1$, one has $\tau(n) = 2$ exactly when n is prime.*

```
theorem tau_eq_two_iff (n : ℕ) (hn : 1 < n) : tau n = 2 ↔
  ↪ Chapter02.isPrime n
```

Proof. The positive divisors include the distinct elements one and n .

```

have hnpos : 0 < n := by
  omega

have hne : n ≠ 0 := ne_of_gt hnpos

have hsub : ({1, n} : Finset ℕ) ⊆ n.divisors := by
  intro d hd
  simp only [Finset.mem_insert, Finset.mem_singleton] at hd
  rcases hd with he | he

  · rw [he]
    exact Nat.one_mem_divisors.mpr (ne_of_gt hnpos)

  · rw [he]
    exact Nat.mem_divisors_self n (ne_of_gt hnpos)

have hcard : ({1, n} : Finset ℕ).card = 2 := by
  rw [Finset.card_insert_of_notMem, Finset.card_singleton]
  simp only [Finset.mem_singleton] using (by omega : (1 : ℕ) ≠ n)

```

If there are only two divisors, these are all of them, so n is prime.

```

constructor

· intro ht

have heq : n.divisors = {1, n} :=
  (Finset.eq_of_subset_of_card_le hsub (by
    change tau n ≤ _
    rw [ht, hcard])).symm

refine ⟨hn, ?_⟩
intro d hd

have hmem := Nat.mem_divisors.mpr ⟨hd, ne_of_gt hnpos⟩
rw [heq, Finset.mem_insert, Finset.mem_singleton] at hmem
exact hmem

```

Conversely, primality says that one and n are exactly the divisors. Their number is two.

```

· intro hp

have heq : n.divisors = {1, n} := by
  apply Finset.Subset.antisymm _ hsub
  intro d hd
  exact Finset.mem_insert.mpr ((hp.2 d (Nat.dvd_of_mem_divisors
    ↪ hd)).imp_right

```

```

    Finset.mem_singleton.mpr)

rw [tau, heq, hcard]

```

□

Theorem 5.1.22. *For $n > 1$, one has $\sigma(n) = n + 1$ exactly when n is prime.*

```

theorem sigma_eq_succ_iff (n : ℕ) (hn : 1 < n) : sigma n = n + 1 ↔
  ↔ Chapter02.isPrime n

```

Proof. One and n already contribute $n + 1$ to the divisor sum. If the sum equals $n + 1$, consider any divisor other than these two.

```

have hnpos : 0 < n := by
  omega

have hnne : n ≠ 0 := ne_of_gt hnpos

constructor

· intro hs
  refine ⟨hn, ?_⟩
  intro d hd
  by_cases hd1 : d = 1

  · exact 0r.inl hd1

  · by_cases hdn : d = n

    · exact 0r.inr hdn

    · have hmem := Nat.mem_divisors.mpr ⟨hd, ne_of_gt hnpos⟩

      have hdpos := Nat.pos_of_mem_divisors hmem

      have hsub : ({d, 1, n} : Finset ℕ) ⊆ n.divisors := by
        intro k hk
        simp only [Finset.mem_insert, Finset.mem_singleton] at hk
        rcases hk with he | he | he

        · rwa [he]

        · rw [he]

```

```

    exact Nat.one_mem_divisors.mpr (ne_of_gt hnpos)

  · rw [he]
    exact Nat.mem_divisors_self n (ne_of_gt hnpos)

```

This further divisor is positive. Its contribution would make the divisor sum strictly greater than $n + 1$, a contradiction.

```

have hle := Finset.sum_le_sum_of_subset hsub (f := fun k => k)
rw [Finset.sum_insert (by simp only [Finset.mem_insert,
  ↪ Finset.mem_singleton,
  hd1, hdn, or_self, not_false_eq_true]),
  Finset.sum_insert (by simp only [Finset.mem_singleton] using
    (by omega : (1 : ℕ) ≠ n)), Finset.sum_singleton] at hle
change d + (1 + n) ≤ sigma n at hle
omega

```

Conversely, a prime has exactly the two divisors one and n . Their sum is $n + 1$.

```

  · intro hp

  have heq : n.divisors = {1, n} := by
    ext d
    constructor

    · intro hd
      exact Finset.mem_insert.mpr ((hp.2 d (Nat.dvd_of_mem_divisors
        ↪ hd)).imp_right
        Finset.mem_singleton.mpr)

    · intro hd
      rcases Finset.mem_insert.mp hd with he | he

      · rw [he]
        exact Nat.one_mem_divisors.mpr hnne

      · rw [Finset.mem_singleton.mp he]
        exact Nat.mem_divisors_self n hnne

  rw [sigma, heq, Finset.sum_insert (by simp only
    ↪ [Finset.mem_singleton] using
      (by omega : (1 : ℕ) ≠ n)), Finset.sum_singleton]
  omega

```

□

Theorem 5.1.23. *For positive n , the product of its divisors satisfies*

$$\left(\prod_{d|n} d \right)^2 = n^{\tau(n)}.$$

Equivalently, as positive real numbers, the product is $n^{\tau(n)/2}$; the squared form avoids truncating the half-exponent.

```
theorem prod_divisors_sq (n : ℕ) (hn : 0 < n) :
  (∏ d ∈ n.divisors, d) ^ 2 = n ^ tau n
```

Proof. The map $d \mapsto n/d$ permutes the positive divisors, so it preserves their product.

```
have hperm : (∏ d ∈ n.divisors, n / d) = ∏ d ∈ n.divisors, d := by
  apply Finset.prod_bij (fun d _ => n / d)

  · intro d hd
    exact Nat.mem_divisors.mpr
      (Nat.div_dvd_of_dvd (Nat.dvd_of_mem_divisors hd), ne_of_gt hn)

  · intro d hd e he hde

    have h := congrArg (fun k => n / k) hde
    rwa [Nat.div_div_self (Nat.dvd_of_mem_divisors hd) (ne_of_gt hn),
      Nat.div_div_self (Nat.dvd_of_mem_divisors he) (ne_of_gt hn)] at h

  · intro d hd
    refine ⟨n / d, Nat.mem_divisors.mpr
      (Nat.div_dvd_of_dvd (Nat.dvd_of_mem_divisors hd), ne_of_gt hn),
      ↪ ?_⟩

    exact Nat.div_div_self (Nat.dvd_of_mem_divisors hd) (ne_of_gt hn)

  · intro d hd
    rfl
```

Multiply the original product by this permuted product. Each paired factor is $d(n/d) = n$, giving one factor n for each divisor.

```

rw [pow_two]
nth_rw 2 [← hperm]
rw [← Finset.prod_mul_distrib]

have hterm : ∀ d ∈ n.divisors, d * (n / d) = n := by
  intro d hd
  exact Nat.mul_div_cancel' (Nat.dvd_of_mem_divisors hd)

rw [Finset.prod_congr rfl hterm, Finset.prod_const]
rfl

```

□

5.2 The Möbius Inversion Formula

Definition 5.2.1. The local contribution of a prime exponent is $w(0) = 1$, $w(1) = -1$, and $w(k) = 0$ for $k \geq 2$.

```

def mobiusWeight (k : ℕ) : ℤ := if k = 0 then 1 else if k = 1 then -1
  ↪ else 0

```

Definition 5.2.2. Define the Möbius function for $n > 0$ by multiplying $w(e_n(p))$ over the prime support. Thus $\mu(1) = 1$, a prime square forces value zero, and a product of r distinct primes has value $(-1)^r$. Set $\mu(0) = 0$ in the total definition.

```

noncomputable def mobius (n : ℕ) : ℤ :=
  if n = 0 then 0 else (primeExponents n).prod (fun _ k => mobiusWeight
    ↪ k)

```

Theorem 5.2.3. The Möbius function has $\mu(1) = 1$.

```

theorem mobius_one : mobius 1 = 1

```

Proof. The prime support of one is empty, so the defining product is one.

```

simp only [mobius, one_ne_zero, ↓reduceIte, primeExponents_one,
  ↪ Finsupp.prod_zero_index]

```

□

Theorem 5.2.4. For a prime p , $\mu(p^k) = w(k)$: the values are one for $k = 0$, minus one for $k = 1$, and zero thereafter.

```
theorem mobius_prime_pow (p k : ℕ) (hp : Chapter02.isPrime p) :
  mobius (p ^ k) = mobiusWeight k
```

Proof. The exponent vector of p^k has the single coordinate k at p . Evaluate the defining product.

```
have hpos : 0 < p ^ k := pow_pos (by
  have := hp.1
  omega) k
rw [mobius, ite_eq_right (ne_of_gt hpos), primeExponents_prime_pow p k
  ↪ hp]
exact Finsupp.prod_single_index (by simp only [mobiusWeight,
  ↪ ↓reduceIte])
```

□

Theorem 5.2.5. If $n > 0$ and $p^2 \mid n$ for a prime p , then $\mu(n) = 0$.

```
theorem mobius_eq_zero_of_prime_sq_dvd (n p : ℕ) (hn : 0 < n)
  (hp : Chapter02.isPrime p) (hdiv : p ^ 2 ∣ n) : mobius n = 0
```

Proof. The divisibility criterion gives $e_n(p) \geq 2$, so p belongs to the prime support.

```
have hle := (dvd_iff_primeExponents_le (p ^ 2) n
  (pow_pos (by
    have := hp.1
    omega) 2) hn).1 hdiv
rw [primeExponents_prime_pow p 2 hp, Finsupp.single_le_iff] at hle
rw [mobius, ite_eq_right (ne_of_gt hn), Finsupp.prod]

have hmem : p ∈ (primeExponents n).support :=
  Finsupp.mem_support_iff.mpr (by omega)
```

The contribution at p is zero. Therefore the whole defining product for $\mu(n)$ vanishes.

```

apply Finset.prod_eq_zero hmem
simp only [mobiusWeight, ite_eq_right (by omega : primeExponents n p ≠
  ↪ 0),
  ite_eq_right (by omega : primeExponents n p ≠ 1)]

```

□

Theorem 5.2.6. *If positive n has no prime-square divisor and has r distinct prime factors, then $\mu(n) = (-1)^r$.*

```

theorem mobius_eq_sign (n : ℕ) (hn : 0 < n)
  (hsq : ∀ p : ℕ, Chapter02.isPrime p → ¬p ^ 2 ∣ n) :
  mobius n = (-1 : ℤ) ^ (primeExponents n).support.card

```

Proof. Every exponent in the prime support is positive. The absence of prime-square divisors makes it less than two, hence equal to one. Each corresponding factor is therefore minus one.

```

rw [mobius, ite_eq_right (ne_of_gt hn), Finsupp.prod]

have hweight : ∀ p ∈ (primeExponents n).support, mobiusWeight
  ↪ (primeExponents n p) = -1 := by
  intro p hps

  have hp := (primeExponents_spec n hn).1 p hps

  have hpos := Nat.pos_of_ne_zero (Finsupp.mem_support_iff.mp hps)

  have hlt : primeExponents n p < 2 := by
    by_contra hh
    apply hsq p hp
    apply (dvd_iff_primeExponents_le _ n (pow_pos (by
      have := hp.1
      omega) 2) hn).2
  rw [primeExponents_prime_pow p 2 hp, Finsupp.single_le_iff]
  omega

have he : primeExponents n p = 1 := by
  omega
simp only [he, mobiusWeight, one_ne_zero, ↓reduceIte]

```

The defining product is consequently minus one raised to the number of distinct prime factors.

```
rw [Finset.prod_congr rfl hweight, Finset.prod_const]
```

□

Theorem 5.2.7. *The Möbius function is multiplicative.*

```
theorem mobius_multiplicative : IsMultiplicative mobius
```

Proof. Coprime integers have disjoint prime supports: a common prime would divide their gcd, one. Multiplication adds exponent vectors, so the defining product splits into the two disjoint products.

```
intro m n hm hn hcop
rw [mobius, mobius, mobius, ite_eq_right (ne_of_gt (mul_pos hm hn)),
  ite_eq_right (ne_of_gt hm), ite_eq_right (ne_of_gt hn),
  ↪ primeExponents_mul m n hm hn]

exact Finsupp.prod_add_index_of_disjoint (prime_support_disjoint m n hm
  ↪ hn hcop) _
```

□

Theorem 5.2.8. *For positive n ,*

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & n = 1, \\ 0 & n > 1. \end{cases}$$

```
theorem sum_mobius (n : ℕ) (hn : 0 < n) :
  divisorSum mobius n = if n = 1 then 1 else 0
```

Proof. At one the sole summand is $\mu(1) = 1$.

```
by_cases he : n = 1

· subst n
  simp only [divisorSum, Nat.divisors_one, Finset.sum_singleton,
    ↪ mobius_one, ↓reduceIte]
```

For $n > 1$, the divisor sum of μ is multiplicative, so it is the product of its values on the prime powers. The prime support is nonempty.

```

· rw [ite_eq_right he, multiplicative_prime_powers _
  (divisorSum_multiplicative _ mobius_multiplicative) n (by omega)]

have hs : (primeExponents n).support.Nonempty := by
  by_contra h

have hh := (primeExponents_spec n hn).2
change (∏ p ∈ (primeExponents n).support, p ^ primeExponents n p) =
  ↪ n at hh
rw [Finset.not_nonempty_iff_eq_empty.mp h, Finset.prod_empty] at hh
exact he hh.symm

```

Choose a prime in this support. The divisor sum on its positive power is $1 - 1 + 0 + \dots + 0 = 0$, so the entire product vanishes.

```

obtain ⟨p, hp⟩ := hs
apply Finset.prod_eq_zero hp
exact divisorSum_mobius_prime_pow p _ ((primeExponents_spec n hn).1 p
  ↪ hp)
(Nat.pos_of_ne_zero (Finsupp.mem_support_iff.mp hp))

```

□

Theorem 5.2.9. *Let $F(n) = \sum_{d|n} f(d)$ for every positive n . Then*

$$f(n) = \sum_{d|n} \mu(d) F(n/d).$$

```

theorem mobius_inversion (f F : ℕ → ℤ)
  (hF : ∀ n : ℕ, 0 < n → F n = divisorSum f n) (n : ℕ) (hn : 0 < n) :
  f n = ∑ d ∈ n.divisors, mobius d * F (n / d)

```

Proof. Extend the functions by zero at zero. Dirichlet convolution is the finite sum over pairs with product n , and multiplication by the constant-one arithmetic function is divisor summation.

```

have hF' : arithmeticFunction F = arithmeticFunction (divisorSum f) :=
  ↪ by
    ext k
    by_cases hk : k = 0

    · subst k
      rfl

    · rw [arithmeticFunction_apply _ k (Nat.pos_of_ne_zero hk),
          arithmeticFunction_apply _ k (Nat.pos_of_ne_zero hk), hF k
          ↪ (Nat.pos_of_ne_zero hk)]

```

The proved cancellation identity says that convolution of μ with the constant-one function is the convolution identity. Associativity is just regrouping the finite sum over triples with product n . Therefore $\mu * F = \mu * (f * 1) = f$. Expanding at n gives the stated formula.

```

have hconv : arithmeticFunction mobius * arithmeticFunction F =
  ↪ arithmeticFunction f := by
    rw [hF', arithmeticFunction_divisorSum, mul_left_comm,
        ↪ mobius_mul_zeta, mul_one]

have h := congrArg (fun g : ArithmeticFunction  $\mathbb{Z} \Rightarrow g\ n$ ) hconv
  rw [convolution_apply _ _ n hn, arithmeticFunction_apply f n hn] at h

exact h.symm

```

□

Theorem 5.2.10. *Under the same hypotheses,*

$$f(n) = \sum_{d|n} \mu(n/d) F(d).$$

```

theorem mobius_inversion' (f F :  $\mathbb{N} \rightarrow \mathbb{Z}$ )
  (hF :  $\forall n : \mathbb{N}, 0 < n \rightarrow F\ n = \text{divisorSum } f\ n$ ) (n :  $\mathbb{N}$ ) (hn :  $0 < n$ ) :
  f n =  $\sum d \in n.\text{divisors}, \text{mobius } (n / d) * F\ d$ 

```

Proof. Interchange the two factors in the convolution, or equivalently permute the divisors by $d \mapsto n/d$.

```

rw [mobius_inversion f F hF n hn, ← convolution_apply _ _ n hn,
    ↪ mul_comm,
    convolution_apply _ _ n hn]

apply Finset.sum_congr rfl
intro d hd
exact mul_comm _ _

```

□

Theorem 5.2.11. *If the divisor sum of f is multiplicative, then f is multiplicative.*

```

theorem multiplicative_of_divisorSum (f : ℕ → ℤ)
  (hf : IsMultiplicative (divisorSum f)) : IsMultiplicative f

```

Proof. The convolution of μ with the multiplicative function given by the divisor sum is multiplicative. The convolution lemma proves this by splitting divisors of a coprime product and factoring the double sum.

```

have hconv := convolution_multiplicative mobius (divisorSum f)
    ↪ mobius_multiplicative hf

```

Möbius inversion identifies this convolution with f . Substitute the inversion formula into its multiplicativity identity to obtain multiplicativity of f .

```

intro m n hm hn hcop

have hh := hconv m n hm hn hcop
dsimp only at hh

rw [mobius_inversion f (divisorSum f) (fun _ _ => rfl) (m * n) (mul_pos
    ↪ hm hn),
    hh,
    ← mobius_inversion f (divisorSum f) (fun _ _ => rfl) m hm,
    ← mobius_inversion f (divisorSum f) (fun _ _ => rfl) n hn]

```

□

Theorem 5.2.12. *An arithmetic function is multiplicative if and only if its divisor sum is multiplicative.*

```
theorem multiplicative_iff_divisorSum (f : ℕ → ℤ) :
  IsMultiplicative f ↔ IsMultiplicative (divisorSum f)
```

Proof. Combine preservation of multiplicativity with the converse just proved.

```
(divisorSum_multiplicative f, multiplicative_of_divisorSum f)
```

□

Definition 5.2.13. The summatory Möbius function is $M(N) = \sum_{k=1}^N \mu(k)$. It is the number of square-free integers in this interval with an even number of prime factors, minus the number with an odd number; one belongs to the even group.

```
noncomputable def mertens (N : ℕ) : ℤ := ∑ k ∈ Finset.Icc 1 N, mobius k
```

5.3 The Greatest Integer Function

Theorem 5.3.1. For every real x , its greatest integer satisfies $\lfloor x \rfloor \leq x < \lfloor x \rfloor + 1$. In particular, $x = \lfloor x \rfloor + \theta$ for a real $0 \leq \theta < 1$.

```
theorem floor_spec (x : ℝ) :
  (⌊x⌋ : ℝ) ≤ x ∧ x < (⌊x⌋ : ℝ) + 1 ∧
  ∃ θ : ℝ, 0 ≤ θ ∧ θ < 1 ∧ x = ⌊x⌋ + θ
```

Proof. The defining floor bounds hold, and $\theta = x - \lfloor x \rfloor$ gives the decomposition. Its bounds follow by subtraction; its value is forced by the displayed equality.

```
have hlo := Int.floor_le x

have hhi := Int.lt_floor_add_one x

exact (hlo, hhi, x - ⌊x⌋, by linarith, by linarith, by ring)
```

□

Theorem 5.3.2. A real number equals its floor exactly when it is an integer.

```
theorem floor_eq_self_iff (x : ℝ) : (⌊x⌋ : ℝ) = x ↔ ∃ z : ℤ, x = z
```

Proof. Equality with the floor supplies the integer witness. Conversely, the floor of an integer is that integer.

```
constructor

· intro h
  exact (⌊x⌋, h.symm)

· rintro (z, rfl)
  rw [Int.floor_intCast]
```

□

Theorem 5.3.3. For natural N and positive k , the integer quotient is $N/k = \lfloor N/k \rfloor$, where the quotient inside the floor is taken in the real numbers.

```
theorem floor_nat_div (N k : ℕ) (hk : 0 < k) :
  ⌊(N : ℝ) / k⌋ = ((N / k : ℕ) : ℤ)
```

Proof. Write $N = kq + r$ with $0 \leq r < k$. Then $q \leq N/k$.

```
have hkreal : (0 : ℝ) < k := by
  exact_mod_cast hk
have heq := Nat.mod_add_div N k
have hlt := Nat.mod_lt N hk

apply Int.floor_eq_iff.mpr
constructor

· apply (le_div_iff₀ hkreal).mpr
  exact_mod_cast Nat.div_mul_le_self N k
```

The strict bound $r < k$ gives $N/k < q + 1$. The two inequalities identify the floor with q .

```

· apply (div_lt_iff hkreal).mpr
  simp only [Int.cast_natCast]

  have hreal : (N % k : ℕ) + (k : ℝ) * (N / k : ℕ) = N := by
    exact_mod_cast heq
  have hrlt : ((N % k : ℕ) : ℝ) < k := by
    exact_mod_cast hlt
  linarith

```

□

Theorem 5.3.4. *The number of multiples of positive k in $\{1, \dots, N\}$ is $\lfloor N/k \rfloor$.*

```

theorem card_multiples (N k : ℕ) (hk : 0 < k) :
  ((Finset.Icc 1 N).filter (fun n => k ∣ n)).card = N / k

```

Proof. Send a positive multiple $n \leq N$ to n/k . This quotient lies between one and $\lfloor N/k \rfloor$.

```

classical

have hcard : ((Finset.Icc 1 N).filter (fun n => k ∣ n)).card =
  (Finset.Icc 1 (N / k)).card := by

  apply Finset.card_bij (fun n _ => n / k)

  · intro n hn

    obtain ⟨hrange, hdiv⟩ := Finset.mem_filter.mp hn
    obtain ⟨hnpos, hnle⟩ := Finset.mem_Icc.mp hrange

    apply Finset.mem_Icc.mpr
    exact ⟨Nat.div_pos (Nat.le_of_dvd hnpos hdiv) hk,
      ↪ Nat.div_le_div_right hnle⟩

```

Two such multiples with equal quotients are equal, since multiplication by k recovers each multiple.

```

· intro m hm n hn heq

have hmd := (Finset.mem_filter.mp hm).2

have hnd := (Finset.mem_filter.mp hn).2

have h := congrArg (fun q => q * k) heq
  rwa [Nat.div_mul_cancel hmd, Nat.div_mul_cancel hnd] at h

```

Conversely, for every $1 \leq j \leq \lfloor N/k \rfloor$, the integer jk is a positive multiple at most N and has quotient j .

```

· intro j hj

obtain (hjpos, hjle) := Finset.mem_Icc.mp hj

refine (j * k, ?_, Nat.mul_div_cancel j hk)

apply Finset.mem_filter.mpr
constructor

· apply Finset.mem_Icc.mpr
  exact ⟨mul_pos hjpos hk, (Nat.le_div_iff_mul_le hk).1 hjle⟩

· exact ⟨j, by ring⟩

```

This bijection identifies the number of multiples with the number of integers from one to $\lfloor N/k \rfloor$, which is $\lfloor N/k \rfloor$.

```

rw [hcard, Nat.card_Icc, Nat.add_sub_cancel]

```

□

Theorem 5.3.5. *For a prime p and natural n , the exponent of p in $n!$ is*

$$v_p(n!) = \sum_{j=1}^n \left\lfloor \frac{n}{p^j} \right\rfloor.$$

Larger indices contribute zero, so this is also the usual finite sum over $j \geq 1$.

```

theorem legendre (n p : ℕ) (hp : Chapter02.isPrime p) :
  primeExponents n.factorial p = ∑ j ∈ Finset.Icc 1 n, n / p ^ j

```

Proof. In the checked bounded formula, sum $v_p(k)$ over $1 \leq k \leq n$. The exponent $v_p(k)$ counts the positive powers of p dividing k . Interchange these finite sums. For each j , the inner count is the number of multiples of p^j up to n , namely $\lfloor n/p^j \rfloor$. The bound $v_p(k) \leq k$ justifies truncation.

```
legendre_with_bound n p n hp le_rfl
```

□

Theorem 5.3.6. *For real a, b , the difference $\lfloor a + b \rfloor - \lfloor a \rfloor - \lfloor b \rfloor$ is either zero or one.*

```
theorem floor_add_carry (a b : ℝ) :  
  ⌊a + b⌋ - ⌊a⌋ - ⌊b⌋ = 0 ∨ ⌊a + b⌋ - ⌊a⌋ - ⌊b⌋ = 1
```

Proof. Add the floor bounds for a and b and compare them with those for $a + b$. The difference $\lfloor a + b \rfloor - \lfloor a \rfloor - \lfloor b \rfloor$ is greater than minus one and less than two.

```
have ha := floor_spec a  
  
have hb := floor_spec b  
  
have hab := floor_spec (a + b)  
  
have hlow : (⌊a⌋ : ℝ) + ⌊b⌋ - 1 < ⌊a + b⌋ := by  
  linarith  
  
have hhigh : (⌊a + b⌋ : ℝ) < ⌊a⌋ + ⌊b⌋ + 2 := by  
  linarith
```

These bounds also hold as inequalities between integers. The only possible integer differences are zero and one.

```
have hlow' : ⌊a⌋ + ⌊b⌋ - 1 < ⌊a + b⌋ := by  
  exact_mod_cast hlow  
  
have hhigh' : ⌊a + b⌋ < ⌊a⌋ + ⌊b⌋ + 2 := by  
  exact_mod_cast hhigh  
  
omega
```

□

Theorem 5.3.7. *For all real a, b , $\lfloor a + b \rfloor \geq \lfloor a \rfloor + \lfloor b \rfloor$.*

```
theorem floor_add_ge (a b : ℝ) : ⌊a⌋ + ⌊b⌋ ≤ ⌊a + b⌋
```

Proof. The difference is zero or one, hence is nonnegative.

```
rcases floor_add_carry a b with h | h <=> omega
```

□

Theorem 5.3.8. *For natural $r \leq n$, the factorial quotient $n!/(r!(n-r)!)$ is an integer. This includes the endpoint cases $r = 0, n$, when the quotient is one.*

```
theorem factorials_dvd_factorial (n r : ℕ) (hr : r ≤ n) :  
  r.factorial * (n - r).factorial ∣ n.factorial
```

Proof. It suffices to compare every prime exponent in the denominator and numerator. A zero exponent in the denominator needs no further argument.

```
apply (dvd_iff_primeExponents_le _ _  
  (mul_pos (Nat.factorial_pos r) (Nat.factorial_pos (n - r)))  
  ↪ (Nat.factorial_pos n)).2  
  
intro p  
by_cases he : primeExponents (r.factorial * (n - r).factorial) p = 0  
  
· rw [he]  
  exact Nat.zero_le _
```

For a nonzero exponent, the corresponding p is prime. Legendre's formula with a common upper bound reduces the comparison to a sum of quotient inequalities.

```
· have hp := (primeExponents_spec (r.factorial * (n - r).factorial)  
  (mul_pos (Nat.factorial_pos r) (Nat.factorial_pos (n - r)))) 1 p  
  (Finsupp.mem_support_iff.mpr he)  
  
rw [primeExponents_mul _ _ (Nat.factorial_pos r) (Nat.factorial_pos (n  
  ↪ - r)),  
  Finsupp.add_apply, legendre_with_bound r p n hp hr,  
  legendre_with_bound (n - r) p n hp (Nat.sub_le n r), legendre n p  
  ↪ hp,  
  ← Finset.sum_add_distrib]
```

```

apply Finset.sum_le_sum
intro j hj

```

For each j , multiplying the quotients by p^j gives bounds by r and $n - r$. Their sum is at most n , so the sum of the quotients is at most $\lfloor n/p^j \rfloor$. Summing proves the required exponent inequality.

```

have hpos : 0 < p ^ j := pow_pos (by
  have := hp.1
  omega) j

apply (Nat.le_div_iff_mul_le hpos).2

have h1 := Nat.div_mul_le_self r (p ^ j)

have h2 := Nat.div_mul_le_self (n - r) (p ^ j)

have hsum : r + (n - r) = n := Nat.add_sub_cancel' hr

nlinarith

```

□

Theorem 5.3.9. *The product of any r consecutive positive integers is divisible by $r!$. More precisely, $r! \mid (a + 1)(a + 2) \cdots (a + r)$ for natural a, r .*

```

theorem factorial_dvd_consecutive_product (a r : ℕ) :
  r.factorial ∣ ∏ k ∈ Finset.Ico (a + 1) (a + r + 1), k

```

Proof. Multiplication of the displayed product by $a!$ gives $(a + r)!$, as follows by induction on r .

```

have hfactor : (∏ k ∈ Finset.Ico (a + 1) (a + r + 1), k) * a.factorial =
  (a + r).factorial := by
  induction r with
  | zero => simp only [Nat.add_zero, Finset.Ico_self, Finset.prod_empty,
    ↪ one_mul]

  | succ r ih =>
    rw [show a + (r + 1) + 1 = (a + r + 1) + 1 by omega,
      Finset.prod_Ico_succ_top (by omega : a + 1 ≤ a + r + 1)]
    rw [mul_right_comm, ih]
    rw [show a + (r + 1) = (a + r) + 1 by omega, Nat.factorial_succ]

```

```
ring
```

The preceding theorem gives $r!a! \mid (a+r)!$. Cancel the positive factor $a!$ to obtain the claimed divisibility.

```
obtain ⟨k, hk⟩ := factorials_dvd_factorial (a + r) r (by omega)
rw [show a + r - r = a by omega] at hk

refine ⟨k, ?_⟩
apply Nat.eq_of_mul_eq_mul_right (Nat.factorial_pos a)
rw [hfactor, hk]
ring
```

□

Theorem 5.3.10. For $F(n) = \sum_{d|n} f(d)$ and every natural N ,

$$\sum_{n=1}^N F(n) = \sum_{k=1}^N f(k) \left\lfloor \frac{N}{k} \right\rfloor.$$

No multiplicativity assumption is needed.

```
theorem sum_divisorSum (f : ℕ → ℤ) (N : ℕ) :
  (∑ n ∈ Finset.Icc 1 N, divisorSum f n) =
    ∑ k ∈ Finset.Icc 1 N, f k * (N / k : ℕ)
```

Proof. Embed every divisor sum in the common index interval $1, \dots, N$, assigning zero to nondivisors. Interchange the finite sums.

```
classical

have hexpand (n : ℕ) (hn : n ∈ Finset.Icc 1 N) :
  divisorSum f n = ∑ d ∈ Finset.Icc 1 N, if d | n then f d else 0 :=
  ↪ by
obtain ⟨hnpos, hnle⟩ := Finset.mem_Icc.mp hn

have hset : n.divisors = (Finset.Icc 1 N).filter (fun d => d | n) :=
  ↪ by
  ext d
  constructor
```

```

• intro hd
  exact Finset.mem_filter.mpr
    (Finset.mem_Icc.mpr (Nat.pos_of_mem_divisors hd,
      (Nat.le_of_dvd hnpos (Nat.dvd_of_mem_divisors hd)).trans
        ↪ hnle),
      Nat.dvd_of_mem_divisors hd)

• intro hd
  exact Nat.mem_divisors.mpr ((Finset.mem_filter.mp hd).2, ne_of_gt
    ↪ hnpos)

rw [divisorSum, hset, Finset.sum_filter]

```

For a fixed k , the term $f(k)$ occurs once for each multiple of k up to N . The multiple-counting theorem makes this number $\lfloor N/k \rfloor$.

```

rw [Finset.sum_congr rfl hexpand, Finset.sum_comm]
apply Finset.sum_congr rfl
intro k hk

rw [← Finset.sum_filter, Finset.sum_const, nsmul_eq_mul,
  card_multiples N k (Finset.mem_Icc.mp hk).1, mul_comm]

```

□

Theorem 5.3.11. *For natural N ,*

$$\sum_{n=1}^N \tau(n) = \sum_{k=1}^N \left\lfloor \frac{N}{k} \right\rfloor.$$

```

theorem sum_tau (N : ℕ) :
  (∑ n ∈ Finset.Icc 1 N, (tau n : ℤ)) = ∑ k ∈ Finset.Icc 1 N, (N / k :
    ↪ ℕ)

```

Proof. Apply the summatory divisor-sum identity to the constant function one.

```

have h := sum_divisorSum (fun _ => 1) N

simpa only [divisorSum, Finset.sum_const, nsmul_eq_mul, mul_one,
  ↪ one_mul, tau, Nat.cast_sum] using h

```

□

Theorem 5.3.12. *For natural N ,*

$$\sum_{n=1}^N \sigma(n) = \sum_{k=1}^N k \left\lfloor \frac{N}{k} \right\rfloor.$$

```
theorem sum_sigma (N : ℕ) :
  (∑ n ∈ Finset.Icc 1 N, (sigma n : ℤ)) =
    ∑ k ∈ Finset.Icc 1 N, (k : ℤ) * (N / k : ℕ)
```

Proof. Apply the same identity to $f(k) = k$.

```
have h := sum_divisorSum (fun k => (k : ℤ)) N
simp only [divisorSum, sigma, Nat.cast_sum] using h
```

□

5.4 An Application to the Calendar

Chapter 6

Euler's Generalization of Fermat's Theorem

6.1 Euler's Phi-Function

6.2 Euler's Theorem

6.3 Some Properties of the Phi-Function

Chapter 7

Primitive Roots and Indices

7.1 The Order of an Integer Modulo n

7.2 Primitive Roots for Primes

7.3 Composite Numbers Having Primitive Roots

7.4 The Theory of Indices

Chapter 8

The Quadratic Reciprocity Law

8.1 Euler's Criterion

8.2 The Legendre Symbol and Its Properties

8.3 Quadratic Reciprocity

8.4 Quadratic Congruences with Composite Moduli

Chapter 9

Introduction to Cryptography

9.1 From Caesar Cipher to Public Key Cryptography

9.2 The Knapsack Cryptosystem

9.3 An Application of Primitive Roots to Cryptography

Chapter 10

Numbers of Special Form

10.1 Perfect Numbers

10.2 Mersenne Primes and Amicable Numbers

10.3 Fermat Numbers

Chapter 11

Certain Nonlinear Diophantine Equations

11.1 The Equation $x^2 + y^2 = z^2$

11.2 Fermat's Last Theorem

Chapter 12

Representation of Integers as Sums of Squares

12.1 Sums of Two Squares

12.2 Sums of More Than Two Squares

Chapter 13

Fibonacci Numbers

13.1 The Fibonacci Sequence

13.2 Certain Identities Involving Fibonacci Numbers

Chapter 14

Continued Fractions

14.1 Finite Continued Fractions

14.2 Infinite Continued Fractions

14.3 Farey Fractions

14.4 Pell's Equation

Chapter 15

Some Modern Developments

15.1 Primality Testing and Factorization

15.2 An Application to Factoring: Remote Coin Flipping

15.3 The Prime Number Theorem and Zeta Function

Bibliography

- [1] David M. Burton, *Elementary Number Theory*, McGraw-Hill International Editions.